SOFTWARE REQUIREMENTS SPECIFICATION

Cinema — Mobile

Version 3.0

2026-10-28

Table 1: Document metadata

Item	Value
Version	3.0
Baseline date	2026-10-28 (Asia/Saigon)
Language	English
Evidence status	Static code/document baseline; runtime verification not performed
Paired document	`Cinema_Mobile_SRS_VI.md`

Contents

1 Introduction 1

2 Purpose 2

3 Scope 3

4 Overall Description 4

5 Product Perspective 5

6 User Classes / Actors 6

7 Role & Permission 7

8 Business Rules 8

9 Functional Requirements 10

10 Use Case Specifications 64

11 External Interface Requirements 69

12 UI / Screen Requirements 70

13 API Requirements 78

14 Data Requirements 84

15 Business Workflows 85

16 Authentication & Authorization 86

17 Error Handling 87

18 Non-Functional Requirements 88

19 Security Requirements 90

20 Testing Requirements 91

21 Acceptance Criteria 93

22 Requirements Traceability Matrix 94

23 Constraints **117**

24 Out of Scope **118**

25 Future Enhancements **119**

26 Conclusion **120**

27 Implemented vs Requirement **121**

28 Evidence and source reconciliation **126**

List of Tables

1 Document metadata i
2 5. Product Perspective 5
3 6. User Classes / Actors 6
4 7. Role & Permission 7
5 8. Business Rules 8
6 `MOB-FR-AUTH-01` — Register 10
7 `MOB-FR-AUTH-02` — Sign in 10
8 `MOB-FR-AUTH-03` — `JWT` claims 11
9 `MOB-FR-AUTH-04` — Validate `JWT` 12
10 `MOB-FR-AUTH-05` — Authorize roles 13
11 `MOB-FR-AUTH-06` — Unauthenticated and forbidden responses 13
12 `MOB-FR-AUTH-07` — Verify email `OTP` 14
13 `MOB-FR-AUTH-08` — Repeat unverified registration 15
14 `MOB-FR-AUTH-09` — Recover password 16
15 `MOB-FR-AUTH-10` — Refresh and sign out 16
16 `MOB-FR-AUTH-11` — Current account 17
17 `MOB-FR-AUTH-12` — Lock and role-change enforcement 18
18 `MOB-FR-AUTH-13` — Account field validation 19
19 `MOB-FR-AUTH-14` — Authentication rate limits 19
20 `MOB-FR-AUTH-16` — Edit profile 20
21 `MOB-FR-AUTH-17` — Change current password 21
22 `MOB-FR-MOV-02` — Movie details 22
23 `MOB-FR-MOV-03` — Title search 23
24 `MOB-FR-MOV-04` — Active catalogue 23
25 `MOB-FR-MOV-08` — Open trailer 24
26 `MOB-FR-MOV-09` — Age rating 25
27 `MOB-FR-MOV-10` — Now showing and coming soon 25
28 `MOB-FR-MOV-11` — Director and cast 26
29 `MOB-FR-MOV-12` — Movie reviews 27
30 `MOB-FR-SEAT-03` — Read room map 28
31 `MOB-FR-SHOW-03` — Showtime and prices 29
32 `MOB-FR-SHOW-06` — Open future showtimes 29
33 `MOB-FR-SHOW-07` — Showtimes by movie 30
34 `MOB-FR-SHOW-08` — Bookable movie showtimes 30
35 `MOB-FR-SHOW-13` — Screening format 31

36 MOB-FR-BOOK-01 — Hold booking . . . 32
37 MOB-FR-BOOK-02 — Booking details . . . 33
38 MOB-FR-BOOK-04 — Own ticket history . . . 34
39 MOB-FR-BOOK-05 — Confirm paid booking . . . 34
40 MOB-FR-BOOK-06 — Cancel booking . . . 35
41 MOB-FR-BOOK-07 — Occupied seat IDs . . . 36
42 MOB-FR-BOOK-08 — Concurrent seat conflict . . . 37
43 MOB-FR-BOOK-09 — Sweetbox pair . . . 37
44 MOB-FR-BOOK-10 — Expire unpaid hold . . . 38
45 MOB-FR-BOOK-11 — Refresh customer map . . . 39
46 MOB-FR-BOOK-13 — Concessions with tickets . . . 40
47 MOB-FR-BOOK-15 — Promotion and voucher . . . 40
48 MOB-FR-BOOK-17 — Membership points . . . 42
49 MOB-FR-BOOK-19 — Share and calendar . . . 43
50 MOB-FR-PAY-01 — Create/reuse PayOS link . . . 43
51 MOB-FR-PAY-02 — Verify PayOS . . . 44
52 MOB-FR-PAY-03 — Payment details . . . 45
53 MOB-FR-PAY-04 — Own payment history . . . 46
54 MOB-FR-PAY-06 — Refund access . . . 47
55 MOB-FR-PAY-07 — Signed webhook . . . 47
56 MOB-FR-PAY-08 — Late payment . . . 48
57 MOB-FR-PAY-11 — Automatic payment polling . . . 49
58 MOB-FR-REF-01 — Create refund request . . . 50
59 MOB-FR-REF-05 — Reopen rejected refund . . . 51
60 MOB-FR-REF-06 — Own refund status . . . 51
61 MOB-FR-REF-07 — Unique refund per payment . . . 52
62 MOB-FR-CHK-01 — Ticket QR . . . 53
63 MOB-FR-CHK-02 — Lookup ticket code . . . 53
64 MOB-FR-CHK-03 — Check-in validation . . . 54
65 MOB-FR-CHK-04 — Camera/manual scanner . . . 55
66 MOB-FR-CHK-05 — Display check-in time . . . 56
67 MOB-FR-NOTI-01 — Notification history . . . 56
68 MOB-FR-NOTI-04 — Notification details . . . 57
69 MOB-FR-NOTI-06 — Event emails . . . 58
70 MOB-FR-NOTI-07 — OTP and welcome email . . . 59
71 MOB-FR-NOTI-08 — Showtime reminder . . . 59
72 MOB-FR-NOTI-09 — Read/unread notifications . . . 60

MOB-FR-NAV-01 — Route guards and recovery . . . 61
MOB-FR-STATE-01 — Loading, error and empty states . . . 62
MOB-FR-MEDIA-01 — Platform-specific movie media . . . 62
UC-MOB-01 — Authentication and account session . . . 64
UC-MOB-02 — Movie catalogue and reviews . . . 64
UC-MOB-03 — Seat map and availability . . . 65
UC-MOB-04 — Showtimes and prices . . . 65
UC-MOB-05 — Booking, concessions, promotions and loyalty . . . 65
UC-MOB-06 — Online payment and reconciliation . . . 66
UC-MOB-07 — Cancellation and manual refund . . . 67
UC-MOB-08 — Ticket lookup and check-in . . . 67
UC-MOB-09 — Notifications and transactional email . . . 68
11. External Interface Requirements . . . 69
MOB-SCR-LOGIN — Sign in . . . 70
MOB-SCR-REGISTER — Registration . . . 70
MOB-SCR-VERIFY — Verify email . . . 71
MOB-SCR-FORGOT — Forgot password . . . 71
MOB-SCR-RESET — Reset password . . . 71
MOB-SCR-HOME — Customer home . . . 72
MOB-SCR-MOVIE — Movie details . . . 72
MOB-SCR-SEATS — Seat selection . . . 73
MOB-SCR-SUMMARY — Booking summary . . . 73
MOB-SCR-PAYMENT — Payment . . . 73
MOB-SCR-RESULT — Payment result . . . 74
MOB-SCR-TICKETS — My tickets . . . 74
MOB-SCR-TICKET — Ticket details . . . 75
MOB-SCR-PAYHISTORY — Payment history . . . 75
MOB-SCR-NOTIFICATIONS — Notifications . . . 75
MOB-SCR-NOTIFICATION — Notification details . . . 76
MOB-SCR-PROFILE — Account and membership . . . 76
MOB-SCR-MGT-LANDING — Management-account landing . . . 77
MOB-SCR-CHECKIN — Ticket check-in . . . 77
13. API Requirements . . . 78
Request field definitions . . . 82
14. Data Requirements . . . 84
15. Business Workflows . . . 85
State contract . . . 85

110 17. Error Handling 87
111 18. Non-Functional Requirements 88
112 19. Security Requirements 90
113 20. Testing Requirements 91
114 Existing test evidence 91
115 21. Acceptance Criteria 93
116 22. Requirements Traceability Matrix 94
117 23. Constraints 117
118 25. Future Enhancements 119
119 27. Implemented vs Requirement 121
120 27. Implemented vs Requirement 125
121 28. Evidence and source reconciliation 126

1. Introduction

CosmoQ Cinema is one cinema booking and management system with multiple clients. This course-specific SRS refines consolidated SRS 2.0 using the current checkout; it does not define a second independent product. Executable routes/controllers/services/models take precedence over dated prose. SRS 1.0 is historical intent.

H = mandatory; M = important; L = optional. Implemented means source exists, not that integration tests passed. Each EN/VI pair uses identical IDs, scope, priority and criteria; neither language is a summary.

2. Purpose

Provide the contract for development, test-case design, acceptance and course defence. Each `FR` identifies actors, conditions, trigger, flow, exceptions, result and acceptance criteria, and traces to use cases, screens, APIs and source/test evidence.

3. Scope

Include native Flutter customer account/OTP/session/profile, movies/reviews/trailers, open showtimes/seats, concessions/promotions/points at booking, PayOS, owned tickets/payments/refunds, notifications, sharing/calendar and Staff/Admin phone check-in. Shared backend rules do not confer mobile administration privileges. Exclude the admin website despite its source being under Cinema_Mobile.

Technology boundary: no MVC .cshtml, Razor/Blazor .razor or separate .NET UI project is present. “Web .NET” names the course scope: ASP.NET Core REST plus its existing Flutter Web clients. A mandatory .NET-rendered UI is [TO BE CONFIRMED], not implemented evidence.

4. Overall Description

```
Flutter main.dart (Customer + phone check-in)
                | REST/JSON + JWT
        ApiGateway / YARP :5000
          |           |             |
     AuthService  MovieService  BookingService
       :5100        :5168          :5063
          |           |             |
       PostgreSQL (6 databases) + RabbitMQ/outbox
                                  | PayOS + SMTP
```

Three business services replace the initial six-service design: `AuthService` owns identity; `MovieService` owns movie/room/seat/showtime; `BookingService` owns bookings, concessions, promotions, loyalty, reviews, payment/refund and notifications. Cross-database references are logical IDs, not foreign keys.

5. Product Perspective

Table 2: 5. Product Perspective

Area	Responsibility
Shared core	Identity/RBAC/ownership; pricing; seat concurrency; financial states; PayOS verification; refunds; events and data integrity
Web workspace	Counter/cash/printing; movie/room/showtime/account/menu/promotion/member administration; refund settlement; operations/revenue
Native behaviour	External trailer, device share sheet, phone camera check-in, emulator/LAN gateway address and payment verification on resume
Shared presentation	Customer web/native share Dart widgets and repositories; preview/printing is platform conditional

6. User Classes / Actors

Table 3: 6. User Classes / Actors

Actor	Responsibility
Guest	Public catalogue/showtimes; start account flows
Customer	Own profile, tickets, payment, refund state, notifications, reviews and points
Staff/Admin	Phone ticket lookup/check-in; other administration belongs to web
Walk-in guest	Counter-sale beneficiary through staff, not a signed-in mobile actor
`PayOS` / `SMTP` / workers	External settlement/messages; expire holds and exchange committed events

7. Role & Permission

Table 4: 7. Role & Permission

Operation	Guest	Customer	Staff	Admin
Browse public movies/showtimes	✓	✓	✓	✓
Own customer booking/payment/ profile/review	—	✓	Role landing	Role landing
Lookup/check in tickets	—	—	✓	✓

`UI` guards control navigation; controller policies and record-ownership checks are authoritative. Accounts may hold multiple roles. Customer accounts in the admin entry see a customer-site notice; management accounts in the customer entry see management guidance/check-in.

8. Business Rules

Table 5: 8. Business Rules

ID	Rule
MOB-BR-01	Customer accounts must verify their email with an OTP before signing in. Accounts created by an Admin need no OTP.
MOB-BR-02	Every protected API uses JWT; both the gateway and the service check it.
MOB-BR-03	Access depends on the role; Customers cannot use management APIs.
MOB-BR-04	Each seat belongs to one room and its position (row + number) is unique in that room.
MOB-BR-05	A showtime belongs to one movie and one room, and starts before it ends.
MOB-BR-06	A room never has two showtimes at overlapping times (409).
MOB-BR-07	A seat of a showtime belongs to at most one active booking (held or sold).
MOB-BR-08	A booking has one showtime and its seats, and belongs to a customer account, except walk-in tickets sold at the counter.
MOB-BR-09	A payment belongs to a booking; booking/payment/refund transitions must follow the state contract in section 15.
MOB-BR-10	A PayOS webhook must carry a valid HMAC-SHA256 signature before any of its data is used.
MOB-BR-11	Events are written to the outbox in the same transaction as the data and are sent only after commit.
MOB-BR-12	Calls to other services or PayOS use limited retries and a circuit breaker.
MOB-BR-13	An email failure or a stopped RabbitMQ never breaks a booking or payment.
MOB-BR-14	Seats are held for 10 minutes; an unpaid booking then becomes EXPIRED and its seats are released.
MOB-BR-15	Seat price: the showtime price is the standard seat price; VIP adds 20% and sweetbox 30% per seat; the surcharge is rounded to 1,000 VND. The server computes prices and never takes them from the client.
MOB-BR-16	Sweetbox seats are always booked, changed and sold as a pair.
MOB-BR-17	A customer can cancel a paid ticket for a 100% refund until 2 hours before the show; later they must contact the cinema. Staff and Admin can cancel at any time.
MOB-BR-18	A payment has at most one refund request. Refunds are paid by manual bank transfer and recorded with the transfer reference.
MOB-BR-19	When a showtime is cancelled, all its active bookings are cancelled; paid ones get a refund request.
MOB-BR-20	Late payment (booking already expired): if the seats are still free the ticket is confirmed; if they were sold to someone else, a 100% refund request is created.
MOB-BR-21	Only CONFIRMED tickets can be checked in, once each, from 3 hours before to 2 hours after the start time.
MOB-BR-24	Locking an account or changing its roles takes effect on the next request, even while the old token is still valid.
MOB-BR-25	Forgot-password returns 200 regardless of email existence; incorrect login email/password share a message. Registration/resend may reveal account state, so system-wide non-enumeration is not claimed.
MOB-BR-26	Inactive movies disappear from browsing while existing tickets remain. Phone clients cannot edit room layouts.
MOB-BR-27	A promotion reduces tickets only, up to 90%, rounded down to 1,000 VND; one best automatic promotion or an entered eligible code applies. Count usage on successful payment.
MOB-BR-28	Point value is 1,000 VND; redeem at most 50% after promotion; MEMBER/VIP/VVIP spend thresholds are 0/2,000,000/5,000,000 VND and earn rates 5/7/10%. Reverse/release point effects according to booking cancellation/expiry/refund.

Continued on next page

Table 5 — continued

ID	Rule
MOB-BR-29	A booking has at most ten distinct active concession items, quantity 1–20 each; persist purchase-time name and unit price, and include them in payment and refund totals.
MOB-BR-30	Format price adds 0/20,000/40,000 VND for 2D/3D/IMAX before type surcharges; NORMAL/VIP/SWEETBOX prices are returned by the server.

External-operator rules specify shared-backend effects, not additional phone screens. Time limits are current defaults and may be overridden by deployment configuration.

9. Functional Requirements

AC-01 verifies the exact stated behaviour; AC-02 verifies applicable rejection/permission paths. Apply only actions specified by the individual FR; the wider use case supplies context. Criteria are specifications, not recorded test results.

9.1 MOB-FR-AUTH-01 — Register

Table 6: MOB-FR-AUTH-01 — Register

Field	Specification
Name	Register
Description	Customers register with email, password, full name and an optional phone number. The system sends a verification OTP and issues no token yet.
Actor	Guest; signed-in account; system
Preconditions	The account form is reachable and gateway/AuthService are configured; no prior signed-in session is required for registration/login/OTP/recovery. SMTP is required when sending a code.
Trigger	Invoke “Register” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Customers register with email, password, full name and an optional phone number. The system sends a verification OTP and issues no token yet. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (400), bad credentials (401), unverified email (403), stale/incorrect OTP, or resend/rate limits (429). Clear the session for account-lock/role-change responses; show SMTP/dependency failure (503).
Postconditions	An unverified registration and OTP delivery outcome are recorded; no access/refresh tokens are issued before verification.
Priority	H
Acceptance criteria	**MOB-FR-AUTH-01-AC-01** — Given the preconditions/trigger, verify: Customers register with email, password, full name and an optional phone number. The system sends a verification OTP and issues no token yet. **MOB-FR-AUTH-01-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_Mobile/lib/features/auth/screens/register_screen.dart; Cinema_BE/AuthService/AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/ApiGateway/Program.cs
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; MOB-SCR-REGISTER; MOB-API-011

9.2 MOB-FR-AUTH-02 — Sign in

Table 7: MOB-FR-AUTH-02 — Sign in

Field	Specification
Name	Sign in
Description	Sign in with email and password to get an access token (JWT, 60 minutes) and a refresh token (7 days). An unverified email gets 403 EMAIL_NOT_VERIFIED.

Continued on next page

Table 7 — continued

Field	Specification
Actor	Guest; signed-in account; system
Preconditions	The account form is reachable and gateway/`AuthService` are configured; no prior signed-in session is required for registration/login/`OTP`/recovery. `SMTP` is required when sending a code.
Trigger	Invoke “Sign in” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Sign in with email and password to get an access token (`JWT`, 60 minutes) and a refresh token (7 days). An unverified email gets `403 EMAIL_NOT_VERIFIED`. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (`400`), bad credentials (`401`), unverified email (`403`), stale/incorrect `OTP`, or resend/rate limits (`429`). Clear the session for account-lock/role-change responses; show `SMTP`/dependency failure (`503`).
Postconditions	Successful verified authentication stores the returned user/tokens and selects the allowed role landing; failure grants no session.
Priority	H
Acceptance criteria	**`MOB-FR-AUTH-02-AC-01`** — Given the preconditions/trigger, verify: Sign in with email and password to get an access token (`JWT`, 60 minutes) and a refresh token (7 days). An unverified email gets `403 EMAIL_NOT_VERIFIED`. **`MOB-FR-AUTH-02-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs`; `Cinema_Mobile/lib/features/auth/screens/login_screen.dart`; `Cinema_BE/AuthService/AuthService/Service/AuthService.cs`; `Cinema_BE/ApiGateway/ApiGateway/Program.cs`
Business rules	`MOB-BR-01`, `MOB-BR-02`, `MOB-BR-03`, `MOB-BR-24`, `MOB-BR-25`
Use case / screens / APIs	`UC-MOB-01`; `MOB-SCR-LOGIN`; `MOB-API-006`

9.3 `MOB-FR-AUTH-03` — `JWT` claims

Table 8: `MOB-FR-AUTH-03` — `JWT` claims

Field	Specification
Name	`JWT` claims
Description	The `JWT` carries the `userId`, email and roles.
Actor	Guest; signed-in account; system
Preconditions	The gateway is configured. Registration/recovery needs `SMTP`; protected operations require a valid account session.
Trigger	Invoke “`JWT` claims” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: The `JWT` carries the `userId`, email and roles. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (`400`), bad credentials (`401`), unverified email (`403`), stale/incorrect `OTP`, or resend/rate limits (`429`). Clear the session for account-lock/role-change responses; show `SMTP`/dependency failure (`503`).
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H

Continued on next page

Table 8 — continued

Field	Specification
Acceptance criteria	**MOB-FR-AUTH-03-AC-01** — Given the preconditions/trigger, verify: The JWT carries the user Id, email and roles. **MOB-FR-AUTH-03-AC-02** — Exercise applicable invalid/unauthorized/ conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_ Mobile/lib/features/auth/screens/login_screen.dart; Cinema_BE/AuthService/ AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/ApiGateway/ Program.cs
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; MOB-SCR-LOGIN; MOB-API-006, MOB-API-010

9.4 MOB-FR-AUTH-04 — Validate JWT

Table 9: MOB-FR-AUTH-04 — Validate JWT

Field	Specification
Name	Validate JWT
Description	The gateway and services check the JWT signature, expiry, issuer and audience before protected APIs.
Actor	Guest; signed-in account; system
Preconditions	The gateway is configured. Registration/recovery needs SMTP; protected operations require a valid account session.
Trigger	Invoke “Validate JWT” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: The gateway and services check the JWT signature, expiry, issuer and audience before protected APIs. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (400), bad credentials (401), unverified email (403), stale/incorrect OTP, or resend/rate limits (429). Clear the session for account-lock/role-change responses; show SMTP/dependency failure (503).
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/ payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-AUTH-04-AC-01** — Given the preconditions/trigger, verify: The gateway and services check the JWT signature, expiry, issuer and audience before protected APIs. **MOB-FR-AUTH-04-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_BE/ AuthService/AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/Api Gateway/Program.cs
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; ; MOB-API-008

9.5 MOB-FR-AUTH-05 — Authorize roles

Table 10: MOB-FR-AUTH-05 — Authorize roles

Field	Specification
Name	Authorize roles
Description	Mobile routes distinguish Customer and management accounts. Customers use their own data; Staff/Admin can use mobile ticket check-in only. The backend still enforces endpoint and record permissions.
Actor	Guest; signed-in account; system
Preconditions	The gateway is configured. Registration/recovery needs SMTP; protected operations require a valid account session.
Trigger	Invoke “Authorize roles” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Mobile routes distinguish Customer and management accounts. Customers use their own data; Staff/Admin can use mobile ticket check-in only. The backend still enforces endpoint and record permissions. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (400), bad credentials (401), unverified email (403), stale/incorrect OTP, or resend/rate limits (429). Clear the session for account-lock/role-change responses; show SMTP/dependency failure (503).
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-AUTH-05-AC-01** — Given the preconditions/trigger, verify: Mobile routes distinguish Customer and management accounts. Customers use their own data; Staff/Admin can use mobile ticket check-in only. The backend still enforces endpoint and record permissions. **MOB-FR-AUTH-05-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_Mobile/lib/features/profile/screens/management_account_screen.dart; Cinema_BE/AuthService/AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/ApiGateway/Program.cs
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; MOB-SCR-MGT-LANDING; MOB-API-008

9.6 MOB-FR-AUTH-06 — Unauthenticated and forbidden responses

Table 11: MOB-FR-AUTH-06 — Unauthenticated and forbidden responses

Field	Specification
Name	Unauthenticated and forbidden responses
Description	A missing or invalid token gets 401; a signed-in user without the role gets 403.
Actor	Guest; signed-in account; system
Preconditions	The gateway is configured. Registration/recovery needs SMTP; protected operations require a valid account session.
Trigger	Invoke “Unauthenticated and forbidden responses” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: A missing or invalid token gets 401; a signed-in user without the role gets 403. 3. Return/render the authoritative result and refresh affected data after success.

Continued on next page

Table 11 — continued

Field	Specification
Alternative / exception flow	Reject invalid fields (400), bad credentials (401), unverified email (403), stale/incorrect OTP, or resend/rate limits (429). Clear the session for account-lock/role-change responses; show SMTP/dependency failure (503).
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-AUTH-06-AC-01** — Given the preconditions/trigger, verify: A missing or invalid token gets 401; a signed-in user without the role gets 403. **MOB-FR-AUTH-06-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_Mobile/lib/features/auth/screens/login_screen.dart; Cinema_BE/AuthService/AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/ApiGateway/Program.cs
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; MOB-SCR-LOGIN; MOB-API-008

9.7 MOB-FR-AUTH-07 — Verify email OTP

Table 12: MOB-FR-AUTH-07 — Verify email OTP

Field	Specification
Name	Verify email OTP
Description	Email verification with a 6-digit OTP: expires after 5 minutes, at most 5 attempts, 60 seconds between sends, a new code replaces the old one. Verification must include the registered password; on success the user is signed in and gets a welcome email.
Actor	Guest; signed-in account; system
Preconditions	The account form is reachable and gateway/AuthService are configured; no prior signed-in session is required for registration/login/OTP/recovery. SMTP is required when sending a code.
Trigger	Invoke “Verify email OTP” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Email verification with a 6-digit OTP: expires after 5 minutes, at most 5 attempts, 60 seconds between sends, a new code replaces the old one. Verification must include the registered password; on success the user is signed in and gets a welcome email. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (400), bad credentials (401), unverified email (403), stale/incorrect OTP, or resend/rate limits (429). Clear the session for account-lock/role-change responses; show SMTP/dependency failure (503).
Postconditions	Successful verified authentication stores the returned user/tokens and selects the allowed role landing; failure grants no session.
Priority	H
Acceptance criteria	**MOB-FR-AUTH-07-AC-01** — Given the preconditions/trigger, verify: Email verification with a 6-digit OTP: expires after 5 minutes, at most 5 attempts, 60 seconds between sends, a new code replaces the old one. Verification must include the registered password; on success the user is signed in and gets a welcome email. **MOB-FR-AUTH-07-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.

Continued on next page

Table 12 — continued

Field	Specification
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_Mobile/lib/features/auth/screens/verify_email_screen.dart; Cinema_BE/AuthService/AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/ApiGateway/Program.cs
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; MOB-SCR-VERIFY; MOB-API-012, MOB-API-014

9.8 MOB-FR-AUTH-08 — Repeat unverified registration

Table 13: MOB-FR-AUTH-08 — Repeat unverified registration

Field	Specification
Name	Repeat unverified registration
Description	Registering again with an unverified email replaces the old details and sends a new code. A verified email is reported as already registered.
Actor	Guest; signed-in account; system
Preconditions	The account form is reachable and gateway/AuthService are configured; no prior signed-in session is required for registration/login/OTP/recovery. SMTP is required when sending a code.
Trigger	Invoke “Repeat unverified registration” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Registering again with an unverified email replaces the old details and sends a new code. A verified email is reported as already registered. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (400), bad credentials (401), unverified email (403), stale/incorrect OTP, or resend/rate limits (429). Clear the session for account-lock/role-change responses; show SMTP/dependency failure (503).
Postconditions	An unverified registration and OTP delivery outcome are recorded; no access/refresh tokens are issued before verification.
Priority	M
Acceptance criteria	**MOB-FR-AUTH-08-AC-01** — Given the preconditions/trigger, verify: Registering again with an unverified email replaces the old details and sends a new code. A verified email is reported as already registered. **MOB-FR-AUTH-08-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_Mobile/lib/features/auth/screens/register_screen.dart; Cinema_Mobile/lib/features/auth/screens/verify_email_screen.dart; Cinema_BE/AuthService/AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/ApiGateway/Program.cs
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; MOB-SCR-REGISTER, MOB-SCR-VERIFY; MOB-API-011

9.9 MOB-FR-AUTH-09 — Recover password

Table 14: MOB-FR-AUTH-09 — Recover password

Field	Specification
Name	Recover password
Description	Forgot password: an OTP is emailed (the API always returns 200); setting a new password with the OTP revokes every refresh token and marks the email as verified.
Actor	Guest; signed-in account; system
Preconditions	The account form is reachable and gateway/AuthService are configured; no prior signed-in session is required for registration/login/OTP/recovery. SMTP is required when sending a code.
Trigger	Invoke “Recover password” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Forgot password: an OTP is emailed (the API always returns 200); setting a new password with the OTP revokes every refresh token and marks the email as verified. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (400), bad credentials (401), unverified email (403), stale/incorrect OTP, or resend/rate limits (429). Clear the session for account-lock/role-change responses; show SMTP/dependency failure (503).
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**MOB-FR-AUTH-09-AC-01** — Given the preconditions/trigger, verify: Forgot password: an OTP is emailed (the API always returns 200); setting a new password with the OTP revokes every refresh token and marks the email as verified. **MOB-FR-AUTH-09-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_Mobile/lib/features/auth/screens/forgot_password_screen.dart; Cinema_Mobile/lib/features/auth/screens/reset_password_screen.dart; Cinema_BE/AuthService/AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/ApiGateway/Program.cs
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; MOB-SCR-FORGOT, MOB-SCR-RESET; MOB-API-005, MOB-API-013

9.10 MOB-FR-AUTH-10 — Refresh and sign out

Table 15: MOB-FR-AUTH-10 — Refresh and sign out

Field	Specification
Name	Refresh and sign out
Description	Renew the session with the refresh token (rotated; the old one stops working); signing out revokes the refresh token. The app renews automatically when the access token expires.
Actor	Guest; signed-in account; system
Preconditions	The gateway is configured. Registration/recovery needs SMTP; protected operations require a valid account session.
Trigger	Invoke “Refresh and sign out” or its specified backend event/worker.

Continued on next page

Table 15 — continued

Field	Specification
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Renew the session with the refresh token (rotated; the old one stops working); signing out revokes the refresh token. The app renews automatically when the access token expires. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (`400`), bad credentials (`401`), unverified email (`403`), stale/incorrect `OTP`, or resend/rate limits (`429`). Clear the session for account-lock/role-change responses; show `SMTP`/dependency failure (`503`).
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**`MOB-FR-AUTH-10-AC-01`** — Given the preconditions/trigger, verify: Renew the session with the refresh token (rotated; the old one stops working); signing out revokes the refresh token. The app renews automatically when the access token expires. **`MOB-FR-AUTH-10-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs`; `Cinema_Mobile/lib/features/profile/screens/profile_screen.dart`; `Cinema_Mobile/lib/features/profile/screens/management_account_screen.dart`; `Cinema_BE/AuthService/AuthService/Service/AuthService.cs`; `Cinema_BE/ApiGateway/ApiGateway/Program.cs`
Business rules	`MOB-BR-01`, `MOB-BR-02`, `MOB-BR-03`, `MOB-BR-24`, `MOB-BR-25`
Use case / screens / APIs	`UC-MOB-01`; `MOB-SCR-PROFILE`, `MOB-SCR-MGT-LANDING`; `MOB-API-007`, `MOB-API-010`

9.11 `MOB-FR-AUTH-11` — Current account

Table 16: `MOB-FR-AUTH-11` — Current account

Field	Specification
Name	Current account
Description	Show the current account (`GET /api/v1/auth/me`).
Actor	Guest; signed-in account; system
Preconditions	The gateway is configured. Registration/recovery needs `SMTP`; protected operations require a valid account session.
Trigger	Invoke “Current account” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Show the current account (`GET /api/v1/auth/me`). 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (`400`), bad credentials (`401`), unverified email (`403`), stale/incorrect `OTP`, or resend/rate limits (`429`). Clear the session for account-lock/role-change responses; show `SMTP`/dependency failure (`503`).
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	M
Acceptance criteria	**`MOB-FR-AUTH-11-AC-01`** — Given the preconditions/trigger, verify: Show the current account (`GET /api/v1/auth/me`). **`MOB-FR-AUTH-11-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.

Continued on next page

Table 16 — continued

Field	Specification
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_Mobile/lib/features/profile/screens/profile_screen.dart; Cinema_BE/AuthService/AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/ApiGateway/Program.cs
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; MOB-SCR-PROFILE; MOB-API-008

9.12 MOB-FR-AUTH-12 — Lock and role-change enforcement

Table 17: MOB-FR-AUTH-12 — Lock and role-change enforcement

Field	Specification
Name	Lock and role-change enforcement
Description	The gateway checks the account on every protected request: locked returns 401 ACCOUNT_LOCKED; changed roles return 403 ROLE_CHANGED; an unreachable AuthService returns 503. The customer app shows the reason and returns to sign-in.
Actor	Guest; signed-in account; system
Preconditions	The gateway is configured. Registration/recovery needs SMTP; protected operations require a valid account session.
Trigger	Invoke “Lock and role-change enforcement” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: The gateway checks the account on every protected request: locked returns 401 ACCOUNT_LOCKED; changed roles return 403 ROLE_CHANGED; an unreachable AuthService returns 503. The customer app shows the reason and returns to sign-in. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (400), bad credentials (401), unverified email (403), stale/incorrect OTP, or resend/rate limits (429). Clear the session for account-lock/role-change responses; show SMTP/dependency failure (503).
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-AUTH-12-AC-01** — Given the preconditions/trigger, verify: The gateway checks the account on every protected request: locked returns 401 ACCOUNT_LOCKED; changed roles return 403 ROLE_CHANGED; an unreachable AuthService returns 503. The customer app shows the reason and returns to sign-in. **MOB-FR-AUTH-12-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_Mobile/lib/features/auth/screens/login_screen.dart; Cinema_BE/AuthService/AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/ApiGateway/Program.cs
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; MOB-SCR-LOGIN; MOB-API-008

9.13 MOB-FR-AUTH-13 — Account field validation

Table 18: MOB-FR-AUTH-13 — Account field validation

Field	Specification
Name	Account field validation
Description	Mobile validates email (required, max 150, no spaces, domain suffix at least two letters), new passwords (6–72 UTF-8 bytes), name (required, max 150) and optional phone (8–15 digits, max 20 characters). Registration/new-password forms confirm the password; server field errors return 400 and are shown to the user.
Actor	Guest; signed-in account; system
Preconditions	The account form is reachable and gateway/AuthService are configured; no prior signed-in session is required for registration/login/OTP/recovery. SMTP is required when sending a code.
Trigger	Invoke “Account field validation” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Mobile validates email (required, max 150, no spaces, domain suffix at least two letters), new passwords (6–72 UTF-8 bytes), name (required, max 150) and optional phone (8–15 digits, max 20 characters). Registration/new-password forms confirm the password; server field errors return 400 and are shown to the user. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (400), bad credentials (401), unverified email (403), stale/incorrect OTP, or resend/rate limits (429). Clear the session for account-lock/role-change responses; show SMTP/dependency failure (503).
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**MOB-FR-AUTH-13-AC-01** — Given the preconditions/trigger, verify: Mobile validates email (required, max 150, no spaces, domain suffix at least two letters), new passwords (6–72 UTF-8 bytes), name (required, max 150) and optional phone (8–15 digits, max 20 characters). Registration/new-password forms confirm the password; server field errors return 400 and are shown to the user. **MOB-FR-AUTH-13-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_Mobile/lib/features/auth/screens/login_screen.dart; Cinema_Mobile/lib/features/auth/screens/register_screen.dart; Cinema_Mobile/lib/features/auth/screens/forgot_password_screen.dart; Cinema_Mobile/lib/features/auth/screens/reset_password_screen.dart; Cinema_BE/AuthService/AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/ApiGateway/Program.cs
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; MOB-SCR-LOGIN, MOB-SCR-REGISTER, MOB-SCR-FORGOT, MOB-SCR-RESET; MOB-API-006, MOB-API-011, MOB-API-013

9.14 MOB-FR-AUTH-14 — Authentication rate limits

Table 19: MOB-FR-AUTH-14 — Authentication rate limits

Field	Specification
Name	Authentication rate limits
Description	Registration, sign-in, OTP, refresh and forgot / reset password are limited to 10 requests per minute per IP; beyond that the API returns 429 with Retry-After.

Continued on next page

Table 19 — continued

Field	Specification
Actor	Guest; signed-in account; system
Preconditions	The account form is reachable and gateway/`AuthService` are configured; no prior signed-in session is required for registration/login/`OTP`/recovery. `SMTP` is required when sending a code.
Trigger	Invoke “Authentication rate limits” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Registration, sign-in, `OTP`, refresh and forgot / reset password are limited to 10 requests per minute per `IP`; beyond that the `API` returns `429` with `Retry-After`. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (`400`), bad credentials (`401`), unverified email (`403`), stale/incorrect `OTP`, or resend/rate limits (`429`). Clear the session for account-lock/role-change responses; show `SMTP`/dependency failure (`503`).
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	M
Acceptance criteria	**`MOB-FR-AUTH-14-AC-01`** — Given the preconditions/trigger, verify: Registration, sign-in, `OTP`, refresh and forgot / reset password are limited to 10 requests per minute per `IP`; beyond that the `API` returns `429` with `Retry-After`. **`MOB-FR-AUTH-14-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs`; `Cinema_Mobile/lib/features/auth/screens/verify_email_screen.dart`; `Cinema_Mobile/lib/features/auth/screens/forgot_password_screen.dart`; `Cinema_Mobile/lib/features/auth/screens/reset_password_screen.dart`; `Cinema_BE/AuthService/AuthService/Service/AuthService.cs`; `Cinema_BE/ApiGateway/ApiGateway/Program.cs`
Business rules	`MOB-BR-01`, `MOB-BR-02`, `MOB-BR-03`, `MOB-BR-24`, `MOB-BR-25`
Use case / screens / APIs	`UC-MOB-01`; `MOB-SCR-VERIFY`, `MOB-SCR-FORGOT`, `MOB-SCR-RESET`; `MOB-API-006`, `MOB-API-010`, `MOB-API-012`

9.15 `MOB-FR-AUTH-16` — Edit profile

Table 20: `MOB-FR-AUTH-16` — Edit profile

Field	Specification
Name	Edit profile
Description	Users **edit their profile** (Account → Edit profile): full name (required, at most 150 characters) and phone (optional, same rules as registration; empty removes it). The email is the sign-in name and cannot change.
Actor	Guest; signed-in account; system
Preconditions	The gateway is configured. Registration/recovery needs `SMTP`; protected operations require a valid account session.
Trigger	Invoke “Edit profile” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Users **edit their profile** (Account → Edit profile): full name (required, at most 150 characters) and phone (optional, same rules as registration; empty removes it). The email is the sign-in name and cannot change. 3. Return/render the authoritative result and refresh affected data after success.

Continued on next page

Table 20 — continued

Field	Specification
Alternative / exception flow	Reject invalid fields (`400`), bad credentials (`401`), unverified email (`403`), stale/incorrect `OTP`, or resend/rate limits (`429`). Clear the session for account-lock/role-change responses; show `SMTP`/dependency failure (`503`).
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	M
Acceptance criteria	**`MOB-FR-AUTH-16-AC-01`** — Given the preconditions/trigger, verify: Users **edit their profile** (Account → Edit profile): full name (required, at most 150 characters) and phone (optional, same rules as registration; empty removes it). The email is the sign-in name and cannot change. **`MOB-FR-AUTH-16-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs`; `Cinema_Mobile/lib/features/profile/screens/profile_screen.dart`; `Cinema_BE/AuthService/AuthService/Service/AuthService.cs`; `Cinema_BE/ApiGateway/ApiGateway/Program.cs`
Business rules	`MOB-BR-01`, `MOB-BR-02`, `MOB-BR-03`, `MOB-BR-24`, `MOB-BR-25`
Use case / screens / APIs	`UC-MOB-01`; `MOB-SCR-PROFILE`; `MOB-API-009`

9.16 `MOB-FR-AUTH-17` — Change current password

Table 21: `MOB-FR-AUTH-17` — Change current password

Field	Specification
Name	Change current password
Description	**Change the password with the current one** (Account → Change password): current password, new password twice. A wrong current password or the same password is refused (`400`). Afterwards other devices are signed out (refresh tokens revoked) and this device gets new tokens. A forgotten current password leads to the email `OTP` flow. Limited to 10 requests per minute like sign-in.
Actor	Guest; signed-in account; system
Preconditions	The gateway is configured. Registration/recovery needs `SMTP`; protected operations require a valid account session.
Trigger	Invoke “Change current password” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: **Change the password with the current one** (Account → Change password): current password, new password twice. A wrong current password or the same password is refused (`400`). Afterwards other devices are signed out (refresh tokens revoked) and this device gets new tokens. A forgotten current password leads to the email `OTP` flow. Limited to 10 requests per minute like sign-in. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (`400`), bad credentials (`401`), unverified email (`403`), stale/incorrect `OTP`, or resend/rate limits (`429`). Clear the session for account-lock/role-change responses; show `SMTP`/dependency failure (`503`).
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	M

Continued on next page

Table 21 — continued

Field	Specification
Acceptance criteria	**MOB-FR-AUTH-17-AC-01** — Given the preconditions/trigger, verify: **Change the password with the current one** (Account → Change password): current password, new password twice. A wrong current password or the same password is refused (400). Afterwards other devices are signed out (refresh tokens revoked) and this device gets new tokens. A forgotten current password leads to the email OTP flow. Limited to 10 requests per minute like sign-in. **MOB-FR-AUTH-17-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_Mobile/lib/features/profile/screens/profile_screen.dart; Cinema_BE/AuthService/AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/ApiGateway/Program.cs
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; MOB-SCR-PROFILE; MOB-API-004

9.17 MOB-FR-MOV-02 — Movie details

Table 22: MOB-FR-MOV-02 — Movie details

Field	Specification
Name	Movie details
Description	Movie details: title, description, duration, genre, language, release date, poster, trailer, status.
Actor	Guest/Customer
Preconditions	The public catalogue is reachable; the movie exists for detail/trailer operations. Empty search results are valid.
Trigger	Invoke “Movie details” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Movie details: title, description, duration, genre, language, release date, poster, trailer, status. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Show empty results for no matching films/showtimes. Review of another user’s ticket returns 404; a ticket not checked in returns 400. Failed media must retain a poster/fallback.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-MOV-02-AC-01** — Given the preconditions/trigger, verify: Movie details: title, description, duration, genre, language, release date, poster, trailer, status. **MOB-FR-MOV-02-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/MovieService/MovieService/Controllers/MoviesController.cs; Cinema_Mobile/lib/features/movie/screens/movie_detail_screen.dart; Cinema_BE/MovieService/MovieService/Service/Impl/MovieService.cs; Cinema_BE/BookingService/BookingService/Services/Implementations/MovieReviewService.cs
Business rules	MOB-BR-26
Use case / screens / APIs	UC-MOB-02; MOB-SCR-MOVIE; MOB-API-024

9.18 MOB-FR-MOV-03 — Title search

Table 23: MOB-FR-MOV-03 — Title search

Field	Specification
Name	Title search
Description	Search movies by title.
Actor	Guest/Customer
Preconditions	The public catalogue is reachable; the movie exists for detail/trailer operations. Empty search results are valid.
Trigger	Invoke “Title search” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Search movies by title. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Show empty results for no matching films/showtimes. Review of another user’s ticket returns 404; a ticket not checked in returns 400. Failed media must retain a poster/fallback.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	M
Acceptance criteria	**MOB-FR-MOV-03-AC-01** — Given the preconditions/trigger, verify: Search movies by title. **MOB-FR-MOV-03-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/MovieService/MovieService/Controllers/MoviesController.cs; Cinema_Mobile/lib/features/home/screens/home_screen.dart; Cinema_BE/MovieService/MovieService/Service/Impl/MovieService.cs; Cinema_BE/BookingService/BookingService/Services/Implementations/MovieReviewService.cs
Business rules	MOB-BR-26
Use case / screens / APIs	UC-MOB-02; MOB-SCR-HOME; MOB-API-025

9.19 MOB-FR-MOV-04 — Active catalogue

Table 24: MOB-FR-MOV-04 — Active catalogue

Field	Specification
Name	Active catalogue
Description	The customer catalogue displays ACTIVE movies only; INACTIVE movies are excluded from browsing without deleting historical bookings.
Actor	Guest/Customer
Preconditions	The public catalogue is reachable; the movie exists for detail/trailer operations. Empty search results are valid.
Trigger	Invoke “Active catalogue” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: The customer catalogue displays ACTIVE movies only; INACTIVE movies are excluded from browsing without deleting historical bookings. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Show empty results for no matching films/showtimes. Review of another user’s ticket returns 404; a ticket not checked in returns 400. Failed media must retain a poster/fallback.

Continued on next page

Table 24 — continued

Field	Specification
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-MOV-04-AC-01** — Given the preconditions/trigger, verify: The customer catalogue displays ACTIVE movies only; INACTIVE movies are excluded from browsing without deleting historical bookings. **MOB-FR-MOV-04-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/MovieService/MovieService/Controllers/MoviesController.cs; Cinema_Mobile/lib/features/home/screens/home_screen.dart; Cinema_BE/MovieService/MovieService/Service/Impl/MovieService.cs; Cinema_BE/BookingService/BookingService/Services/Implementations/MovieReviewService.cs
Business rules	MOB-BR-26
Use case / screens / APIs	UC-MOB-02; MOB-SCR-HOME; MOB-API-025, MOB-API-027

9.20 MOB-FR-MOV-08 — Open trailer

Table 25: MOB-FR-MOV-08 — Open trailer

Field	Specification
Name	Open trailer
Description	The customer app opens the movie trailer (trailerUrl).
Actor	Guest/Customer
Preconditions	The public catalogue is reachable; the movie exists for detail/trailer operations. Empty search results are valid.
Trigger	Invoke “Open trailer” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: The customer app opens the movie trailer (trailerUrl). 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Show empty results for no matching films/showtimes. Review of another user’s ticket returns 404; a ticket not checked in returns 400. Failed media must retain a poster/fallback.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	L
Acceptance criteria	**MOB-FR-MOV-08-AC-01** — Given the preconditions/trigger, verify: The customer app opens the movie trailer (trailerUrl). **MOB-FR-MOV-08-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/MovieService/MovieService/Controllers/MoviesController.cs; Cinema_Mobile/lib/features/movie/screens/movie_detail_screen.dart; Cinema_BE/MovieService/MovieService/Service/Impl/MovieService.cs; Cinema_BE/BookingService/BookingService/Services/Implementations/MovieReviewService.cs
Business rules	MOB-BR-26
Use case / screens / APIs	UC-MOB-02; MOB-SCR-MOVIE; MOB-API-024

9.21 MOB-FR-MOV-09 — Age rating

Table 26: MOB-FR-MOV-09 — Age rating

Field	Specification
Name	Age rating
Description	Display P/K/T13/T16/T18 on cards, movie details and booking summary. For a non-P film require age-condition acknowledgement before holding seats; mobile check-in reminds Staff/Admin to check identity when rating data is available.
Actor	Guest/Customer
Preconditions	A movie is available; changes need Admin; a review needs the owner's checked-in ticket.
Trigger	Invoke "Age rating" or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Display P/K/T13/T16/T18 on cards, movie details and booking summary. For a non-P film require age-condition acknowledgement before holding seats; mobile check-in reminds Staff/Admin to check identity when rating data is available. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Show empty results for no matching films/showtimes. Review of another user's ticket returns `404`; a ticket not checked in returns `400`. Failed media must retain a poster/fallback.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-MOV-09-AC-01** — Given the preconditions/trigger, verify: Display P/K/T13/T16/T18 on cards, movie details and booking summary. For a non-P film require age-condition acknowledgement before holding seats; mobile check-in reminds Staff/Admin to check identity when rating data is available. **MOB-FR-MOV-09-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/MovieService/MovieService/Controllers/MoviesController.cs; Cinema_Mobile/lib/features/movie/screens/movie_detail_screen.dart; Cinema_Mobile/lib/features/booking/screens/booking_summary_screen.dart; Cinema_Mobile/lib/features/staff/screens/staff_check_in_screen.dart; Cinema_BE/MovieService/MovieService/Service/Impl/MovieService.cs; Cinema_BE/BookingService/BookingService/Services/Implementations/MovieReviewService.cs
Business rules	MOB-BR-26
Use case / screens / APIs	UC-MOB-02; MOB-SCR-MOVIE, MOB-SCR-SUMMARY, MOB-SCR-CHECKIN; MOB-API-024

9.22 MOB-FR-MOV-10 — Now showing and coming soon

Table 27: MOB-FR-MOV-10 — Now showing and coming soon

Field	Specification
Name	Now showing and coming soon
Description	The customer home has two tabs: **Now showing** (`ACTIVE` movies already released or without a release date, newest first) and **Coming soon** (`ACTIVE` movies released after today, earliest first, the release date on the poster). Movies can be filtered by genre (chips built from the genres of the movies on offer) and searched by title through the `API`, ignoring case. A movie coming soon shows its release date; without showtimes the page says they will be added closer to the release.

Continued on next page

Table 27 — continued

Field	Specification
Actor	Guest/Customer
Preconditions	The public catalogue is reachable; the movie exists for detail/trailer operations. Empty search results are valid.
Trigger	Invoke “Now showing and coming soon” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: The customer home has two tabs: **Now showing** (`ACTIVE` movies already released or without a release date, newest first) and **Coming soon** (`ACTIVE` movies released after today, earliest first, the release date on the poster). Movies can be filtered by genre (chips built from the genres of the movies on offer) and searched by title through the `API`, ignoring case. A movie coming soon shows its release date; without showtimes the page says they will be added closer to the release. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Show empty results for no matching films/showtimes. Review of another user’s ticket returns `404`; a ticket not checked in returns `400`. Failed media must retain a poster/fallback.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	M
Acceptance criteria	**`MOB-FR-MOV-10-AC-01`** — Given the preconditions/trigger, verify: The customer home has two tabs: **Now showing** (`ACTIVE` movies already released or without a release date, newest first) and **Coming soon** (`ACTIVE` movies released after today, earliest first, the release date on the poster). Movies can be filtered by genre (chips built from the genres of the movies on offer) and searched by title through the `API`, ignoring case. A movie coming soon shows its release date; without showtimes the page says they will be added closer to the release. **`MOB-FR-MOV-10-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/MovieService/MovieService/Controllers/MoviesController.cs`; `Cinema_Mobile/lib/features/home/screens/home_screen.dart`; `Cinema_BE/MovieService/MovieService/Service/Impl/MovieService.cs`; `Cinema_BE/BookingService/BookingService/Services/Implementations/MovieReviewService.cs`
Business rules	`MOB-BR-26`
Use case / screens / APIs	`UC-MOB-02`; `MOB-SCR-HOME`; `MOB-API-025`, `MOB-API-026`

9.23 `MOB-FR-MOV-11` — Director and cast

Table 28: `MOB-FR-MOV-11` — Director and cast

Field	Specification
Name	Director and cast
Description	Display optional director (max 150 characters) and main cast (max `500`) from movie metadata. Mobile does not edit them.
Actor	Guest/Customer
Preconditions	The public catalogue is reachable; the movie exists for detail/trailer operations. Empty search results are valid.
Trigger	Invoke “Director and cast” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Display optional director (max 150 characters) and main cast (max `500`) from movie metadata. Mobile does not edit them. 3. Return/render the authoritative result and refresh affected data after success.

Continued on next page

Table 28 — continued

Field	Specification
Alternative / exception flow	Show empty results for no matching films/showtimes. Review of another user's ticket returns `404`; a ticket not checked in returns `400`. Failed media must retain a poster/fallback.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	L
Acceptance criteria	**`MOB-FR-MOV-11-AC-01`** — Given the preconditions/trigger, verify: Display optional director (max 150 characters) and main cast (max `500`) from movie metadata. Mobile does not edit them. **`MOB-FR-MOV-11-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/MovieService/MovieService/Controllers/MoviesController.cs`; `Cinema_Mobile/lib/features/movie/screens/movie_detail_screen.dart`; `Cinema_BE/MovieService/MovieService/Service/Impl/MovieService.cs`; `Cinema_BE/BookingService/BookingService/Services/Implementations/MovieReviewService.cs`
Business rules	`MOB-BR-26`
Use case / screens / APIs	`UC-MOB-02`; `MOB-SCR-MOVIE`; `MOB-API-024`

9.24 `MOB-FR-MOV-12` — Movie reviews

Table 29: `MOB-FR-MOV-12` — Movie reviews

Field	Specification
Name	Movie reviews
Description	Display a movie's average, star distribution and newest reviews. A customer can create/update a 1–5 star review with at most 1,000 comment characters from their checked-in ticket, one review per customer per movie, and delete their own review. A ticket not checked in returns `400`; another owner's ticket returns `404`. Display full name or masked email.
Actor	Guest/Customer
Preconditions	A movie is available; changes need Admin; a review needs the owner's checked-in ticket.
Trigger	Invoke "Movie reviews" or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Display a movie's average, star distribution and newest reviews. A customer can create/update a 1–5 star review with at most 1,000 comment characters from their checked-in ticket, one review per customer per movie, and delete their own review. A ticket not checked in returns `400`; another owner's ticket returns `404`. Display full name or masked email. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Show empty results for no matching films/showtimes. Review of another user's ticket returns `404`; a ticket not checked in returns `400`. Failed media must retain a poster/fallback.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	M
Acceptance criteria	**`MOB-FR-MOV-12-AC-01`** — Given the preconditions/trigger, verify: Display a movie's average, star distribution and newest reviews. A customer can create/update a 1–5 star review with at most 1,000 comment characters from their checked-in ticket, one review per customer per movie, and delete their own review. A ticket not checked in returns `400`; another owner's ticket returns `404`. Display full name or masked email. **`MOB-FR-MOV-12-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.

Continued on next page

Table 29 — continued

Field	Specification
Evidence	Cinema_BE/BookingService/BookingService/Controllers/MovieReviewController.cs; Cinema_Mobile/lib/features/movie/screens/movie_detail_screen.dart; Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart; Cinema_BE/MovieService/MovieService/Service/Impl/MovieService.cs; Cinema_BE/BookingService/BookingService/Services/Implementations/MovieReviewService.cs
Business rules	MOB-BR-26
Use case / screens / APIs	UC-MOB-02; MOB-SCR-MOVIE, MOB-SCR-TICKET; MOB-API-039, MOB-API-040, MOB-API-041, MOB-API-042

9.25 MOB-FR-SEAT-03 — Read room map

Table 30: MOB-FR-SEAT-03 — Read room map

Field	Specification
Name	Read room map
Description	Get the seat map of a room (signed-in users).
Actor	Customer
Preconditions	Customer is signed in and the selected future showtime/room exists.
Trigger	Invoke “Read room map” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Get the seat map of a room (signed-in users). 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Remove selections newly occupied by another booking and notify the customer. Reject half sweetboxes, invalid room seats and prohibited layout edits.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-SEAT-03-AC-01** — Given the preconditions/trigger, verify: Get the seat map of a room (signed-in users). **MOB-FR-SEAT-03-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/MovieService/MovieService/Controllers/SeatsController.cs; Cinema_Mobile/lib/features/seat/screens/seat_selection_screen.dart; Cinema_BE/MovieService/MovieService/Service/Impl/SeatService.cs; Cinema_BE/MovieService/MovieService/Service/Impl/ShowtimeService.cs
Business rules	MOB-BR-04, MOB-BR-07, MOB-BR-15, MOB-BR-16, MOB-BR-26
Use case / screens / APIs	UC-MOB-03; MOB-SCR-SEATS; MOB-API-001

9.26 MOB-FR-SHOW-03 — Showtime and prices

Table 31: MOB-FR-SHOW-03 — Showtime and prices

Field	Specification
Name	Showtime and prices
Description	Showtime details with `seatPrices`: the price of one seat of each type.
Actor	Guest/Customer; Staff/Admin read details only during check-in
Preconditions	Movie and room exist; management changes need Staff/Admin.
Trigger	Invoke “Showtime and prices” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Showtime details with `seatPrices`: the price of one seat of each type. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject overlap with `409` and invalid time/format with `400`; cancelled showtimes are final. Customer selection excludes closed or started shows.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-SHOW-03-AC-01** — Given the preconditions/trigger, verify: Showtime details with `seatPrices`: the price of one seat of each type. **MOB-FR-SHOW-03-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/MovieService/MovieService/Controllers/ShowtimesController.cs; Cinema_Mobile/lib/features/movie/screens/movie_detail_screen.dart; Cinema_Mobile/lib/features/seat/screens/seat_selection_screen.dart; Cinema_BE/MovieService/MovieService/Service/Impl/ShowtimeService.cs
Business rules	MOB-BR-05, MOB-BR-06, MOB-BR-15, MOB-BR-19, MOB-BR-30
Use case / screens / APIs	UC-MOB-04; MOB-SCR-MOVIE, MOB-SCR-SEATS; MOB-API-002

9.27 MOB-FR-SHOW-06 — Open future showtimes

Table 32: MOB-FR-SHOW-06 — Open future showtimes

Field	Specification
Name	Open future showtimes
Description	Get showtimes open for sale (`OPEN`, not started).
Actor	Guest/Customer; Staff/Admin read details only during check-in
Preconditions	Movie and room exist; management changes need Staff/Admin.
Trigger	Invoke “Open future showtimes” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Get showtimes open for sale (`OPEN`, not started). 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject overlap with `409` and invalid time/format with `400`; cancelled showtimes are final. Customer selection excludes closed or started shows.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H

Continued on next page

Table 32 — continued

Field	Specification
Acceptance criteria	**MOB-FR-SHOW-06-AC-01** — Given the preconditions/trigger, verify: Get showtimes open for sale (OPEN, not started). **MOB-FR-SHOW-06-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/MovieService/MovieService/Controllers/ShowtimesController.cs; Cinema_Mobile/lib/features/movie/screens/movie_detail_screen.dart; Cinema_BE/MovieService/MovieService/Service/Impl/ShowtimeService.cs
Business rules	MOB-BR-05, MOB-BR-06, MOB-BR-15, MOB-BR-19, MOB-BR-30
Use case / screens / APIs	UC-MOB-04; MOB-SCR-MOVIE; MOB-API-003

9.28 MOB-FR-SHOW-07 — Showtimes by movie

Table 33: MOB-FR-SHOW-07 — Showtimes by movie

Field	Specification
Name	Showtimes by movie
Description	Get showtimes of a movie.
Actor	Guest/Customer; Staff/Admin read details only during check-in
Preconditions	Movie and room exist; management changes need Staff/Admin.
Trigger	Invoke “Showtimes by movie” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Get showtimes of a movie. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject overlap with 409 and invalid time/format with 400; cancelled showtimes are final. Customer selection excludes closed or started shows.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	M
Acceptance criteria	**MOB-FR-SHOW-07-AC-01** — Given the preconditions/trigger, verify: Get showtimes of a movie. **MOB-FR-SHOW-07-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/MovieService/MovieService/Controllers/ShowtimesController.cs; Cinema_Mobile/lib/features/movie/screens/movie_detail_screen.dart; Cinema_BE/MovieService/MovieService/Service/Impl/ShowtimeService.cs
Business rules	MOB-BR-05, MOB-BR-06, MOB-BR-15, MOB-BR-19, MOB-BR-30
Use case / screens / APIs	UC-MOB-04; MOB-SCR-MOVIE; MOB-API-003

9.29 MOB-FR-SHOW-08 — Bookable movie showtimes

Table 34: MOB-FR-SHOW-08 — Bookable movie showtimes

Field	Specification
Name	Bookable movie showtimes
Description	Get open showtimes of a movie; the customer app shows those in the next 7 days.

Continued on next page

Table 34 — continued

Field	Specification
Actor	Guest/Customer; Staff/Admin read details only during check-in
Preconditions	Movie and room exist; management changes need Staff/Admin.
Trigger	Invoke “Bookable movie showtimes” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Get open showtimes of a movie; the customer app shows those in the next 7 days. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject overlap with `409` and invalid time/format with `400`; cancelled showtimes are final. Customer selection excludes closed or started shows.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**`MOB-FR-SHOW-08-AC-01`** — Given the preconditions/trigger, verify: Get open showtimes of a movie; the customer app shows those in the next 7 days. **`MOB-FR-SHOW-08-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/MovieService/MovieService/Controllers/ShowtimesController.cs`; `Cinema_Mobile/lib/features/movie/screens/movie_detail_screen.dart`; `Cinema_BE/MovieService/MovieService/Service/Impl/ShowtimeService.cs`
Business rules	`MOB-BR-05`, `MOB-BR-06`, `MOB-BR-15`, `MOB-BR-19`, `MOB-BR-30`
Use case / screens / APIs	`UC-MOB-04`; `MOB-SCR-MOVIE`; `MOB-API-003`

9.30 `MOB-FR-SHOW-13` — Screening format

Table 35: `MOB-FR-SHOW-13` — Screening format

Field	Specification
Name	Screening format
Description	Display showtime format 2D/3D/`IMAX` beside the room and use server `seatPrices`. The shared backend adds 20,000 `VND` for 3D or 40,000 `VND` for `IMAX` to the 2D standard price before applying `VIP`/sweetbox surcharges. No mobile showtime editor is in scope.
Actor	Guest/Customer; Staff/Admin read details only during check-in
Preconditions	Movie and room exist; management changes need Staff/Admin.
Trigger	Invoke “Screening format” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Display showtime format 2D/3D/`IMAX` beside the room and use server `seatPrices`. The shared backend adds 20,000 `VND` for 3D or 40,000 `VND` for `IMAX` to the 2D standard price before applying `VIP`/sweetbox surcharges. No mobile showtime editor is in scope. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject overlap with `409` and invalid time/format with `400`; cancelled showtimes are final. Customer selection excludes closed or started shows.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	M

Continued on next page

Table 35 — continued

Field	Specification
Acceptance criteria	**MOB-FR-SHOW-13-AC-01** — Given the preconditions/trigger, verify: Display showtime format 2D/3D/IMAX beside the room and use server seatPrices. The shared backend adds 20,000 VND for 3D or 40,000 VND for IMAX to the 2D standard price before applying VIP/sweetbox surcharges. No mobile showtime editor is in scope. **MOB-FR-SHOW-13-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/MovieService/MovieService/Controllers/ShowtimesController.cs; Cinema_Mobile/lib/features/movie/screens/movie_detail_screen.dart; Cinema_Mobile/lib/features/seat/screens/seat_selection_screen.dart; Cinema_BE/MovieService/MovieService/Service/Impl/ShowtimeService.cs
Business rules	MOB-BR-05, MOB-BR-06, MOB-BR-15, MOB-BR-19, MOB-BR-30
Use case / screens / APIs	UC-MOB-04; MOB-SCR-MOVIE, MOB-SCR-SEATS; MOB-API-002

9.31 MOB-FR-BOOK-01 — Hold booking

Table 36: MOB-FR-BOOK-01 — Hold booking

Field	Specification
Name	Hold booking
Description	Create a booking from a showtime and seats: the showtime must be OPEN and not started, with at least one seat and no duplicates. The server takes the show time, movie title and prices from MovieService. The booking is PENDING and holds the seats for 10 minutes.
Actor	Customer
Preconditions	A signed-in customer has selected an OPEN future showtime and seats; counter guests are represented by Staff/Admin.
Trigger	Invoke “Hold booking” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Create a booking from a showtime and seats: the showtime must be OPEN and not started, with at least one seat and no duplicates. The server takes the show time, movie title and prices from MovieService. The booking is PENDING and holds the seats for 10 minutes. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns 409 and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	One PENDING booking with server total and expiresAt is committed; seats are HELD for ten minutes. A rejected transaction leaves no partial holds or point deduction.
Priority	H
Acceptance criteria	**MOB-FR-BOOK-01-AC-01** — Given the preconditions/trigger, verify: Create a booking from a showtime and seats: the showtime must be OPEN and not started, with at least one seat and no duplicates. The server takes the show time, movie title and prices from MovieService. The booking is PENDING and holds the seats for 10 minutes. **MOB-FR-BOOK-01-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/booking/screens/booking_summary_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs; Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs

Continued on next page

Table 36 — continued

Field	Specification
Business rules	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29
Use case / screens / APIs	UC-MOB-05; MOB-SCR-SUMMARY; MOB-API-021

9.32 MOB-FR-BOOK-02 — Booking details

Table 37: MOB-FR-BOOK-02 — Booking details

Field	Specification
Name	Booking details
Description	The customer opens their own booking details, with authoritative seats, amounts, status, expiry, QR/check-in and refund information when applicable; Staff/Admin lookup is limited to the check-in workflow.
Actor	Customer
Preconditions	Customer is signed in and owns the relevant ticket; the requested action must be permitted by ticket state and time. Staff access applies only where stated.
Trigger	Invoke “Booking details” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: The customer opens their own booking details, with authoritative seats, amounts, status, expiry, QR/check-in and refund information when applicable; Staff/Admin lookup is limited to the check-in workflow. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns 409 and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-BOOK-02-AC-01** — Given the preconditions/trigger, verify: The customer opens their own booking details, with authoritative seats, amounts, status, expiry, QR/check-in and refund information when applicable; Staff/Admin lookup is limited to the check-in workflow. **MOB-FR-BOOK-02-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/payment/screens/payment_result_screen.dart; Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs; Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs
Business rules	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29
Use case / screens / APIs	UC-MOB-05; MOB-SCR-RESULT, MOB-SCR-TICKET; MOB-API-017

9.33 MOB-FR-BOOK-04 — Own ticket history

Table 38: MOB-FR-BOOK-04 — Own ticket history

Field	Specification
Name	Own ticket history
Description	Customers see their own tickets in **Active** and **History**.
Actor	Customer
Preconditions	Customer is signed in and owns the relevant ticket; the requested action must be permitted by ticket state and time. Staff access applies only where stated.
Trigger	Invoke “Own ticket history” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Customers see their own tickets in **Active** and **History**. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns 409 and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-BOOK-04-AC-01** — Given the preconditions/trigger, verify: Customers see their own tickets in **Active** and **History**. **MOB-FR-BOOK-04-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/booking/screens/my_bookings_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs; Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs
Business rules	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29
Use case / screens / APIs	UC-MOB-05; MOB-SCR-TICKETS; MOB-API-020

9.34 MOB-FR-BOOK-05 — Confirm paid booking

Table 39: MOB-FR-BOOK-05 — Confirm paid booking

Field	Specification
Name	Confirm paid booking
Description	Confirm a booking after a successful payment (see PAY and POS).
Actor	Shared backend system; customer receives outcome
Preconditions	The corresponding committed event, trusted PayOS result or scheduled-worker condition exists; backend processing is configured. No customer session is required for a background worker or signed webhook.
Trigger	Invoke “Confirm paid booking” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Confirm a booking after a successful payment (see PAY and POS). 3. Return/render the authoritative result and refresh affected data after success.

Continued on next page

Table 39 — continued

Field	Specification
Alternative / exception flow	A concurrent hold returns `409` and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**`MOB-FR-BOOK-05-AC-01`** — Given the preconditions/trigger, verify: Confirm a booking after a successful payment (see `PAY` and `POS`). **`MOB-FR-BOOK-05-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs`; `Cinema_Mobile/lib/features/payment/screens/payment_screen.dart`; `Cinema_Mobile/lib/features/payment/screens/payment_result_screen.dart`; `Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart`; `Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs`; `Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs`; `Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs`
Business rules	`MOB-BR-07`, `MOB-BR-08`, `MOB-BR-14`, `MOB-BR-15`, `MOB-BR-16`, `MOB-BR-17`, `MOB-BR-27`, `MOB-BR-28`, `MOB-BR-29`
Use case / screens / APIs	`UC-MOB-05`; `MOB-SCR-PAYMENT`, `MOB-SCR-RESULT`, `MOB-SCR-TICKET`; `MOB-API-034`, `MOB-API-036`

9.35 `MOB-FR-BOOK-06` — Cancel booking

Table 40: `MOB-FR-BOOK-06` — Cancel booking

Field	Specification
Name	Cancel booking
Description	Cancel a booking: an unpaid one is cancelled and its seats released; a paid one follows `MOB-BR-17`. A customer cancelling a paid ticket creates a refund request: the booking becomes `REFUND_PENDING` and keeps its seats until staff handle it.
Actor	Customer
Preconditions	Customer is signed in and owns the relevant ticket; the requested action must be permitted by ticket state and time. Staff access applies only where stated.
Trigger	Invoke “Cancel booking” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Cancel a booking: an unpaid one is cancelled and its seats released; a paid one follows `MOB-BR-17`. A customer cancelling a paid ticket creates a refund request: the booking becomes `REFUND_PENDING` and keeps its seats until staff handle it. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns `409` and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H

Continued on next page

Table 40 — continued

Field	Specification
Acceptance criteria	**MOB-FR-BOOK-06-AC-01** — Given the preconditions/trigger, verify: Cancel a booking: an unpaid one is cancelled and its seats released; a paid one follows MOB-BR-17. A customer cancelling a paid ticket creates a refund request: the booking becomes REFUND_PENDING and keeps its seats until staff handle it. **MOB-FR-BOOK-06-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs; Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs
Business rules	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29
Use case / screens / APIs	UC-MOB-05; MOB-SCR-TICKET; MOB-API-015

9.36 MOB-FR-BOOK-07 — Occupied seat IDs

Table 41: MOB-FR-BOOK-07 — Occupied seat IDs

Field	Specification
Name	Occupied seat IDs
Description	Get the held / sold seats of a showtime.
Actor	Customer
Preconditions	A signed-in customer has selected an OPEN future showtime and seats; counter guests are represented by Staff/Admin.
Trigger	Invoke “Occupied seat IDs” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Get the held / sold seats of a showtime. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns 409 and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-BOOK-07-AC-01** — Given the preconditions/trigger, verify: Get the held / sold seats of a showtime. **MOB-FR-BOOK-07-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/seat/screens/seat_selection_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs; Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs
Business rules	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29
Use case / screens / APIs	UC-MOB-05; MOB-SCR-SEATS; MOB-API-019

9.37 MOB-FR-BOOK-08 — Concurrent seat conflict

Table 42: MOB-FR-BOOK-08 — Concurrent seat conflict

Field	Specification
Name	Concurrent seat conflict
Description	A seat held or sold by another booking returns 409.
Actor	Shared backend system; customer receives outcome
Preconditions	A signed-in customer has selected an OPEN future showtime and seats; counter guests are represented by Staff/Admin.
Trigger	Invoke “Concurrent seat conflict” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: A seat held or sold by another booking returns 409. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns 409 and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**MOB-FR-BOOK-08-AC-01** — Given the preconditions/trigger, verify: A seat held or sold by another booking returns 409. **MOB-FR-BOOK-08-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/seat/screens/seat_selection_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs; Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs
Business rules	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29
Use case / screens / APIs	UC-MOB-05; MOB-SCR-SEATS; MOB-API-021

9.38 MOB-FR-BOOK-09 — Sweetbox pair

Table 43: MOB-FR-BOOK-09 — Sweetbox pair

Field	Specification
Name	Sweetbox pair
Description	Sweetbox seats must be booked as a full pair (the server refuses half a pair).
Actor	Shared backend system; customer receives outcome
Preconditions	A signed-in customer has selected an OPEN future showtime and seats; counter guests are represented by Staff/Admin.
Trigger	Invoke “Sweetbox pair” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Sweetbox seats must be booked as a full pair (the server refuses half a pair). 3. Return/render the authoritative result and refresh affected data after success.

Continued on next page

Table 43 — continued

Field	Specification
Alternative / exception flow	A concurrent hold returns `409` and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**`MOB-FR-BOOK-09-AC-01`** — Given the preconditions/trigger, verify: Sweetbox seats must be booked as a full pair (the server refuses half a pair). **`MOB-FR-BOOK-09-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs`; `Cinema_Mobile/lib/features/seat/screens/seat_selection_screen.dart`; `Cinema_Mobile/lib/features/booking/screens/booking_summary_screen.dart`; `Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs`; `Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs`; `Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs`
Business rules	`MOB-BR-07`, `MOB-BR-08`, `MOB-BR-14`, `MOB-BR-15`, `MOB-BR-16`, `MOB-BR-17`, `MOB-BR-27`, `MOB-BR-28`, `MOB-BR-29`
Use case / screens / APIs	`UC-MOB-05`; `MOB-SCR-SEATS`, `MOB-SCR-SUMMARY`; `MOB-API-021`

9.39 `MOB-FR-BOOK-10` — Expire unpaid hold

Table 44: `MOB-FR-BOOK-10` — Expire unpaid hold

Field	Specification
Name	Expire unpaid hold
Description	A background job runs every 30 seconds, marks bookings past their hold `EXPIRED` and releases their seats.
Actor	Shared backend system; customer receives outcome
Preconditions	A signed-in customer has selected an `OPEN` future showtime and seats; counter guests are represented by Staff/Admin.
Trigger	Invoke “Expire unpaid hold” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: A background job runs every 30 seconds, marks bookings past their hold `EXPIRED` and releases their seats. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns `409` and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**`MOB-FR-BOOK-10-AC-01`** — Given the preconditions/trigger, verify: A background job runs every 30 seconds, marks bookings past their hold `EXPIRED` and releases their seats. **`MOB-FR-BOOK-10-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.

Continued on next page

Table 44 — continued

Field	Specification
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/payment/screens/payment_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs; Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs
Business rules	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29
Use case / screens / APIs	UC-MOB-05; MOB-SCR-PAYMENT; MOB-API-017

9.40 MOB-FR-BOOK-11 — Refresh customer map

Table 45: MOB-FR-BOOK-11 — Refresh customer map

Field	Specification
Name	Refresh customer map
Description	The seat picker refreshes every 5 seconds; a selected seat taken by someone else is unselected and the customer is told.
Actor	Customer
Preconditions	A signed-in customer has selected an OPEN future showtime and seats; counter guests are represented by Staff/Admin.
Trigger	Invoke “Refresh customer map” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: The seat picker refreshes every 5 seconds; a selected seat taken by someone else is unselected and the customer is told. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns 409 and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	M
Acceptance criteria	**MOB-FR-BOOK-11-AC-01** — Given the preconditions/trigger, verify: The seat picker refreshes every 5 seconds; a selected seat taken by someone else is unselected and the customer is told. **MOB-FR-BOOK-11-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/MovieService/MovieService/Controllers/SeatsController.cs; Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/seat/screens/seat_selection_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs; Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs
Business rules	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29
Use case / screens / APIs	UC-MOB-05; MOB-SCR-SEATS; MOB-API-001, MOB-API-019

9.41 MOB-FR-BOOK-13 — Concessions with tickets

Table 46: MOB-FR-BOOK-13 — Concessions with tickets

Field	Specification
Name	Concessions with tickets
Description	At booking summary choose active concessions, quantity 1–20 each and at most ten distinct items. The server prices the active menu and stores purchase name/price snapshots. Include concessions in total, payment/refund and ticket display; an inactive item is rejected with 400.
Actor	Customer
Preconditions	A signed-in customer has selected an OPEN future showtime and seats; counter guests are represented by Staff/Admin.
Trigger	Invoke “Concessions with tickets” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: At booking summary choose active concessions, quantity 1–20 each and at most ten distinct items. The server prices the active menu and stores purchase name/price snapshots. Include concessions in total, payment/refund and ticket display; an inactive item is rejected with 400. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns 409 and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	M
Acceptance criteria	**MOB-FR-BOOK-13-AC-01** — Given the preconditions/trigger, verify: At booking summary choose active concessions, quantity 1–20 each and at most ten distinct items. The server prices the active menu and stores purchase name/price snapshots. Include concessions in total, payment/refund and ticket display; an inactive item is rejected with 400. **MOB-FR-BOOK-13-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_BE/BookingService/BookingService/Controllers/ConcessionController.cs; Cinema_Mobile/lib/features/booking/screens/booking_summary_screen.dart; Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs; Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs
Business rules	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29
Use case / screens / APIs	UC-MOB-05; MOB-SCR-SUMMARY, MOB-SCR-TICKET; MOB-API-021, MOB-API-022

9.42 MOB-FR-BOOK-15 — Promotion and voucher

Table 47: MOB-FR-BOOK-15 — Promotion and voucher

Field	Specification
Name	Promotion and voucher

Continued on next page

Table 47 — continued

Field	Specification
Description	Apply one eligible entered promotion code or the best automatic promotion to tickets only (not concessions), using percentage/fixed discount and a maximum of 90% of ticket amount rounded down to 1,000 `VND`. Validate active dates, show weekday, minimum ticket count and usage limit. Show the quote in summary and savings on ticket/payment; the server recalculates on booking and counts usage only after payment. Invalid/ineligible codes return `400` with a reason.
Actor	Customer
Preconditions	A signed-in customer has selected an `OPEN` future showtime and seats; counter guests are represented by Staff/Admin.
Trigger	Invoke “Promotion and voucher” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Apply one eligible entered promotion code or the best automatic promotion to tickets only (not concessions), using percentage/fixed discount and a maximum of 90% of ticket amount rounded down to 1,000 `VND`. Validate active dates, show weekday, minimum ticket count and usage limit. Show the quote in summary and savings on ticket/payment; the server recalculates on booking and counts usage only after payment. Invalid/ineligible codes return `400` with a reason. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns `409` and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	M
Acceptance criteria	**`MOB-FR-BOOK-15-AC-01`** — Given the preconditions/trigger, verify: Apply one eligible entered promotion code or the best automatic promotion to tickets only (not concessions), using percentage/fixed discount and a maximum of 90% of ticket amount rounded down to 1,000 `VND`. Validate active dates, show weekday, minimum ticket count and usage limit. Show the quote in summary and savings on ticket/payment; the server recalculates on booking and counts usage only after payment. Invalid/ineligible codes return `400` with a reason. **`MOB-FR-BOOK-15-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs`; `Cinema_BE/BookingService/BookingService/Controllers/PromotionController.cs`; `Cinema_Mobile/lib/features/booking/screens/booking_summary_screen.dart`; `Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart`; `Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs`; `Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs`; `Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs`
Business rules	`MOB-BR-07`, `MOB-BR-08`, `MOB-BR-14`, `MOB-BR-15`, `MOB-BR-16`, `MOB-BR-17`, `MOB-BR-27`, `MOB-BR-28`, `MOB-BR-29`
Use case / screens / APIs	`UC-MOB-05`; `MOB-SCR-SUMMARY`, `MOB-SCR-TICKET`; `MOB-API-021`, `MOB-API-038`

9.43 MOB-FR-BOOK-17 — Membership points

Table 48: MOB-FR-BOOK-17 — Membership points

Field	Specification
Name	Membership points
Description	Show the customer membership card, points, tier, progress and history on Profile. Earn on paid amount at MEMBER 5%, VIP 7% (spend ≥2,000,000 VND), VVIP 10% (≥5,000,000); one point is 1,000 VND. Redeem at most 50% of amount after promotion; expiry/unpaid cancellation restores redeemed points; paid cancellation/refund restores used points, reverses earned points and spend through shared backend rules. Mobile has no manual point-adjustment screen.
Actor	Customer
Preconditions	A signed-in customer has selected an OPEN future showtime and seats; counter guests are represented by Staff/Admin.
Trigger	Invoke “Membership points” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Show the customer membership card, points, tier, progress and history on Profile. Earn on paid amount at MEMBER 5%, VIP 7% (spend ≥2,000,000 VND), VVIP 10% (≥5,000,000); one point is 1,000 VND. Redeem at most 50% of amount after promotion; expiry/unpaid cancellation restores redeemed points; paid cancellation/refund restores used points, reverses earned points and spend through shared backend rules. Mobile has no manual point-adjustment screen. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns 409 and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	M
Acceptance criteria	**MOB-FR-BOOK-17-AC-01** — Given the preconditions/trigger, verify: Show the customer membership card, points, tier, progress and history on Profile. Earn on paid amount at MEMBER 5%, VIP 7% (spend ≥2,000,000 VND), VVIP 10% (≥5,000,000); one point is 1,000 VND. Redeem at most 50% of amount after promotion; expiry/unpaid cancellation restores redeemed points; paid cancellation/refund restores used points, reverses earned points and spend through shared backend rules. Mobile has no manual point-adjustment screen. **MOB-FR-BOOK-17-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_BE/BookingService/BookingService/Controllers/LoyaltyController.cs; Cinema_Mobile/lib/features/booking/screens/booking_summary_screen.dart; Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart; Cinema_Mobile/lib/features/profile/screens/profile_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs; Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs; Cinema_BE/BookingService/BookingService/Services/Implementations/LoyaltyService.cs
Business rules	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29
Use case / screens / APIs	UC-MOB-05; MOB-SCR-SUMMARY, MOB-SCR-TICKET, MOB-SCR-PROFILE; MOB-API-021, MOB-API-023

9.44 MOB-FR-BOOK-19 — Share and calendar

Table 49: MOB-FR-BOOK-19 — Share and calendar

Field	Specification
Name	Share and calendar
Description	Paid tickets have **Share ticket** (movie, show time, seats and ticket code sent to friends through the device's share sheet; copied to the clipboard where the browser cannot share) and **Add to calendar** (a Google Calendar event in Vietnam time ending when the show ends; only for shows still to come).
Actor	Customer
Preconditions	Customer is signed in and owns the relevant ticket; the requested action must be permitted by ticket state and time. Staff access applies only where stated.
Trigger	Invoke "Share and calendar" or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Paid tickets have **Share ticket** (movie, show time, seats and ticket code sent to friends through the device's share sheet; copied to the clipboard where the browser cannot share) and **Add to calendar** (a Google Calendar event in Vietnam time ending when the show ends; only for shows still to come). 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns 409 and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	L
Acceptance criteria	**MOB-FR-BOOK-19-AC-01** — Given the preconditions/trigger, verify: Paid tickets have **Share ticket** (movie, show time, seats and ticket code sent to friends through the device's share sheet; copied to the clipboard where the browser cannot share) and **Add to calendar** (a Google Calendar event in Vietnam time ending when the show ends; only for shows still to come). **MOB-FR-BOOK-19-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/MovieService/MovieService/Controllers/ShowtimesController.cs; Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs; Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs
Business rules	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29
Use case / screens / APIs	UC-MOB-05; MOB-SCR-TICKET; MOB-API-002

9.45 MOB-FR-PAY-01 — Create/reuse PayOS link

Table 50: MOB-FR-PAY-01 — Create/reuse PayOS link

Field	Specification
Name	Create/reuse PayOS link

Continued on next page

Table 50 — continued

Field	Specification
Description	Create a PayOS payment link for a PENDING booking. One payment per booking; asking again returns the existing link. The PayOS order code is the booking id × 1,000,000 plus the six random digits that end the booking code: stable per booking and never repeating an order of another environment on the shared PayOS account. The link expires with the seat hold.
Actor	Customer; PayOS/system as external/shared dependencies
Preconditions	A payable booking exists; PayOS credentials and callback/return settings are configured for live payment.
Trigger	Invoke “Create/reuse PayOS link” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Create a PayOS payment link for a PENDING booking. One payment per booking; asking again returns the existing link. The PayOS order code is the booking id × 1,000,000 plus the six random digits that end the booking code: stable per booking and never repeating an order of another environment on the shared PayOS account. The link expires with the seat hold. 3. Return/ render the authoritative result and refresh affected data after success.
Alternative / exception flow	Never trust redirect query parameters as proof of payment. Handle cancellation, timeout and 503 without showing false success; late payment confirms only if seats can be reclaimed, otherwise creates a refund.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**MOB-FR-PAY-01-AC-01** — Given the preconditions/trigger, verify: Create a PayOS payment link for a PENDING booking. One payment per booking; asking again returns the existing link. The PayOS order code is the booking id × 1,000,000 plus the six random digits that end the booking code: stable per booking and never repeating an order of another environment on the shared PayOS account. The link expires with the seat hold. **MOB-FR-PAY-01-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs; Cinema_Mobile/lib/features/payment/screens/payment_screen.dart; Cinema_BE/ BookingService/BookingService/Services/Implementations/PaymentService.cs; Cinema_BE/BookingService/BookingService/Helpers/PayOsSignature.cs
Business rules	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20
Use case / screens / APIs	UC-MOB-06; MOB-SCR-PAYMENT; MOB-API-035

9.46 MOB-FR-PAY-02 — Verify PayOS

Table 51: MOB-FR-PAY-02 — Verify PayOS

Field	Specification
Name	Verify PayOS
Description	Verify a payment by order code (or booking id): paid at PayOS means payment SUCCESS and booking CONFIRMED; cancelled on the PayOS page means payment FAILED, booking CANCELLED and seats released at once. Calling it again is safe.
Actor	Customer; PayOS/system as external/shared dependencies
Preconditions	A payable booking exists; PayOS credentials and callback/return settings are configured for live payment.
Trigger	Invoke “Verify PayOS” or its specified backend event/worker.

Continued on next page

Table 51 — continued

Field	Specification
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Verify a payment by order code (or booking id): paid at `PayOS` means payment `SUCCESS` and booking `CONFIRMED`; cancelled on the `PayOS` page means payment `FAILED`, booking `CANCELLED` and seats released at once. Calling it again is safe. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Never trust redirect query parameters as proof of payment. Handle cancellation, timeout and `503` without showing false success; late payment confirms only if seats can be reclaimed, otherwise creates a refund.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**`MOB-FR-PAY-02-AC-01`** — Given the preconditions/trigger, verify: Verify a payment by order code (or booking id): paid at `PayOS` means payment `SUCCESS` and booking `CONFIRMED`; cancelled on the `PayOS` page means payment `FAILED`, booking `CANCELLED` and seats released at once. Calling it again is safe. **`MOB-FR-PAY-02-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs`; `Cinema_Mobile/lib/features/payment/screens/payment_screen.dart`; `Cinema_Mobile/lib/features/payment/screens/payment_result_screen.dart`; `Cinema_BE/BookingService/BookingService/Services/Implementations/PaymentService.cs`; `Cinema_BE/BookingService/BookingService/Helpers/PayOsSignature.cs`
Business rules	`MOB-BR-09`, `MOB-BR-10`, `MOB-BR-11`, `MOB-BR-12`, `MOB-BR-13`, `MOB-BR-14`, `MOB-BR-20`
Use case / screens / APIs	`UC-MOB-06`; `MOB-SCR-PAYMENT`, `MOB-SCR-RESULT`; `MOB-API-034`

9.47 `MOB-FR-PAY-03` — Payment details

Table 52: `MOB-FR-PAY-03` — Payment details

Field	Specification
Name	Payment details
Description	Show the customer their own payment amount, method and status in payment history and related ticket context; no separate mobile payment-detail route or administrative payment workflow is implemented.
Actor	Customer; `PayOS`/system as external/shared dependencies
Preconditions	A payable booking exists; `PayOS` credentials and callback/return settings are configured for live payment.
Trigger	Invoke “Payment details” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Show the customer their own payment amount, method and status in payment history and related ticket context; no separate mobile payment-detail route or administrative payment workflow is implemented. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Never trust redirect query parameters as proof of payment. Handle cancellation, timeout and `503` without showing false success; late payment confirms only if seats can be reclaimed, otherwise creates a refund.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H

Continued on next page

Table 52 — continued

Field	Specification
Acceptance criteria	**MOB-FR-PAY-03-AC-01** — Given the preconditions/trigger, verify: Show the customer their own payment amount, method and status in payment history and related ticket context; no separate mobile payment-detail route or administrative payment workflow is implemented. **MOB-FR-PAY-03-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs; Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart; Cinema_Mobile/lib/features/payment/screens/payment_history_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/PaymentService.cs; Cinema_BE/BookingService/BookingService/Helpers/PayOsSignature.cs
Business rules	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20
Use case / screens / APIs	UC-MOB-06; MOB-SCR-TICKET, MOB-SCR-PAYHISTORY; MOB-API-037

9.48 MOB-FR-PAY-04 — Own payment history

Table 53: MOB-FR-PAY-04 — Own payment history

Field	Specification
Name	Own payment history
Description	Customers see their payment history.
Actor	Customer; PayOS/system as external/shared dependencies
Preconditions	A payable booking exists; PayOS credentials and callback/return settings are configured for live payment.
Trigger	Invoke “Own payment history” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Customers see their payment history. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Never trust redirect query parameters as proof of payment. Handle cancellation, timeout and 503 without showing false success; late payment confirms only if seats can be reclaimed, otherwise creates a refund.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	M
Acceptance criteria	**MOB-FR-PAY-04-AC-01** — Given the preconditions/trigger, verify: Customers see their payment history. **MOB-FR-PAY-04-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs; Cinema_Mobile/lib/features/payment/screens/payment_history_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/PaymentService.cs; Cinema_BE/BookingService/BookingService/Helpers/PayOsSignature.cs
Business rules	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20
Use case / screens / APIs	UC-MOB-06; MOB-SCR-PAYHISTORY; MOB-API-037

9.49 MOB-FR-PAY-06 — Refund access

Table 54: MOB-FR-PAY-06 — Refund access

Field	Specification
Name	Refund access
Description	Customers request a refund by cancelling an eligible paid ticket and read the result/rejection reason. Approval, rejection and recording bank transfers belong to the web workspace, not the mobile application.
Actor	Customer; PayOS/system as external/shared dependencies
Preconditions	A payable booking exists; PayOS credentials and callback/return settings are configured for live payment.
Trigger	Invoke “Refund access” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Customers request a refund by cancelling an eligible paid ticket and read the result/rejection reason. Approval, rejection and recording bank transfers belong to the web workspace, not the mobile application. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Never trust redirect query parameters as proof of payment. Handle cancellation, timeout and 503 without showing false success; late payment confirms only if seats can be reclaimed, otherwise creates a refund.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**MOB-FR-PAY-06-AC-01** — Given the preconditions/trigger, verify: Customers request a refund by cancelling an eligible paid ticket and read the result/rejection reason. Approval, rejection and recording bank transfers belong to the web workspace, not the mobile application. **MOB-FR-PAY-06-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs; Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/PaymentService.cs; Cinema_BE/BookingService/BookingService/Helpers/PayOsSignature.cs
Business rules	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20
Use case / screens / APIs	UC-MOB-06; MOB-SCR-TICKET; MOB-API-015, MOB-API-033

9.50 MOB-FR-PAY-07 — Signed webhook

Table 55: MOB-FR-PAY-07 — Signed webhook

Field	Specification
Name	Signed webhook
Description	Receive the PayOS webhook: check the HMAC-SHA256 signature (400 when wrong); an unknown order or wrong amount returns 200 and is ignored; a repeated webhook confirms and emails nothing twice. The payer’s account (number, name, bank) is stored for refunds.
Actor	Shared backend system; customer receives outcome

Continued on next page

Table 55 — continued

Field	Specification
Preconditions	The corresponding committed event, trusted PayOS result or scheduled-worker condition exists; backend processing is configured. No customer session is required for a background worker or signed webhook.
Trigger	Invoke “Signed webhook” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Receive the PayOS webhook: check the HMAC-SHA256 signature (400 when wrong); an unknown order or wrong amount returns 200 and is ignored; a repeated webhook confirms and emails nothing twice. The payer’s account (number, name, bank) is stored for refunds. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Never trust redirect query parameters as proof of payment. Handle cancellation, timeout and 503 without showing false success; late payment confirms only if seats can be reclaimed, otherwise creates a refund.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**MOB-FR-PAY-07-AC-01** — Given the preconditions/trigger, verify: Receive the PayOS webhook: check the HMAC-SHA256 signature (400 when wrong); an unknown order or wrong amount returns 200 and is ignored; a repeated webhook confirms and emails nothing twice. The payer’s account (number, name, bank) is stored for refunds. **MOB-FR-PAY-07-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs; Cinema_BE/BookingService/BookingService/Services/Implementations/PaymentService.cs; Cinema_BE/BookingService/BookingService/Helpers/PayOsSignature.cs
Business rules	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20
Use case / screens / APIs	UC-MOB-06; ; MOB-API-036

9.51 MOB-FR-PAY-08 — Late payment

Table 56: MOB-FR-PAY-08 — Late payment

Field	Specification
Name	Late payment
Description	When PayOS reports payment after the hold expired, confirm the booking only if all seats can be reclaimed; otherwise create a full refund request. Repeated verification/webhook must not duplicate financial effects.
Actor	Shared backend system; customer receives outcome
Preconditions	The corresponding committed event, trusted PayOS result or scheduled-worker condition exists; backend processing is configured. No customer session is required for a background worker or signed webhook.
Trigger	Invoke “Late payment” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: When PayOS reports payment after the hold expired, confirm the booking only if all seats can be reclaimed; otherwise create a full refund request. Repeated verification/webhook must not duplicate financial effects. 3. Return/render the authoritative result and refresh affected data after success.

Continued on next page

Table 56 — continued

Field	Specification
Alternative / exception flow	Never trust redirect query parameters as proof of payment. Handle cancellation, timeout and `503` without showing false success; late payment confirms only if seats can be reclaimed, otherwise creates a refund.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	M
Acceptance criteria	**`MOB-FR-PAY-08-AC-01`** — Given the preconditions/trigger, verify: When `PayOS` reports payment after the hold expired, confirm the booking only if all seats can be reclaimed; otherwise create a full refund request. Repeated verification/webhook must not duplicate financial effects. **`MOB-FR-PAY-08-AC-02`** — Exercise applicable invalid/unauthorized/ conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs`; `Cinema_Mobile/lib/features/payment/screens/payment_result_screen.dart`; `Cinema_BE/BookingService/BookingService/Services/Implementations/Payment Service.cs`; `Cinema_BE/BookingService/BookingService/Helpers/PayOsSignature. cs`
Business rules	`MOB-BR-09`, `MOB-BR-10`, `MOB-BR-11`, `MOB-BR-12`, `MOB-BR-13`, `MOB-BR-14`, `MOB-BR-20`
Use case / screens / APIs	`UC-MOB-06`; `MOB-SCR-RESULT`; `MOB-API-034`, `MOB-API-036`

9.52 `MOB-FR-PAY-11` — Automatic payment polling

Table 57: `MOB-FR-PAY-11` — Automatic payment polling

Field	Specification
Name	Automatic payment polling
Description	While open, the app’s payment page checks the booking every 5 seconds and asks `PayOS` every 30 seconds; there is no ”I have paid” button.
Actor	Customer; `PayOS`/system as external/shared dependencies
Preconditions	A payable booking exists; `PayOS` credentials and callback/return settings are configured for live payment.
Trigger	Invoke “Automatic payment polling” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: While open, the app’s payment page checks the booking every 5 seconds and asks `PayOS` every 30 seconds; there is no ”I have paid” button. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Never trust redirect query parameters as proof of payment. Handle cancellation, timeout and `503` without showing false success; late payment confirms only if seats can be reclaimed, otherwise creates a refund.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	M
Acceptance criteria	**`MOB-FR-PAY-11-AC-01`** — Given the preconditions/trigger, verify: While open, the app’s payment page checks the booking every 5 seconds and asks `PayOS` every 30 seconds; there is no ”I have paid” button. **`MOB-FR-PAY-11-AC-02`** — Exercise applicable invalid/unauthorized/ conflicting input and verify the stated rejection, no unauthorized changes and no false success.

Continued on next page

Table 57 — continued

Field	Specification
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs; Cinema_Mobile/lib/features/payment/screens/payment_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/PaymentService.cs; Cinema_BE/BookingService/BookingService/Helpers/PayOsSignature.cs
Business rules	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20
Use case / screens / APIs	UC-MOB-06; MOB-SCR-PAYMENT; MOB-API-017, MOB-API-034

9.53 MOB-FR-REF-01 — Create refund request

Table 58: MOB-FR-REF-01 — Create refund request

Field	Specification
Name	Create refund request
Description	The shared backend creates one PENDING refund and sets payment REFUND_PENDING for an eligible customer cancellation, a cinema cancellation or an unrecoverable late payment. The mobile customer sees the resulting request and notification; the phone does not initiate an administrative refund.
Actor	Customer
Preconditions	The corresponding committed event, trusted PayOS result or scheduled-worker condition exists; backend processing is configured. No customer session is required for a background worker or signed webhook.
Trigger	Invoke “Create refund request” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: The shared backend creates one PENDING refund and sets payment REFUND_PENDING for an eligible customer cancellation, a cinema cancellation or an unrecoverable late payment. The mobile customer sees the resulting request and notification; the phone does not initiate an administrative refund. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A cancellation inside two hours is refused with 409. Display backend refund/rejection states; the phone provides no approve/reject/transfer-record action.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**MOB-FR-REF-01-AC-01** — Given the preconditions/trigger, verify: The shared backend creates one PENDING refund and sets payment REFUND_PENDING for an eligible customer cancellation, a cinema cancellation or an unrecoverable late payment. The mobile customer sees the resulting request and notification; the phone does not initiate an administrative refund. **MOB-FR-REF-01-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs; Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/RefundService.cs; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs
Business rules	MOB-BR-17, MOB-BR-18, MOB-BR-19, MOB-BR-20
Use case / screens / APIs	UC-MOB-07; MOB-SCR-TICKET; MOB-API-015, MOB-API-036

9.54 MOB-FR-REF-05 — Reopen rejected refund

Table 59: MOB-FR-REF-05 — Reopen rejected refund

Field	Specification
Name	Reopen rejected refund
Description	A rejected request is reopened if the cinema later cancels the ticket or the showtime.
Actor	Shared backend system; customer receives outcome
Preconditions	The corresponding committed event, trusted PayOS result or scheduled-worker condition exists; backend processing is configured. No customer session is required for a background worker or signed webhook.
Trigger	Invoke “Reopen rejected refund” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: A rejected request is reopened if the cinema later cancels the ticket or the showtime. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A cancellation inside two hours is refused with 409. Display backend refund/rejection states; the phone provides no approve/reject/transfer-record action.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	M
Acceptance criteria	**MOB-FR-REF-05-AC-01** — Given the preconditions/trigger, verify: A rejected request is reopened if the cinema later cancels the ticket or the showtime. **MOB-FR-REF-05-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/RefundService.cs; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs
Business rules	MOB-BR-17, MOB-BR-18, MOB-BR-19, MOB-BR-20
Use case / screens / APIs	UC-MOB-07; MOB-SCR-TICKET; MOB-API-015

9.55 MOB-FR-REF-06 — Own refund status

Table 60: MOB-FR-REF-06 — Own refund status

Field	Specification
Name	Own refund status
Description	Customers see the refund status of their ticket.
Actor	Customer
Preconditions	An owned booking exists; a paid customer cancellation must be at least two hours before showtime.
Trigger	Invoke “Own refund status” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Customers see the refund status of their ticket. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A cancellation inside two hours is refused with 409. Display backend refund/rejection states; the phone provides no approve/reject/transfer-record action.

Continued on next page

Table 60 — continued

Field	Specification
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	M
Acceptance criteria	**MOB-FR-REF-06-AC-01** — Given the preconditions/trigger, verify: Customers see the refund status of their ticket. **MOB-FR-REF-06-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs`; `Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart`; `Cinema_BE/BookingService/BookingService/Services/Implementations/RefundService.cs`; `Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs`
Business rules	`MOB-BR-17`, `MOB-BR-18`, `MOB-BR-19`, `MOB-BR-20`
Use case / screens / APIs	`UC-MOB-07`; `MOB-SCR-TICKET`; `MOB-API-033`

9.56 `MOB-FR-REF-07` — Unique refund per payment

Table 61: `MOB-FR-REF-07` — Unique refund per payment

Field	Specification
Name	Unique refund per payment
Description	Each payment has only one refund request, even when an event is delivered again.
Actor	Shared backend system; customer receives outcome
Preconditions	The corresponding committed event, trusted `PayOS` result or scheduled-worker condition exists; backend processing is configured. No customer session is required for a background worker or signed webhook.
Trigger	Invoke “Unique refund per payment” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Each payment has only one refund request, even when an event is delivered again. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A cancellation inside two hours is refused with `409`. Display backend refund/rejection states; the phone provides no approve/reject/transfer-record action.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**MOB-FR-REF-07-AC-01** — Given the preconditions/trigger, verify: Each payment has only one refund request, even when an event is delivered again. **MOB-FR-REF-07-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs`; `Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs`; `Cinema_BE/BookingService/BookingService/Services/Implementations/RefundService.cs`; `Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs`
Business rules	`MOB-BR-17`, `MOB-BR-18`, `MOB-BR-19`, `MOB-BR-20`
Use case / screens / APIs	`UC-MOB-07`; ; `MOB-API-015`, `MOB-API-033`

9.57 MOB-FR-CHK-01 — Ticket QR

Table 62: MOB-FR-CHK-01 — Ticket QR

Field	Specification
Name	Ticket QR
Description	Show a paid customer ticket with a QR containing its booking code, suitable for staff lookup; confirmation email is a shared-backend output.
Actor	Customer
Preconditions	Staff/Admin is signed in; the ticket is CONFIRMED, unused and within 180 minutes before to 120 minutes after start.
Trigger	Invoke “Ticket QR” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Show a paid customer ticket with a QR containing its booking code, suitable for staff lookup; confirmation email is a shared-backend output. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Keep manual entry available if camera is unavailable/denied. Unknown code returns 404; unpaid, cancelled, pending refund, out-of-window or already-used ticket returns 409. Concurrent second check-in fails. Optional metadata failure must not hide the ticket.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-CHK-01-AC-01** — Given the preconditions/trigger, verify: Show a paid customer ticket with a QR containing its booking code, suitable for staff lookup; confirmation email is a shared-backend output. **MOB-FR-CHK-01-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Configuration/CheckInOptions.cs
Business rules	MOB-BR-21
Use case / screens / APIs	UC-MOB-08; MOB-SCR-TICKET; MOB-API-017

9.58 MOB-FR-CHK-02 — Lookup ticket code

Table 63: MOB-FR-CHK-02 — Lookup ticket code

Field	Specification
Name	Lookup ticket code
Description	Staff/Admin look a ticket up by code (case-insensitive): movie, time, room, seats, total, status.
Actor	Staff/Admin
Preconditions	Staff/Admin is signed in; the ticket is CONFIRMED, unused and within 180 minutes before to 120 minutes after start.
Trigger	Invoke “Lookup ticket code” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Staff/Admin look a ticket up by code (case-insensitive): movie, time, room, seats, total, status. 3. Return/render the authoritative result and refresh affected data after success.

Continued on next page

Table 63 — continued

Field	Specification
Alternative / exception flow	Keep manual entry available if camera is unavailable/denied. Unknown code returns `404`; unpaid, cancelled, pending refund, out-of-window or already-used ticket returns `409`. Concurrent second check-in fails. Optional metadata failure must not hide the ticket.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**`MOB-FR-CHK-02-AC-01`** — Given the preconditions/trigger, verify: Staff/Admin look a ticket up by code (case-insensitive): movie, time, room, seats, total, status. **`MOB-FR-CHK-02-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs`; `Cinema_Mobile/lib/features/staff/screens/staff_check_in_screen.dart`; `Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs`; `Cinema_BE/BookingService/BookingService/Configuration/CheckInOptions.cs`
Business rules	`MOB-BR-21`
Use case / screens / APIs	`UC-MOB-08`; `MOB-SCR-CHECKIN`; `MOB-API-018`

9.59 `MOB-FR-CHK-03` — Check-in validation

Table 64: `MOB-FR-CHK-03` — Check-in validation

Field	Specification
Name	Check-in validation
Description	Check-in follows `MOB-BR-21`; otherwise `409` with the reason (unpaid, cancelled, refund pending, too early, too late, already checked in at a given time). When two counters scan the same ticket, the second gets `409`.
Actor	Staff/Admin
Preconditions	Staff/Admin is signed in; the ticket is `CONFIRMED`, unused and within 180 minutes before to 120 minutes after start.
Trigger	Invoke “Check-in validation” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Check-in follows `MOB-BR-21`; otherwise `409` with the reason (unpaid, cancelled, refund pending, too early, too late, already checked in at a given time). When two counters scan the same ticket, the second gets `409`. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Keep manual entry available if camera is unavailable/denied. Unknown code returns `404`; unpaid, cancelled, pending refund, out-of-window or already-used ticket returns `409`. Concurrent second check-in fails. Optional metadata failure must not hide the ticket.
Postconditions	Only one check-in timestamp is stored; check-in is an attribute, not another booking status.
Priority	H
Acceptance criteria	**`MOB-FR-CHK-03-AC-01`** — Given the preconditions/trigger, verify: Check-in follows `MOB-BR-21`; otherwise `409` with the reason (unpaid, cancelled, refund pending, too early, too late, already checked in at a given time). When two counters scan the same ticket, the second gets `409`. **`MOB-FR-CHK-03-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.

Continued on next page

Table 64 — continued

Field	Specification
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/staff/screens/staff_check_in_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Configuration/CheckInOptions.cs
Business rules	MOB-BR-21
Use case / screens / APIs	UC-MOB-08; MOB-SCR-CHECKIN; MOB-API-016

9.60 MOB-FR-CHK-04 — Camera/manual scanner

Table 65: MOB-FR-CHK-04 — Camera/manual scanner

Field	Specification
Name	Camera/manual scanner
Description	Staff/Admin scan a ticket QR with the phone camera or type/paste its code and submit. Camera denial/unavailability must leave manual lookup usable. This does not expose web counter or management screens.
Actor	Staff/Admin
Preconditions	Staff/Admin is signed in; the ticket is CONFIRMED, unused and within 180 minutes before to 120 minutes after start.
Trigger	Invoke “Camera/manual scanner” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Staff/Admin scan a ticket QR with the phone camera or type/paste its code and submit. Camera denial/unavailability must leave manual lookup usable. This does not expose web counter or management screens. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Keep manual entry available if camera is unavailable/denied. Unknown code returns 404; unpaid, cancelled, pending refund, out-of-window or already-used ticket returns 409. Concurrent second check-in fails. Optional metadata failure must not hide the ticket.
Postconditions	Only one check-in timestamp is stored; check-in is an attribute, not another booking status.
Priority	H
Acceptance criteria	**MOB-FR-CHK-04-AC-01** — Given the preconditions/trigger, verify: Staff/Admin scan a ticket QR with the phone camera or type/paste its code and submit. Camera denial/unavailability must leave manual lookup usable. This does not expose web counter or management screens. **MOB-FR-CHK-04-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/profile/screens/management_account_screen.dart; Cinema_Mobile/lib/features/staff/screens/staff_check_in_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Configuration/CheckInOptions.cs
Business rules	MOB-BR-21
Use case / screens / APIs	UC-MOB-08; MOB-SCR-MGT-LANDING, MOB-SCR-CHECKIN; MOB-API-016, MOB-API-018

9.61 MOB-FR-CHK-05 — Display check-in time

Table 66: MOB-FR-CHK-05 — Display check-in time

Field	Specification
Name	Display check-in time
Description	Show checkedInAt on the customer ticket and on the mobile check-in result; after successful admission prepare the code field for the next ticket.
Actor	Staff/Admin
Preconditions	Staff/Admin is signed in; the ticket is CONFIRMED, unused and within 180 minutes before to 120 minutes after start.
Trigger	Invoke “Display check-in time” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Show checkedInAt on the customer ticket and on the mobile check-in result; after successful admission prepare the code field for the next ticket. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Keep manual entry available if camera is unavailable/denied. Unknown code returns 404; unpaid, cancelled, pending refund, out-of-window or already-used ticket returns 409. Concurrent second check-in fails. Optional metadata failure must not hide the ticket.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	M
Acceptance criteria	**MOB-FR-CHK-05-AC-01** — Given the preconditions/trigger, verify: Show checkedInAt on the customer ticket and on the mobile check-in result; after successful admission prepare the code field for the next ticket. **MOB-FR-CHK-05-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs; Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart; Cinema_Mobile/lib/features/staff/screens/staff_check_in_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Configuration/CheckInOptions.cs
Business rules	MOB-BR-21
Use case / screens / APIs	UC-MOB-08; MOB-SCR-TICKET, MOB-SCR-CHECKIN; MOB-API-017, MOB-API-018

9.62 MOB-FR-NOTI-01 — Notification history

Table 67: MOB-FR-NOTI-01 — Notification history

Field	Specification
Name	Notification history
Description	Customers list and open their own notifications and navigate to the related ticket. Mobile does not list every user’s notifications.
Actor	Customer; backend system produces emails/reminders
Preconditions	An account session exists for in-app history; SMTP/event processing is configured for actual delivery.
Trigger	Invoke “Notification history” or its specified backend event/worker.

Continued on next page

Table 67 — continued

Field	Specification
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Customers list and open their own notifications and navigate to the related ticket. Mobile does not list every user's notifications. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Owner checks reject another user's data; show loading/empty/error-retry states. Sending failure retains `FAILED` for retry and must not undo payment. Event deduplication reduces duplicates but `SMTP` plus database commit is not a guaranteed exactly-once transaction.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	M
Acceptance criteria	**`MOB-FR-NOTI-01-AC-01`** — Given the preconditions/trigger, verify: Customers list and open their own notifications and navigate to the related ticket. Mobile does not list every user's notifications. **`MOB-FR-NOTI-01-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/BookingService/BookingService/Controllers/NotificationController.cs`; `Cinema_Mobile/lib/features/notification/screens/notification_list_screen.dart`; `Cinema_BE/BookingService/BookingService/Services/Implementations/NotificationService.cs`; `Cinema_BE/BookingService/BookingService/Workers/ShowtimeReminderWorker.cs`
Business rules	`MOB-BR-11`, `MOB-BR-13`
Use case / screens / APIs	`UC-MOB-09`; `MOB-SCR-NOTIFICATIONS`; `MOB-API-032`

9.63 `MOB-FR-NOTI-04` — Notification details

Table 68: `MOB-FR-NOTI-04` — Notification details

Field	Specification
Name	Notification details
Description	Show a notification.
Actor	Customer; backend system produces emails/reminders
Preconditions	An account session exists for in-app history; `SMTP`/event processing is configured for actual delivery.
Trigger	Invoke "Notification details" or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Show a notification. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Owner checks reject another user's data; show loading/empty/error-retry states. Sending failure retains `FAILED` for retry and must not undo payment. Event deduplication reduces duplicates but `SMTP` plus database commit is not a guaranteed exactly-once transaction.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	M
Acceptance criteria	**`MOB-FR-NOTI-04-AC-01`** — Given the preconditions/trigger, verify: Show a notification. **`MOB-FR-NOTI-04-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.

Continued on next page

Table 68 — continued

Field	Specification
Evidence	Cinema_BE/BookingService/BookingService/Controllers/NotificationController.cs; Cinema_Mobile/lib/features/notification/screens/notification_detail_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/NotificationService.cs; Cinema_BE/BookingService/BookingService/Workers/ShowtimeReminderWorker.cs
Business rules	MOB-BR-11, MOB-BR-13
Use case / screens / APIs	UC-MOB-09; MOB-SCR-NOTIFICATION; MOB-API-029

9.64 MOB-FR-NOTI-06 — Event emails

Table 69: MOB-FR-NOTI-06 — Event emails

Field	Specification
Name	Event emails
Description	Consume committed booking/payment events to send confirmation with inline QR, cancellation, expiry and refund emails. Deduplicate by event ID and skip tickets without email. Delivery is retryable; SMTP and database commit do not guarantee absolute exactly-once delivery after a crash.
Actor	Shared backend system; customer receives outcome
Preconditions	The corresponding committed event, trusted PayOS result or scheduled-worker condition exists; backend processing is configured. No customer session is required for a background worker or signed webhook.
Trigger	Invoke “Event emails” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Consume committed booking/payment events to send confirmation with inline QR, cancellation, expiry and refund emails. Deduplicate by event ID and skip tickets without email. Delivery is retryable; SMTP and database commit do not guarantee absolute exactly-once delivery after a crash. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Owner checks reject another user’s data; show loading/empty/error-retry states. Sending failure retains FAILED for retry and must not undo payment. Event deduplication reduces duplicates but SMTP plus database commit is not a guaranteed exactly-once transaction.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	H
Acceptance criteria	**MOB-FR-NOTI-06-AC-01** — Given the preconditions/trigger, verify: Consume committed booking/payment events to send confirmation with inline QR, cancellation, expiry and refund emails. Deduplicate by event ID and skip tickets without email. Delivery is retryable; SMTP and database commit do not guarantee absolute exactly-once delivery after a crash. **MOB-FR-NOTI-06-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/NotificationController.cs; Cinema_BE/BookingService/BookingService/Services/Implementations/NotificationService.cs; Cinema_BE/BookingService/BookingService/Workers/ShowtimeReminderWorker.cs
Business rules	MOB-BR-11, MOB-BR-13
Use case / screens / APIs	UC-MOB-09; ; MOB-API-032

9.65 MOB-FR-NOTI-07 — OTP and welcome email

Table 70: MOB-FR-NOTI-07 — OTP and welcome email

Field	Specification
Name	OTP and welcome email
Description	AuthService emails OTP codes (verification, password reset) and a welcome email.
Actor	Shared backend system; customer receives outcome
Preconditions	The corresponding committed event, trusted PayOS result or scheduled-worker condition exists; backend processing is configured. No customer session is required for a background worker or signed webhook.
Trigger	Invoke "OTP and welcome email" or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Auth Service emails OTP codes (verification, password reset) and a welcome email. 3. Return/ render the authoritative result and refresh affected data after success.
Alternative / exception flow	Owner checks reject another user's data; show loading/empty/error-retry states. Sending failure retains FAILED for retry and must not undo payment. Event deduplication reduces duplicates but SMTP plus database commit is not a guaranteed exactly-once transaction.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**MOB-FR-NOTI-07-AC-01** — Given the preconditions/trigger, verify: AuthService emails OTP codes (verification, password reset) and a welcome email. **MOB-FR-NOTI-07-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs; Cinema_BE/ BookingService/BookingService/Services/Implementations/NotificationService. cs; Cinema_BE/BookingService/BookingService/Workers/ShowtimeReminderWorker. cs
Business rules	MOB-BR-11, MOB-BR-13
Use case / screens / APIs	UC-MOB-09; ; MOB-API-005, MOB-API-011, MOB-API-014

9.66 MOB-FR-NOTI-08 — Showtime reminder

Table 71: MOB-FR-NOTI-08 — Showtime reminder

Field	Specification
Name	Showtime reminder
Description	**Showtime reminders**: a background job (every minute, ShowtimeReminder settings) emails and records a notification for paid tickets not checked in whose show starts within 2 hours; each ticket once. Tickets bought inside that window are skipped (the customer just got the confirmation).
Actor	Shared backend system; customer receives outcome
Preconditions	The corresponding committed event, trusted PayOS result or scheduled-worker condition exists; backend processing is configured. No customer session is required for a background worker or signed webhook.
Trigger	Invoke "Showtime reminder" or its specified backend event/worker.

Continued on next page

Table 71 — continued

Field	Specification
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: **Showtime reminders**: a background job (every minute, `ShowtimeReminder` settings) emails and records a notification for paid tickets not checked in whose show starts within 2 hours; each ticket once. Tickets bought inside that window are skipped (the customer just got the confirmation). 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Owner checks reject another user's data; show loading/empty/error-retry states. Sending failure retains `FAILED` for retry and must not undo payment. Event deduplication reduces duplicates but `SMTP` plus database commit is not a guaranteed exactly-once transaction.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	M
Acceptance criteria	**`MOB-FR-NOTI-08-AC-01`** — Given the preconditions/trigger, verify: **Showtime reminders**: a background job (every minute, `ShowtimeReminder` settings) emails and records a notification for paid tickets not checked in whose show starts within 2 hours; each ticket once. Tickets bought inside that window are skipped (the customer just got the confirmation). **`MOB-FR-NOTI-08-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/BookingService/BookingService/Controllers/NotificationController.cs`; `Cinema_BE/BookingService/BookingService/Services/Implementations/NotificationService.cs`; `Cinema_BE/BookingService/BookingService/Workers/ShowtimeReminderWorker.cs`
Business rules	`MOB-BR-11`, `MOB-BR-13`
Use case / screens / APIs	`UC-MOB-09`; ; `MOB-API-032`

9.67 `MOB-FR-NOTI-09` — Read/unread notifications

Table 72: `MOB-FR-NOTI-09` — Read/unread notifications

Field	Specification
Name	Read/unread notifications
Description	Show an unread dot/bold title and unread-count badge; opening detail marks that notification read, and Mark all read updates the customer's notifications. Only the recipient may mark them (`403` otherwise).
Actor	Customer; backend system produces emails/reminders
Preconditions	An account session exists for in-app history; `SMTP`/event processing is configured for actual delivery.
Trigger	Invoke "Read/unread notifications" or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Show an unread dot/bold title and unread-count badge; opening detail marks that notification read, and Mark all read updates the customer's notifications. Only the recipient may mark them (`403` otherwise). 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Owner checks reject another user's data; show loading/empty/error-retry states. Sending failure retains `FAILED` for retry and must not undo payment. Event deduplication reduces duplicates but `SMTP` plus database commit is not a guaranteed exactly-once transaction.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	M

Continued on next page

Table 72 — continued

Field	Specification
Acceptance criteria	**MOB-FR-NOTI-09-AC-01** — Given the preconditions/trigger, verify: Show an unread dot/bold title and unread-count badge; opening detail marks that notification read, and Mark all read updates the customer's notifications. Only the recipient may mark them (403 otherwise). **MOB-FR-NOTI-09-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Controllers/NotificationController.cs; Cinema_Mobile/lib/features/home/screens/home_screen.dart; Cinema_Mobile/lib/features/notification/screens/notification_list_screen.dart; Cinema_Mobile/lib/features/notification/screens/notification_detail_screen.dart; Cinema_BE/BookingService/BookingService/Services/Implementations/NotificationService.cs; Cinema_BE/BookingService/BookingService/Workers/ShowtimeReminderWorker.cs
Business rules	MOB-BR-11, MOB-BR-13
Use case / screens / APIs	UC-MOB-09; MOB-SCR-HOME, MOB-SCR-NOTIFICATIONS, MOB-SCR-NOTIFICATION; MOB-API-028, MOB-API-030, MOB-API-031

9.68 MOB-FR-NAV-01 — Route guards and recovery

Table 73: MOB-FR-NAV-01 — Route guards and recovery

Field	Specification
Name	Route guards and recovery
Description	Guard named routes: a signed-out protected destination opens login with its pending route; successful sign-in resumes an allowed destination. Unknown routes fall back to the role landing. A summary lacking its in-memory BookingDraft returns to seat selection.
Actor	Guest; signed-in account; system
Preconditions	The account form is reachable and gateway/AuthService are configured; no prior signed-in session is required for registration/login/OTP/recovery. SMTP is required when sending a code.
Trigger	Invoke "Route guards and recovery" or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Guard named routes: a signed-out protected destination opens login with its pending route; successful sign-in resumes an allowed destination. Unknown routes fall back to the role landing. A summary lacking its in-memory BookingDraft returns to seat selection. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Reject invalid fields (400), bad credentials (401), unverified email (403), stale/incorrect OTP, or resend/rate limits (429). Clear the session for account-lock/role-change responses; show SMTP/dependency failure (503).
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	H
Acceptance criteria	**MOB-FR-NAV-01-AC-01** — Given the preconditions/trigger, verify: Guard named routes: a signed-out protected destination opens login with its pending route; successful sign-in resumes an allowed destination. Unknown routes fall back to the role landing. A summary lacking its in-memory BookingDraft returns to seat selection. **MOB-FR-NAV-01-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_Mobile/lib/features/auth/screens/login_screen.dart; Cinema_BE/AuthService/AuthService/Service/AuthService.cs; Cinema_BE/ApiGateway/ApiGateway/Program.cs

Continued on next page

Table 73 — continued

Field	Specification
Business rules	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25
Use case / screens / APIs	UC-MOB-01; MOB-SCR-LOGIN; Client behaviour; no dedicated endpoint

9.69 MOB-FR-STATE-01 — Loading, error and empty states

Table 74: MOB-FR-STATE-01 — Loading, error and empty states

Field	Specification
Name	Loading, error and empty states
Description	Provide loading, error/retry and empty states on data screens; disable repeated submission while a mutation is running and preserve an honest pending/failed result when the network fails.
Actor	Customer
Preconditions	A data screen starts loading or a request is in progress.
Trigger	Invoke “Loading, error and empty states” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: Provide loading, error/retry and empty states on data screens; disable repeated submission while a mutation is running and preserve an honest pending/failed result when the network fails. 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	A concurrent hold returns 409 and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	The successful observable result matches this requirement; any specified state change is persisted. Rejection does not grant access or leave an unauthorized partial change.
Priority	M
Acceptance criteria	**MOB-FR-STATE-01-AC-01** — Given the preconditions/trigger, verify: Provide loading, error/retry and empty states on data screens; disable repeated submission while a mutation is running and preserve an honest pending/failed result when the network fails. **MOB-FR-STATE-01-AC-02** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	Cinema_BE/BookingService/BookingService/Services/Implementations/BookingService.cs; Cinema_BE/BookingService/BookingService/Helpers/PromotionRules.cs; Cinema_BE/BookingService/BookingService/Helpers/LoyaltyRules.cs
Business rules	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29
Use case / screens / APIs	UC-MOB-05; ; Client behaviour; no dedicated endpoint

9.70 MOB-FR-MEDIA-01 — Platform-specific movie media

Table 75: MOB-FR-MEDIA-01 — Platform-specific movie media

Field	Specification
Name	Platform-specific movie media

Continued on next page

Table 75 — continued

Field	Specification
Description	On native mobile retain movie posters and open `trailerUrl` with an external application via url_launcher; web hover/autoplay preview is not implemented on native platforms (`supports TrailerPreview`=false).
Actor	Admin
Preconditions	An Admin session is active; a target record exists for edit/deactivate/adjust operations.
Trigger	Invoke “Platform-specific movie media” or its specified backend event/worker.
Main flow	1. Read current input/resource and the applicable session. 2. Execute this operation: On native mobile retain movie posters and open `trailerUrl` with an external application via url_launcher; web hover/autoplay preview is not implemented on native platforms (`supports TrailerPreview`=false). 3. Return/render the authoritative result and refresh affected data after success.
Alternative / exception flow	Show empty results for no matching films/showtimes. Review of another user’s ticket returns `404`; a ticket not checked in returns `400`. Failed media must retain a poster/fallback.
Postconditions	The requested authorized data/state is returned or displayed; a read does not alter booking/payment state. Notification read markers are changed only by their separate write operation.
Priority	L
Acceptance criteria	**`MOB-FR-MEDIA-01-AC-01`** — Given the preconditions/trigger, verify: On native mobile retain movie posters and open `trailerUrl` with an external application via url_launcher; web hover/autoplay preview is not implemented on native platforms (`supportsTrailer Preview`=false). **`MOB-FR-MEDIA-01-AC-02`** — Exercise applicable invalid/unauthorized/conflicting input and verify the stated rejection, no unauthorized changes and no false success.
Evidence	`Cinema_BE/MovieService/MovieService/Controllers/MoviesController.cs`; `Cinema_Mobile/lib/features/home/screens/home_screen.dart`; `Cinema_BE/Movie Service/MovieService/Service/Impl/MovieService.cs`; `Cinema_BE/Booking Service/BookingService/Services/Implementations/MovieReviewService.cs`
Business rules	`MOB-BR-26`
Use case / screens / APIs	`UC-MOB-02`; `MOB-SCR-HOME`; `MOB-API-024`

10. Use Case Specifications

10.1 UC-MOB-01 — Authentication and account session

Table 76: UC-MOB-01 — Authentication and account session

Field	Use-case specification
Actor	Guest; signed-in account; system
Goal	Authentication and account session
Preconditions	The gateway is configured. Registration/recovery needs SMTP; protected operations require a valid account session.
Trigger	Open the relevant screen or invoke the stated operation/event.
Main flow	1. Enter account fields or request the session operation. 2. Validate fields; submit to AuthService through the gateway. 3. For verification/recovery, enter the emailed OTP; for sign-in/refresh, store returned tokens and user. 4. Route by role and resume an allowed pending destination.
Alternative / exception flow	Reject invalid fields (400), bad credentials (401), unverified email (403), stale/incorrect OTP, or resend/rate limits (429). Clear the session for account-lock/role-change responses; show SMTP/ dependency failure (503).
Postconditions	Only a successful token-issuing operation authenticates the account; rejected requests do not grant access.
Requirements / priority	MOB-FR-AUTH-01 (H), MOB-FR-AUTH-02 (H), MOB-FR-AUTH-03 (H), MOB-FR-AUTH-04 (H), MOB-FR-AUTH-05 (H), MOB-FR-AUTH-06 (H), MOB-FR-AUTH-07 (H), MOB-FR-AUTH-08 (M), MOB-FR-AUTH-09 (H), MOB-FR-AUTH-10 (H), MOB-FR-AUTH-11 (M), MOB-FR-AUTH-12 (H), MOB-FR-AUTH-13 (H), MOB-FR-AUTH-14 (M), MOB-FR-AUTH-16 (M), MOB-FR-AUTH-17 (M), MOB-FR-NAV-01 (H)
Acceptance	Execute AC-01/AC-02 for every included FR with permitted actors.

10.2 UC-MOB-02 — Movie catalogue and reviews

Table 77: UC-MOB-02 — Movie catalogue and reviews

Field	Use-case specification
Actor	Guest/Customer
Goal	Movie catalogue and reviews
Preconditions	A movie is available; changes need Admin; a review needs the owner’s checked-in ticket.
Trigger	Open the relevant screen or invoke the stated operation/event.
Main flow	1. Browse now showing/coming soon, search by title and filter genre. 2. Open metadata, external trailer and seven-day showtimes; read review summary. 3. From an owned checked-in ticket, submit/edit a 1–5 star review (max 1,000 characters) or delete it. 4. Refresh affected data.
Alternative / exception flow	Show empty results for no matching films/showtimes. Review of another user’s ticket returns 404; a ticket not checked in returns 400. Failed media must retain a poster/fallback.
Postconditions	The catalogue/review reflects the authoritative response; deactivation preserves existing tickets.
Requirements / priority	MOB-FR-MOV-02 (H), MOB-FR-MOV-03 (M), MOB-FR-MOV-04 (H), MOB-FR-MOV-08 (L), MOB-FR-MOV-09 (H), MOB-FR-MOV-10 (M), MOB-FR-MOV-11 (L), MOB-FR-MOV-12 (M), MOB-FR-MEDIA-01 (L)
Acceptance	Execute AC-01/AC-02 for every included FR with permitted actors.

10.3 UC-MOB-03 — Seat map and availability

Table 78: UC-MOB-03 — Seat map and availability

Field	Use-case specification
Actor	Customer
Goal	Seat map and availability
Preconditions	Customer is signed in and the selected future showtime/room exists.
Trigger	Open the relevant screen or invoke the stated operation/event.
Main flow	1. Load room seats, showtime prices and occupied IDs. 2. Select free seats, adding/removing sweetbox partners together. 3. Refresh every five seconds and remove seats taken by others. 4. Continue with the in-memory booking draft; seats are not held yet.
Alternative / exception flow	Remove selections newly occupied by another booking and notify the customer. Reject half sweetboxes, invalid room seats and prohibited layout edits.
Postconditions	The client shows the refreshed map; selecting seats alone does not reserve them.
Requirements / priority	MOB-FR-SEAT-03 (H)
Acceptance	Execute AC-01/AC-02 for every included FR with permitted actors.

10.4 UC-MOB-04 — Showtimes and prices

Table 79: UC-MOB-04 — Showtimes and prices

Field	Use-case specification
Actor	Guest/Customer; Staff/Admin read details only during check-in
Goal	Showtimes and prices
Preconditions	Movie and room exist; management changes need Staff/Admin.
Trigger	Open the relevant screen or invoke the stated operation/event.
Main flow	1. Open movie detail and choose a day in the next seven days. 2. Load OPEN showtimes that have not started. 3. Display room, format, start/end and prices by seat type. 4. Select a showtime and open seat selection after sign-in.
Alternative / exception flow	Reject overlap with 409 and invalid time/format with 400; cancelled showtimes are final. Customer selection excludes closed or started shows.
Postconditions	The schedule and computed seat prices are authoritative; cancellations propagate asynchronously.
Requirements / priority	MOB-FR-SHOW-03 (H), MOB-FR-SHOW-06 (H), MOB-FR-SHOW-07 (M), MOB-FR-SHOW-08 (H), MOB-FR-SHOW-13 (M)
Acceptance	Execute AC-01/AC-02 for every included FR with permitted actors.

10.5 UC-MOB-05 — Booking, concessions, promotions and loyalty

Table 80: UC-MOB-05 — Booking, concessions, promotions and loyalty

Field	Use-case specification
Actor	Customer
Goal	Booking, concessions, promotions and loyalty

Continued on next page

Table 80 — continued

Field	Use-case specification
Preconditions	A signed-in customer has selected an OPEN future showtime and seats; counter guests are represented by Staff/Admin.
Trigger	Open the relevant screen or invoke the stated operation/event.
Main flow	1. Review seats; optionally select active concessions, a promotion code and redeemable points. 2. Confirm age-rating acknowledgement where required. 3. Submit IDs/quantities; the server recalculates all prices, validates pairs, locks reservations and creates a PENDING booking with a ten-minute expiry. 4. Open payment or read/cancel an owned booking according to its state.
Alternative / exception flow	A concurrent hold returns 409 and must not charge money or leave partial seats/points. Invalid concessions/code/points return validation errors. Expiry releases seats and restores reserved points.
Postconditions	Successful creation returns the persisted booking, breakdown and expiry; unsuccessful creation rolls back its transaction.
Requirements / priority	MOB-FR-BOOK-01 (H), MOB-FR-BOOK-02 (H), MOB-FR-BOOK-04 (H), MOB-FR-BOOK-05 (H), MOB-FR-BOOK-06 (H), MOB-FR-BOOK-07 (H), MOB-FR-BOOK-08 (H), MOB-FR-BOOK-09 (H), MOB-FR-BOOK-10 (H), MOB-FR-BOOK-11 (M), MOB-FR-BOOK-13 (M), MOB-FR-BOOK-15 (M), MOB-FR-BOOK-17 (M), MOB-FR-BOOK-19 (L), MOB-FR-STATE-01 (M)
Acceptance	Execute AC-01/AC-02 for every included FR with permitted actors.

10.6 UC-MOB-06 — Online payment and reconciliation

Table 81: UC-MOB-06 — Online payment and reconciliation

Field	Use-case specification
Actor	Customer; PayOS/system as external/shared dependencies
Goal	Online payment and reconciliation
Preconditions	A payable booking exists; PayOS credentials and callback/return settings are configured for live payment.
Trigger	Open the relevant screen or invoke the stated operation/event.
Main flow	1. Request/reuse a PayOS checkout for the booking. 2. Open checkout and display the hold countdown. 3. Query booking every five seconds and verify PayOS every thirty seconds while the customer payment page is active; also reconcile when returning to the app. 4. Signed webhook or verified PayOS data updates payment and booking; load the result from the server.
Alternative / exception flow	Never trust redirect query parameters as proof of payment. Handle cancellation, timeout and 503 without showing false success; late payment confirms only if seats can be reclaimed, otherwise creates a refund.
Postconditions	Payment status is authoritative; SUCCESS confirms the booking and schedules ticket notification without duplicate business effects.
Requirements / priority	MOB-FR-PAY-01 (H), MOB-FR-PAY-02 (H), MOB-FR-PAY-03 (H), MOB-FR-PAY-04 (M), MOB-FR-PAY-06 (H), MOB-FR-PAY-07 (H), MOB-FR-PAY-08 (M), MOB-FR-PAY-11 (M)
Acceptance	Execute AC-01/AC-02 for every included FR with permitted actors.

10.7 UC-MOB-07 — Cancellation and manual refund

Table 82: UC-MOB-07 — Cancellation and manual refund

Field	Use-case specification
Actor	Customer
Goal	Cancellation and manual refund
Preconditions	An owned booking exists; a paid customer cancellation must be at least two hours before showtime.
Trigger	Open the relevant screen or invoke the stated operation/event.
Main flow	1. Open an owned paid ticket and confirm cancellation. 2. Backend checks the two-hour cutoff, creates/reuses refund and retains seats while the customer request is pending. 3. Display pending/completed/rejected result and reason after staff process it on the web. 4. Reload ticket and notifications.
Alternative / exception flow	A cancellation inside two hours is refused with 409. Display backend refund/rejection states; the phone provides no approve/reject/transfer-record action.
Postconditions	Completion yields COMPLETED/REFUNDED and releases cancelled seats; allowed rejection restores SUCCESS/CONFIRMED; customer-requested pending refunds retain seats.
Requirements / priority	MOB-FR-REF-01 (H), MOB-FR-REF-05 (M), MOB-FR-REF-06 (M), MOB-FR-REF-07 (H)
Acceptance	Execute AC-01/AC-02 for every included FR with permitted actors.

10.8 UC-MOB-08 — Ticket lookup and check-in

Table 83: UC-MOB-08 — Ticket lookup and check-in

Field	Use-case specification
Actor	Staff/Admin scan and admit; Customer displays their ticket
Goal	Ticket lookup and check-in
Preconditions	Staff/Admin is signed in; the ticket is CONFIRMED, unused and within 180 minutes before to 120 minutes after start.
Trigger	Open the relevant screen or invoke the stated operation/event.
Main flow	1. Scan QR or type the booking code and press Enter. 2. Lookup ticket; enrich room and age rating when catalogue calls succeed. 3. Check identity/rating as prompted, then explicitly check in. 4. Store checkedInAt, show confirmation and prepare for the next code.
Alternative / exception flow	Keep manual entry available if camera is unavailable/denied. Unknown code returns 404; unpaid, cancelled, pending refund, out-of-window or already-used ticket returns 409. Concurrent second check-in fails. Optional metadata failure must not hide the ticket.
Postconditions	Only one check-in timestamp is stored; check-in is an attribute, not another booking status.
Requirements / priority	MOB-FR-CHK-01 (H), MOB-FR-CHK-02 (H), MOB-FR-CHK-03 (H), MOB-FR-CHK-04 (H), MOB-FR-CHK-05 (M)
Acceptance	Execute AC-01/AC-02 for every included FR with permitted actors.

10.9 UC-MOB-09 — Notifications and transactional email

Table 84: UC-MOB-09 — Notifications and transactional email

Field	Use-case specification
Actor	Customer; backend system produces emails/reminders
Goal	Notifications and transactional email
Preconditions	An account session exists for in-app history; SMTP/event processing is configured for actual delivery.
Trigger	Open the relevant screen or invoke the stated operation/event.
Main flow	1. Load the customer's notification list and unread count. 2. Open detail, mark it read, or mark all own notifications read. 3. Open the related owned ticket. 4. Receive email and in-app history generated by backend events/reminders; no phone push integration is implemented.
Alternative / exception flow	Owner checks reject another user's data; show loading/empty/error-retry states. Sending failure retains FAILED for retry and must not undo payment. Event deduplication reduces duplicates but SMTP plus database commit is not a guaranteed exactly-once transaction.
Postconditions	Read state/badge matches the response; successful delivery is recorded separately from booking/payment state.
Requirements / priority	MOB-FR-NOTI-01 (M), MOB-FR-NOTI-04 (M), MOB-FR-NOTI-06 (H), MOB-FR-NOTI-07 (H), MOB-FR-NOTI-08 (M), MOB-FR-NOTI-09 (M)
Acceptance	Execute AC-01/AC-02 for every included FR with permitted actors.

11. External Interface Requirements

Table 85: 11. External Interface Requirements

Interface	Contract
Client/gateway	REST/JSON UTF-8; Accept application/json; Content-Type application/json for request bodies; Bearer JWT on protected calls. Clients never connect to PostgreSQL or private service ports.
PayOS	Checkout URL/VietQR through backend; signed webhook; client verifies server state rather than trusting a redirect. No client card-data collection.
SMTP / RabbitMQ	Server-only OTP, ticket/refund delivery and committed events; clients read notification APIs, not the broker.
Device/browser	mobile_scanner camera; url_launcher external apps; share_plus/clipboard; secure-storage plugin. Keep manual code entry after camera denial.

12. UI / Screen Requirements

Screen IDs represent routed pages or role landings. Scanner overlays, review editors, profile/password dialogs and seat-type painters are components, not invented routes. Related FR/API lists are functional-area associations; §13 gives exact endpoint contracts.

12.1 MOB-SCR-LOGIN — Sign in

Table 86: MOB-SCR-LOGIN — Sign in

Field	Screen requirement
Route	/login
Actor	Guest/all account roles
Purpose	Sign in
Main components	Email, password, field errors, recovery/register links
Actions	Sign in; recover password; register on customer site
Navigation	Role landing or pending destination; /forgot-password; /register (customer only)
Related FRs	MOB-FR-AUTH-02, MOB-FR-AUTH-03, MOB-FR-AUTH-06, MOB-FR-AUTH-12, MOB-FR-AUTH-13, MOB-FR-NAV-01
Related API area	MOB-API-006, MOB-API-008, MOB-API-010, MOB-API-011, MOB-API-013
Source	Cinema_Mobile/lib/features/auth/screens/login_screen.dart

12.2 MOB-SCR-REGISTER — Registration

Table 87: MOB-SCR-REGISTER — Registration

Field	Screen requirement
Route	/register
Actor	Guest
Purpose	Registration
Main components	Name, email, phone, password and confirmation
Actions	Validate and register
Navigation	/login → /register → /verify-email
Related FRs	MOB-FR-AUTH-01, MOB-FR-AUTH-08, MOB-FR-AUTH-13
Related API area	MOB-API-006, MOB-API-011, MOB-API-013
Source	Cinema_Mobile/lib/features/auth/screens/register_screen.dart

12.3 MOB-SCR-VERIFY — Verify email

Table 88: MOB-SCR-VERIFY — Verify email

Field	Screen requirement
Route	/verify-email
Actor	Guest
Purpose	Verify email
Main components	OTP input; resend countdown; registered password context
Actions	Verify or resend
Navigation	Registration/login → OTP → role landing
Related FRs	MOB-FR-AUTH-07, MOB-FR-AUTH-08, MOB-FR-AUTH-14
Related API area	MOB-API-006, MOB-API-010, MOB-API-011, MOB-API-012, MOB-API-014
Source	Cinema_Mobile/lib/features/auth/screens/verify_email_screen.dart

12.4 MOB-SCR-FORGOT — Forgot password

Table 89: MOB-SCR-FORGOT — Forgot password

Field	Screen requirement
Route	/forgot-password
Actor	Guest/account owner
Purpose	Forgot password
Main components	Email field and submission status
Actions	Request reset OTP
Navigation	Login/Profile → reset request → /reset-password
Related FRs	MOB-FR-AUTH-09, MOB-FR-AUTH-13, MOB-FR-AUTH-14
Related API area	MOB-API-005, MOB-API-006, MOB-API-010, MOB-API-011, MOB-API-012, MOB-API-013
Source	Cinema_Mobile/lib/features/auth/screens/forgot_password_screen.dart

12.5 MOB-SCR-RESET — Reset password

Table 90: MOB-SCR-RESET — Reset password

Field	Screen requirement
Route	/reset-password
Actor	OTP recipient
Purpose	Reset password
Main components	OTP, new password and confirmation
Actions	Reset using emailed code
Navigation	Forgot-password → reset → sign-in

Continued on next page

Table 90 — continued

Field	Screen requirement
Related FRs	MOB-FR-AUTH-09, MOB-FR-AUTH-13, MOB-FR-AUTH-14
Related API area	MOB-API-005, MOB-API-006, MOB-API-010, MOB-API-011, MOB-API-012, MOB-API-013
Source	Cinema_Mobile/lib/features/auth/screens/reset_password_screen.dart

12.6 MOB-SCR-HOME — Customer home

Table 91: MOB-SCR-HOME — Customer home

Field	Screen requirement
Route	/home
Actor	Guest/Customer
Purpose	Customer home
Main components	Now/soon tabs; search; genre chips; movie cards; account/notification navigation
Actions	Search/filter; open movie
Navigation	/home → /movies/{id}; customer navigation to bookings, payments, notifications, profile
Related FRs	MOB-FR-MOV-03, MOB-FR-MOV-04, MOB-FR-MOV-10, MOB-FR-NOTI-09, MOB-FR-MEDIA-01
Related API area	MOB-API-024, MOB-API-025, MOB-API-026, MOB-API-027, MOB-API-028, MOB-API-030, MOB-API-031
Source	Cinema_Mobile/lib/features/home/screens/home_screen.dart

12.7 MOB-SCR-MOVIE — Movie details

Table 92: MOB-SCR-MOVIE — Movie details

Field	Screen requirement
Route	/movies/{id}
Actor	Guest/Customer
Purpose	Movie details
Main components	Poster, rating, metadata, cast/director, trailer, review summary, seven-day showtimes
Actions	Play trailer; select showtime; read reviews
Navigation	Home → movie → /showtimes/{id}/seats
Related FRs	MOB-FR-MOV-02, MOB-FR-MOV-08, MOB-FR-MOV-09, MOB-FR-MOV-11, MOB-FR-MOV-12, MOB-FR-SHOW-03, MOB-FR-SHOW-06, MOB-FR-SHOW-07, MOB-FR-SHOW-08, MOB-FR-SHOW-13
Related API area	MOB-API-002, MOB-API-003, MOB-API-024, MOB-API-039, MOB-API-040, MOB-API-041, MOB-API-042
Source	Cinema_Mobile/lib/features/movie/screens/movie_detail_screen.dart

12.8 MOB-SCR-SEATS — Seat selection

Table 93: MOB-SCR-SEATS — Seat selection

Field	Screen requirement
Route	/showtimes/{id}/seats
Actor	Customer
Purpose	Seat selection
Main components	Rows, type/occupied legend, selection, pair pricing, total
Actions	Select/unselect free seats; refresh
Navigation	Movie → seats → /showtimes/{id}/summary
Related FRs	MOB-FR-SEAT-03, MOB-FR-SHOW-03, MOB-FR-SHOW-13, MOB-FR-BOOK-07, MOB-FR-BOOK-08, MOB-FR-BOOK-09, MOB-FR-BOOK-11
Related API area	MOB-API-001, MOB-API-002, MOB-API-019, MOB-API-021
Source	Cinema_Mobile/lib/features/seat/screens/seat_selection_screen.dart

12.9 MOB-SCR-SUMMARY — Booking summary

Table 94: MOB-SCR-SUMMARY — Booking summary

Field	Screen requirement
Route	/showtimes/{id}/summary
Actor	Customer
Purpose	Booking summary
Main components	BookingDraft, seats, concessions picker, promotion quote, points, age acknowledgement
Actions	Adjust extras; hold seats
Navigation	Seats → summary → booking payment; missing draft → seats
Related FRs	MOB-FR-MOV-09, MOB-FR-BOOK-01, MOB-FR-BOOK-09, MOB-FR-BOOK-13, MOB-FR-BOOK-15, MOB-FR-BOOK-17
Related API area	MOB-API-021, MOB-API-022, MOB-API-023, MOB-API-024, MOB-API-038
Source	Cinema_Mobile/lib/features/booking/screens/booking_summary_screen.dart

12.10 MOB-SCR-PAYMENT — Payment

Table 95: MOB-SCR-PAYMENT — Payment

Field	Screen requirement
Route	/bookings/{id}/payment
Actor	Customer
Purpose	Payment
Main components	Booking/price breakdown; expiry countdown; PayOS link; automatic polling
Actions	Open/continue PayOS; return and reconcile

Continued on next page

Table 95 — continued

Field	Screen requirement
Navigation	Summary/ticket → payment → external PayOS → result
Related FRs	MOB-FR-BOOK-05, MOB-FR-BOOK-10, MOB-FR-PAY-01, MOB-FR-PAY-02, MOB-FR-PAY-11
Related API area	MOB-API-017, MOB-API-034, MOB-API-035
Source	Cinema_Mobile/lib/features/payment/screens/payment_screen.dart

12.11 MOB-SCR-RESULT — Payment result

Table 96: MOB-SCR-RESULT — Payment result

Field	Screen requirement
Route	/bookings/{id}/result
Actor	Customer
Purpose	Payment result
Main components	Verification/loading, successful/pending/failed result
Actions	Reload server result; open ticket
Navigation	PayOS/payment → result → /bookings/{id}
Related FRs	MOB-FR-BOOK-02, MOB-FR-BOOK-05, MOB-FR-PAY-02, MOB-FR-PAY-08
Related API area	MOB-API-017, MOB-API-034
Source	Cinema_Mobile/lib/features/payment/screens/payment_result_screen.dart

12.12 MOB-SCR-TICKETS — My tickets

Table 97: MOB-SCR-TICKETS — My tickets

Field	Screen requirement
Route	/bookings
Actor	Customer
Purpose	My tickets
Main components	Active/History tabs, ticket cards, refresh and empty/error states
Actions	Open ticket; refresh
Navigation	Customer navigation → tickets → /bookings/{id}
Related FRs	MOB-FR-BOOK-04
Related API area	MOB-API-020
Source	Cinema_Mobile/lib/features/booking/screens/my_bookings_screen.dart

12.13 MOB-SCR-TICKET — Ticket details

Table 98: MOB-SCR-TICKET — Ticket details

Field	Screen requirement
Route	/bookings/{id}
Actor	Customer
Purpose	Ticket details
Main components	Seats, extras, totals, QR, checkedInAt, cancel/refund, review and share/calendar
Actions	Pay pending ticket; cancel/request refund; review after check-in; share/calendar
Navigation	Tickets/notification → ticket → payment or return
Related FRs	MOB-FR-MOV-12, MOB-FR-BOOK-02, MOB-FR-BOOK-05, MOB-FR-BOOK-06, MOB-FR-BOOK-13, MOB-FR-BOOK-15, MOB-FR-BOOK-17, MOB-FR-BOOK-19, MOB-FR-PAY-03, MOB-FR-PAY-06, MOB-FR-REF-01, MOB-FR-REF-05, MOB-FR-REF-06, MOB-FR-CHK-01, MOB-FR-CHK-05
Related API area	MOB-API-002, MOB-API-015, MOB-API-017, MOB-API-018, MOB-API-021, MOB-API-022, MOB-API-023, MOB-API-033, MOB-API-034, MOB-API-037, MOB-API-038, MOB-API-039, MOB-API-040, MOB-API-041, MOB-API-042
Source	Cinema_Mobile/lib/features/booking/screens/booking_detail_screen.dart

12.14 MOB-SCR-PAYHISTORY — Payment history

Table 99: MOB-SCR-PAYHISTORY — Payment history

Field	Screen requirement
Route	/payments
Actor	Customer
Purpose	Payment history
Main components	Owned transaction rows, amounts, statuses
Actions	Refresh; inspect related booking where provided
Navigation	Customer navigation → payments → related ticket
Related FRs	MOB-FR-PAY-03, MOB-FR-PAY-04
Related API area	MOB-API-037
Source	Cinema_Mobile/lib/features/payment/screens/payment_history_screen.dart

12.15 MOB-SCR-NOTIFICATIONS — Notifications

Table 100: MOB-SCR-NOTIFICATIONS — Notifications

Field	Screen requirement
Route	/notifications
Actor	Customer
Purpose	Notifications
Main components	Notification list, unread badge, mark-all-read

Continued on next page

Table 100 — continued

Field	Screen requirement
Actions	Open notification; mark all read
Navigation	Customer bell/navigation → /notifications/{id}
Related FRs	MOB-FR-NOTI-01, MOB-FR-NOTI-09
Related API area	MOB-API-028, MOB-API-030, MOB-API-031, MOB-API-032
Source	Cinema_Mobile/lib/features/notification/screens/notification_list_screen.dart

12.16 MOB-SCR-NOTIFICATION — Notification details

Table 101: MOB-SCR-NOTIFICATION — Notification details

Field	Screen requirement
Route	/notifications/{id}
Actor	Customer
Purpose	Notification details
Main components	Subject, content, read state, related booking
Actions	Mark read on open; open ticket
Navigation	Notifications → detail → /bookings/{id}
Related FRs	MOB-FR-NOTI-04, MOB-FR-NOTI-09
Related API area	MOB-API-028, MOB-API-029, MOB-API-030, MOB-API-031
Source	Cinema_Mobile/lib/features/notification/screens/notification_detail_screen.dart

12.17 MOB-SCR-PROFILE — Account and membership

Table 102: MOB-SCR-PROFILE — Account and membership

Field	Screen requirement
Route	/profile
Actor	Customer
Purpose	Account and membership
Main components	Identity, edit-profile/password dialogs, membership card/history, logout
Actions	Edit name/phone; change/recover password; sign out
Navigation	Customer navigation → profile; password recovery → forgot/reset
Related FRs	MOB-FR-AUTH-10, MOB-FR-AUTH-11, MOB-FR-AUTH-16, MOB-FR-AUTH-17, MOB-FR-BOOK-17
Related API area	MOB-API-004, MOB-API-007, MOB-API-008, MOB-API-009, MOB-API-010, MOB-API-021, MOB-API-023
Source	Cinema_Mobile/lib/features/profile/screens/profile_screen.dart

12.18 MOB-SCR-MGT-LANDING — Management-account landing

Table 103: MOB-SCR-MGT-LANDING — Management-account landing

Field	Screen requirement
Route	/home (management account)
Actor	Staff/Admin
Purpose	Management-account landing
Main components	Role explanation; configured admin website link; ticket check-in; logout
Actions	Open configured website or mobile check-in
Navigation	Management sign-in → landing → /staff/check-in
Related FRs	MOB-FR-AUTH-05, MOB-FR-AUTH-10, MOB-FR-CHK-04
Related API area	MOB-API-007, MOB-API-008, MOB-API-010, MOB-API-016, MOB-API-018
Source	Cinema_Mobile/lib/features/profile/screens/management_account_screen.dart

12.19 MOB-SCR-CHECKIN — Ticket check-in

Table 104: MOB-SCR-CHECKIN — Ticket check-in

Field	Screen requirement
Route	/staff/check-in
Actor	Staff/Admin
Purpose	Ticket check-in
Main components	Code field; camera scanner overlay; ticket details; age reminder; admission button
Actions	Scan/type; lookup; check in once
Navigation	Management landing/navigation → check-in → next ticket
Related FRs	MOB-FR-MOV-09, MOB-FR-CHK-02, MOB-FR-CHK-03, MOB-FR-CHK-04, MOB-FR-CHK-05
Related API area	MOB-API-016, MOB-API-017, MOB-API-018, MOB-API-024
Source	Cinema_Mobile/lib/features/staff/screens/staff_check_in_screen.dart

Web authentication screens are reused between entry points, but admin login disables registration. Profile editing/password changes are dialogs. The phone management landing is selected by role, not a separate administrative URL.

13. API Requirements

Endpoints are extracted from controller HTTP attributes, removing :long constraints from parameter notation only. Direct client means matching method/path found in the scoped Dart services; API-only/shared means a controller exists without a detected dedicated client call. Controller policy must be combined with gateway policy and record ownership.

Table 105: 13. API Requirements

ID	Method / endpoint	Controller policy	Input contract	Usage / status	Evidence
MOB-API-001	GET /api/seats/room/{roomId}	authenticated	—	Direct client; implemented	Cinema_BE/MovieService/MovieService/Controllers/SeatsController.cs:36
MOB-API-002	GET /api/showtimes/{showtimeId}	public	—	Direct client; implemented	Cinema_BE/MovieService/MovieService/Controllers/ShowtimesController.cs:41
MOB-API-003	GET /api/showtimes/movie/{movieId}/open	public	Query: int pageNumber; Query: int pageSize; Query: string sortBy; Query: string sortDir	Direct client; implemented	Cinema_BE/MovieService/MovieService/Controllers/ShowtimesController.cs:160
MOB-API-004	POST /api/v1/auth/change-password	public	Body: ChangePasswordRequest request	Direct client; implemented	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs:105
MOB-API-005	POST /api/v1/auth/forgot-password	public	Body: EmailRequest request	Direct client; implemented	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs:54
MOB-API-006	POST /api/v1/auth/login	public	Body: LoginRequest request	Direct client; implemented	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs:46
MOB-API-007	POST /api/v1/auth/logout	public	Body: RefreshTokenRequest request	Direct client; implemented	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs:79
MOB-API-008	GET /api/v1/auth/me	public	—	Direct client; implemented	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs:88

Continued on next page

Table 105 — continued

ID	Method / endpoint	Controller policy	Input contract	Usage / status	Evidence
MOB-API-009	PUT /api/v1/auth/me	public	Body: UpdateProfileRequest request	Direct client; implemented	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs:97
MOB-API-010	POST /api/v1/auth/refresh	public	Body: RefreshTokenRequest request	Direct client; implemented	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs:70
MOB-API-011	POST /api/v1/auth/register	public	Body: RegisterRequest request	Direct client; implemented	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs:22
MOB-API-012	POST /api/v1/auth/resend-verification	public	Body: EmailRequest request	Direct client; implemented	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs:38
MOB-API-013	POST /api/v1/auth/reset-password	public	Body: ResetPasswordRequest request	Direct client; implemented	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs:62
MOB-API-014	POST /api/v1/auth/verify-email	public	Body: VerifyEmailRequest request	Direct client; implemented	Cinema_BE/AuthService/AuthService/Controllers/AuthController.cs:30
MOB-API-015	POST /api/v1/bookings/{id}/cancel	Policy = AuthorizationPolicies.UserOrStaff	Query: string? reason	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs:201
MOB-API-016	POST /api/v1/bookings/{id}/check-in	Policy = AuthorizationPolicies.StaffOrAdmin	—	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs:147
MOB-API-017	GET /api/v1/bookings/{id}	Policy = AuthorizationPolicies.AnyRole	—	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs:37
MOB-API-018	GET /api/v1/bookings/code/{bookingCode}	Policy = AuthorizationPolicies.StaffOrAdmin	—	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs:137
MOB-API-019	GET /api/v1/bookings/showtime/{showtimeId}/occupied-seats	Policy = AuthorizationPolicies.AnyRole	—	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs:100

Continued on next page

Table 105 — continued

ID	Method / endpoint	Controller policy	Input contract	Usage / status	Evidence
MOB-API-020	GET /api/v1/bookings/user/{userId}	Policy = Authorization Policies.AnyRole	Query: int page; Query: int size; Query: string sortBy; Query: string sortDir	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs:50
MOB-API-021	POST /api/v1/bookings	Policy = Authorization Policies.AnyRole	Body: BookingRequest request	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/BookingController.cs:156
MOB-API-022	GET /api/v1/concessions	public	Query: bool includeInactive	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/ConcessionController.cs:24
MOB-API-023	GET /api/v1/loyalty/me	authenticated	—	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/LoyaltyController.cs:24
MOB-API-024	GET /api/v1/movies/{id}	public	—	Direct client; implemented	Cinema_BE/MovieService/MovieService/Controllers/MoviesController.cs:34
MOB-API-025	GET /api/v1/movies/browse	public	Query: string? when; Query: string? genre; Query: string? keyword; Query: int page; Query: int size	Direct client; implemented	Cinema_BE/MovieService/MovieService/Controllers/MoviesController.cs:57
MOB-API-026	GET /api/v1/movies/genres	public	—	Direct client; implemented	Cinema_BE/MovieService/MovieService/Controllers/MoviesController.cs:70
MOB-API-027	GET /api/v1/movies/status/{status}	public	Query: int page; Query: int size; Query: string sortBy; Query: string sortDir	Direct client; implemented	Cinema_BE/MovieService/MovieService/Controllers/MoviesController.cs:42
MOB-API-028	PATCH /api/v1/notifications/{id}/read	Policy = Authorization Policies.AnyRole	—	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/NotificationController.cs:77
MOB-API-029	GET /api/v1/notifications/{id}	Policy = Authorization Policies.AnyRole	—	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/NotificationController.cs:36

Continued on next page

Table 105 — continued

ID	Method / endpoint	Controller policy	Input contract	Usage / status	Evidence
MOB-API-030	POST /api/v1/notifications/read-all	Policy = Authorization Policies.AnyRole	—	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/NotificationController.cs:89
MOB-API-031	GET /api/v1/notifications/unread-count	Policy = Authorization Policies.AnyRole	—	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/NotificationController.cs:67
MOB-API-032	GET /api/v1/notifications/user/{userId}	Policy = Authorization Policies.AnyRole	Query: int page; Query: int size; Query: string sortBy; Query: string sortDir	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/NotificationController.cs:49
MOB-API-033	GET /api/v1/payments/{id}/refund	Policy = Authorization Policies.AnyRole	—	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs:208
MOB-API-034	POST /api/v1/payments/payos/{orderCode}/verify	Policy = Authorization Policies.AnyRole	Query: string? recipient Email	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs:147
MOB-API-035	POST /api/v1/payments/payos/checkout	Policy = Authorization Policies.AnyRole	Body: PayOsCreateRequest request	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs:122
MOB-API-036	POST /api/v1/payments/payos/webhook	public	Body: JsonElement body	Shared backend/API-only; implemented	Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs:172
MOB-API-037	GET /api/v1/payments/user/{userId}	Policy = Authorization Policies.AnyRole	Query: int page; Query: int size; Query: string sortBy; Query: string sortDir	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/PaymentController.cs:70
MOB-API-038	POST /api/v1/promotions/quote	authenticated	Body: PromotionQuote Request request	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/PromotionController.cs:33
MOB-API-039	DELETE /api/v1/reviews/{id}	authenticated	—	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/MovieReviewController.cs:58
MOB-API-040	GET /api/v1/reviews/booking/{bookingId}	authenticated	—	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/MovieReviewController.cs:43

Continued on next page

Table 105 — continued

ID	Method / endpoint	Controller policy	Input contract	Usage / status	Evidence
MOB-API-041	GET /api/v1/reviews/movie/{movieId}	public	Query: int page; Query: int size	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/MovieReviewController.cs:25
MOB-API-042	POST /api/v1/reviews	authenticated	Body: MovieReviewRequest request	Direct client; implemented	Cinema_BE/BookingService/BookingService/Controllers/MovieReviewController.cs:50

POST /api/showtimes/{showtimeId}/seats is called internally by BookingService with the forwarded token. Its controller permits authenticated callers while the gateway general showtime-write route requires StaffOrAdmin; do not treat it as a Customer client call. PayOS webhook is anonymous to JWT but authenticated by HMAC. Internal calls and webhook remain dependencies, not extra mobile screens.

Pagination is endpoint-specific: page/size for many catalogue/booking/payment APIs; pageNumber/pageSize for declared showtime/room APIs. Paged sizes are capped, usually at 50; genre/menu/seat/loyalty arrays are not all paginated. ApiClient uses a 15-second timeout and UTF-8 JSON/ProblemDetails parsing. Status/result shapes follow current controller DTOs.

13.1 Request field definitions

Table 106: Request field definitions

DTO	Fields / types	Source
ChangePasswordRequest	CurrentPassword: string; NewPassword: string	Cinema_BE/AuthService/AuthService/DTOs/Request/ChangePasswordRequest.cs
EmailRequest	Email: string	Cinema_BE/AuthService/AuthService/DTOs/Request/EmailRequest.cs
LoginRequest	Email: string; Password: string	Cinema_BE/AuthService/AuthService/DTOs/Request/LoginRequest.cs

Continued on next page

Table 106 — continued

DTO	Fields / types	Source
RefreshTokenRequest	RefreshToken: string	Cinema_BE/AuthService/AuthService/DTOs/Request/RefreshTokenRequest.cs
RegisterRequest	Email: string; Password: string; FullName: string; Phone: string?	Cinema_BE/AuthService/AuthService/DTOs/Request/RegisterRequest.cs
ResetPasswordRequest	Email: string; Otp: string; NewPassword: string	Cinema_BE/AuthService/AuthService/DTOs/Request/ResetPasswordRequest.cs
UpdateProfileRequest	FullName: string; Phone: string?	Cinema_BE/AuthService/AuthService/DTOs/Request/UpdateProfileRequest.cs
VerifyEmailRequest	Email: string; Otp: string; Password: string	Cinema_BE/AuthService/AuthService/DTOs/Request/VerifyEmailRequest.cs
BookingRequest	UserId: long?; ShowtimeId: long?; MovieTitle: string?; CustomerEmail: string?; CustomerPhone: string?; ShowTime: DateTime?; Seats: List<SeatRequest>; PromotionCode: string?; UsePoints: int?; Concessions: List<BookingConcessionRequest>	Cinema_BE/BookingService/BookingService/DTOs/Requests/BookingRequest.cs
MovieReviewRequest	BookingId: long?; Rating: int?; Comment: string?	Cinema_BE/BookingService/BookingService/DTOs/Requests/MovieReviewRequest.cs
PayOsCreateRequest	BookingId: long?; UserId: long?; Amount: decimal?; Description: string?; ReturnUrl: string?; CancelUrl: string?	Cinema_BE/BookingService/BookingService/DTOs/Requests/PayOsCreateRequest.cs
PromotionQuoteRequest	ShowtimeId: long?; TicketAmount: decimal; Tickets: int; Code: string?	Cinema_BE/BookingService/BookingService/DTOs/Requests/PromotionQuoteRequest.cs

14. Data Requirements

Project-Cinema-DB is uninitialized in this checkout (git submodule status begins with "-"). Its SQL dumps/seed/migrations are unavailable; exact DDL and live migration state are [TO BE CONFIRMED]. This contract uses local EF models/DbContexts and available backend SQL migrations. Model mappings do not prove a live DB has been migrated.

Table 107: 14. Data Requirements

Database	Entities / tables	Integrity and fields
cinema_auth_db	User/Role; users, roles, users_roles, refresh_tokens, otp_codes	Unique email/token; BCrypt hashes; role joins; OTP lifetime and token revocation
cinema_movie_db	Movie; movies	Title/duration/genre/language/rating/director/cast; poster/trailer/preview; ACTIVE/INACTIVE
cinema_showtime_db	Room/Seat/Showtime; rooms, seats, showtimes, outbox_messages	Unique room/row/number; format, time, price; room-local foreign keys
cinema_booking_db	Booking, BookingSeat, SeatReservation, ConcessionItem, BookingConcession, Promotion, LoyaltyAccount, LoyaltyTransaction, MovieReview; bookings, booking_seats, seat_reservations, concession_items, booking_concessions, promotions, loyalty_accounts, loyalty_transactions, movie_reviews, outbox_messages	Unique booking code; unique showtime/seat reservation; unique user/movie review; amount snapshots; nullable counter userId; checkedInAt/expiresAt/remindedAt
cinema_payment_db	Payment/Refund; payments, refunds, outbox_messages	Booking, amount, method/state, PayOS reference and payer bank; one refund per payment
cinema_notification_db	Notification; notifications	Recipient/booking/event; deduplicate event; sentAt/readAt; state and error

Client persistence is session tokens/user metadata via flutter_secure_storage and in-memory booking drafts, not six local databases. Server decimal totals snapshot seat/concession/discount/point values. Cinema time uses Vietnam local time and timestamp without time zone mappings. Cross-service IDs are application/event-validated logical references.

15. Business Workflows

Table 108: 15. Business Workflows

ID	Workflow
MOB-WF-01	Register → verify OTP → role landing; unverified login goes to OTP. A failed recovery never authenticates an account.
MOB-WF-02	Browse → movie → open showtime → seats → summary with extras/code/points/age → create PENDING hold → PayOS → server verify → CONFIRMED + ticket notification.
MOB-WF-03	expiresAt reached → expiry worker (default every 30 seconds) → EXPIRED and release seats/points. Late payment reclaims seats and confirms, or creates full refund.
MOB-WF-04	Paid customer cancellation ≥2 hours before show → REFUND_PENDING with seats retained → web staff manual transfer/record → REFUNDED/CANCELLED. Eligible rejection restores SUCCESS/CONFIRMED.
MOB-WF-05	1. Scan QR or type the booking code and press Enter. 2. Lookup ticket; enrich room and age rating when catalogue calls succeed. 3. Check identity/rating as prompted, then explicitly check in. 4. Store checkedInAt, show confirmation and prepare for the next code.

15.1 State contract

Table 109: State contract

Object	Transitions
Booking	PENDING → CONFIRMED / CANCELLED / EXPIRED; EXPIRED → CONFIRMED (verified late payment and available seats); CONFIRMED → REFUND_PENDING / CANCELLED; REFUND_PENDING → CONFIRMED / CANCELLED
Payment	PENDING → SUCCESS / FAILED; FAILED → SUCCESS (verified late payment); SUCCESS → REFUND_PENDING → REFUNDED; REFUND_PENDING → SUCCESS
Refund	PENDING → COMPLETED / REJECTED; REJECTED → PENDING (cinema cancels later)
Showtime	OPEN ↔ CLOSED; OPEN/CLOSED → CANCELLED
Notification	PENDING → SENT/FAILED; FAILED → SENT; readAt independent of send state

checkedInAt is an attribute of a confirmed booking, not a separate booking status.

16. Authentication & Authorization

Gateway and services validate `JWT` signature/expiry/issuer/audience. Defaults: access 60 minutes, refresh seven days rotated on use; `OTP` six digits, five minutes, five attempts, 60-second resend. Admin-created accounts skip registration `OTP`. Password and `OTP` values are BCrypt-hashed.

`ApiClient` renews and retries a protected request once. Protected checks reject `ACCOUNT_LOCKED` (`401`), `ROLE_CHANGED` (`403`) or unavailable `AuthService` (`503`). Public catalogue GETs omit bearer token; a signed-in client probes /auth/me at most once per minute, ignores ordinary availability errors for browsing and expires the session for lock/role-change signals. Logout clears storage/revokes refresh; reset/change password revokes old refresh tokens. Refresh revocation alone does not certify immediate invalidation of every issued access token.

17. Error Handling

Table 110: 17. Error Handling

Code / condition	Handling
`400`	Show field/business validation; do not advance
`401`	Renew once if possible; otherwise clear session/sign in; retain lock/session reason
`403`	Deny role/ownership action; `EMAIL_NOT_VERIFIED` opens `OTP`; `ROLE_CHANGED` signs in again
`404`	Show missing record without leaking another owner's review ticket
`409`	Explain seat/state/time conflict and reload before retrying
`429`	Respect Retry-After/countdown; preserve inputs
`500` / `503`	Show retryable service error; no false booking/payment success
Timeout/offline/invalid `JSON`	Bounded error (15-second client timeout); retry and honest pending/failure state
Loading/empty/camera denial	Explicit state without fake records; preserve manual ticket-code entry

18. Non-Functional Requirements

Table 111: 18. Non-Functional Requirements

ID	Requirement	Allocation
MOB-NFR-SEC-01	Passwords and OTP codes are hashed with BCrypt; responses never include stack traces or internal errors.	Client or shared-backend dependency
MOB-NFR-SEC-02	JWTs are signed with a secret of at least 32 characters; access tokens last 60 minutes; refresh tokens last 7 days, rotate, and are revoked on sign-out, password reset and role change.	Client or shared-backend dependency
MOB-NFR-SEC-03	Authorization is checked at the gateway (per route) and in the service (per record, e.g. only one's own tickets).	Client or shared-backend dependency
MOB-NFR-SEC-04	Rate limits: 100 requests per minute per IP for all APIs, 10 for authentication APIs.	Client or shared-backend dependency
MOB-NFR-SEC-05	Only the gateway is reachable from the network; services, PostgreSQL, RabbitMQ and monitoring listen on 127.0.0.1.	Client or shared-backend dependency
MOB-NFR-SEC-06	Secrets are not in the source code; they are shared with SOPS + age.	Client or shared-backend dependency
MOB-NFR-SEC-07	The server validates every input (DataAnnotations + validators); clients validate first to report mistakes early.	Client or shared-backend dependency
MOB-NFR-PERF-01	Paged endpoints cap size, normally 50; genres/seats/menu arrays are exempt. Ticket UI currently loads only the first page. Response-time/concurrent-user SLA is [TO BE CONFIRMED], not a measured guarantee.	Client or shared-backend dependency
MOB-NFR-PERF-02	Holding seats, confirming tickets and taking payment lock the booking row (SELECT … FOR UPDATE) in a transaction, so seats are never sold twice and money is never taken twice under concurrent use.	Client or shared-backend dependency
MOB-NFR-REL-01	Calls to MovieService / PayOS retry transient errors at most twice (creating a PayOS link is never retried); a circuit breaker opens for 15 seconds after repeated failures and returns 503 at once.	Client or shared-backend dependency
MOB-NFR-REL-02	Outbox: no event is lost while RabbitMQ is down; it is sent when RabbitMQ is back.	Client or shared-backend dependency
MOB-NFR-REL-03	Consumers are idempotent; failing messages are retried 3 times and then moved to a DLQ.	Client or shared-backend dependency
MOB-NFR-REL-04	The gateway checks each service's /health/live every 10 seconds and returns 503 at once for a dead service.	Client or shared-backend dependency
MOB-NFR-OBS-01	Prometheus metrics for HTTP, runtime and business events (bookings, payments, refunds, outbox); Grafana dashboard "Cinema - Tổng quan".	Client or shared-backend dependency
MOB-NFR-OBS-02	OpenTelemetry traces to Jaeger across the gateway, services, SQL, PayOS and RabbitMQ.	Client or shared-backend dependency
MOB-NFR-OBS-03	Logs carry TraceId, SpanId and CorrelationId; every response has an X-Correlation-ID header.	Client or shared-backend dependency
MOB-NFR-MAINT-01	Layered services, DTOs, validators and a global exception handler everywhere; documentation in docs/.	Client or shared-backend dependency
MOB-NFR-MAINT-02	CI declares .NET tests, Flutter analyze/test and customer/admin web builds. Current Flutter workflow pins 3.47.4, superseding older prose; no current successful CI run is asserted.	Client or shared-backend dependency
MOB-NFR-DEP-01	One command, docker compose up -d --build, runs the whole backend; monitoring through the monitoring profile.	Client or shared-backend dependency
MOB-NFR-USE-01	The customer app supports Android, web and Windows; the admin website is designed for desktops and still works on narrow screens.	Client or shared-backend dependency

Continued on next page

Table 111 — continued

ID	Requirement	Allocation
MOB-NFR-COMP-01	Web cameras need `localhost` or `HTTPS`; a real deployment must use `HTTPS`.	Client or shared-backend dependency

`NFR` mechanisms may be evidenced in code; availability, throughput, universal accessibility and compatibility require runtime validation and are not established by static inspection.

19. Security Requirements

Table 112: 19. Security Requirements

ID	Security requirement and verification
MOB-SEC-01	Reject forged/expired/wrong-issuer JWT and forbidden roles at gateway/service; test record-ID changes against ownership.
MOB-SEC-02	Recompute prices and validate seat pairs/menu/code/points on server; do not trust client totals or redirect parameters as payment.
MOB-SEC-03	Verify PayOS HMAC before effects; test repeat webhook, wrong signature, unknown order and amount mismatch. Keep merchant keys in backend.
MOB-SEC-04	Use deployment HTTPS and restricted CORS/private-service exposure. Development HTTP is not proof of production TLS.
MOB-SEC-05	Use platform secure-storage abstraction; never embed backend secrets in client. Clear/revoke credentials on logout/reset and handle unreadable storage as signed out.
MOB-SEC-06	Encode user content in email; suppress stack traces/secrets. Generic forgot-password does not imply universal resistance to email enumeration.

20. Testing Requirements

Derive tests from each FR AC-01/AC-02. Existing test files evidence coverage intent, not a newly run report. Backend fixtures use Testcontainers PostgreSQL/RabbitMQ and configured fakes for external PayOS/SMTP/ catalogue dependencies. Live PayOS/SMTP, camera/share/printing require separate controlled acceptance.

Table 113: 20. Testing Requirements

ID	Required test
MOB-TEST-01	OTP expiry/attempts/resend; UTF-8 password bounds; refresh reuse; lock/role changes; wrong owner/role.
MOB-TEST-02	Two simultaneous holds on one seat: one success, one 409; no partial seats/points. Reject half sweetbox.
MOB-TEST-03	Payment success/cancel/timeout, duplicate signed webhook, late payment with available/lost seats; never false paid UI.
MOB-TEST-04	Cancellation exactly at and around two hours; complete/reject/reopen refund; one refund, pending-seat retention and point reversal.
MOB-TEST-05	Check-in immediately before/at/after time limits; two simultaneous scans; manual lookup after camera denial.
MOB-TEST-06	Route restoration, missing draft, loading/empty/error, timeout/retry, unread counts and return from external app.

20.1 Existing test evidence

Table 114: Existing test evidence

Area / use case	Files
UC-MOB-01	Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart; Cinema_Mobile/test/register_validation_test.dart
UC-MOB-02	Cinema_BE/BookingService/BookingService.Tests/Integration/MovieReviewTests.cs; Cinema_Mobile/test/age_rating_test.dart; Cinema_Mobile/test/coming_soon_test.dart; Cinema_Mobile/test/liquid_glass_responsive_preview_test.dart; Cinema_Mobile/test/review_test.dart
UC-MOB-03	Cinema_BE/MovieService/MovieService.Tests/Integration/SeatTests.cs; Cinema_Mobile/test/booking_flow_test.dart; Cinema_Mobile/test/seat_map_test.dart
UC-MOB-04	Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_BE/MovieService/MovieService.Tests/Integration/ShowtimeFormatTests.cs; Cinema_BE/MovieService/MovieService.Tests/Integration/ShowtimeTests.cs; Cinema_Mobile/test/showtime_format_test.dart
UC-MOB-05	Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PromotionTests.cs; Cinema_Mobile/test/booking_flow_test.dart; Cinema_Mobile/test/concession_test.dart; Cinema_Mobile/test/loyalty_test.dart; Cinema_Mobile/test/promotion_test.dart

Continued on next page

Table 114 — continued

Area / use case	Files
UC-MOB-06	Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PaymentSummaryTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PayOsOrderCodeTests.cs; Cinema_BE/BookingService/BookingService.Tests/Unit/PayOsSignatureTests.cs; Cinema_Mobile/test/booking_flow_test.dart
UC-MOB-07	Cinema_BE/BookingService/BookingService.Tests/Integration/RefundTests.cs; Cinema_Mobile/test/booking_details_ui_test.dart; Cinema_Mobile/test/refund_details_ui_test.dart; Cinema_Mobile/test/staff_booking_details_ui_test.dart
UC-MOB-08	Cinema_BE/BookingService/BookingService.Tests/Integration/CheckInTests.cs; Cinema_Mobile/test/check_in_test.dart
UC-MOB-09	Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_Mobile/test/notification_read_test.dart

21. Acceptance Criteria

All H requirements must meet both criteria; security, ownership, financial and concurrency tests must pass. Disclose every M/L omission and current implementation gap. This document does not certify a passing run.

Table 115: 21. Acceptance Criteria

ID	Acceptance gate
MOB-ACC-01	Demonstrate account → movie → showtime → seats → booking → payment → ticket/ notification with real backend or documented fixtures.
MOB-ACC-02	Reject cross-account unauthorized access and duplicate concurrent seat sale/check-in/payment effects.
MOB-ACC-03	Demonstrate refund and late-payment outcomes with correct seats/points and no false confirmed ticket.
MOB-ACC-04	Verify navigation, loading/error/empty and device fallbacks; disclose §27 limitations.

22. Requirements Traceability Matrix

Complete FR matrix. Source IDs provide backward mapping; NAV/STATE/MEDIA are code-derived refinements. API associations are by functional area, not a claim that every screen invokes every listed operation. Existing tests are coverage candidates, not measured test results.

Table 116: 22. Requirements Traceability Matrix

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-AUTH-01 ← FR-AUTH-01	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-REGISTER	MOB-API-011	MOB-FR-AUTH-01-AC-01, MOB-FR-AUTH-01-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart
MOB-FR-AUTH-02 ← FR-AUTH-02	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-LOGIN	MOB-API-006	MOB-FR-AUTH-02-AC-01, MOB-FR-AUTH-02-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart
MOB-FR-AUTH-03 ← FR-AUTH-03	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-LOGIN	MOB-API-006, MOB-API-010	MOB-FR-AUTH-03-AC-01, MOB-FR-AUTH-03-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-AUTH-04 ← FR-AUTH-04	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01		MOB-API-008	MOB-FR-AUTH-04-AC-01, MOB-FR-AUTH-04-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart
MOB-FR-AUTH-05 ← FR-AUTH-05	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-MGT-LANDING	MOB-API-008	MOB-FR-AUTH-05-AC-01, MOB-FR-AUTH-05-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart
MOB-FR-AUTH-06 ← FR-AUTH-06	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-LOGIN	MOB-API-008	MOB-FR-AUTH-06-AC-01, MOB-FR-AUTH-06-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-AUTH-07 ← FR-AUTH-07	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-VERIFY	MOB-API-012, MOB-API-014	MOB-FR-AUTH-07-AC-01, MOB-FR-AUTH-07-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart
MOB-FR-AUTH-08 ← FR-AUTH-08	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-REGISTER, MOB-SCR-VERIFY	MOB-API-011	MOB-FR-AUTH-08-AC-01, MOB-FR-AUTH-08-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart
MOB-FR-AUTH-09 ← FR-AUTH-09	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-FORGOT, MOB-SCR-RESET	MOB-API-005, MOB-API-013	MOB-FR-AUTH-09-AC-01, MOB-FR-AUTH-09-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-AUTH-10 ← FR-AUTH-10	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-PROFILE, MOB-SCR-MGT-LANDING	MOB-API-007, MOB-API-010	MOB-FR-AUTH-10-AC-01, MOB-FR-AUTH-10-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart
MOB-FR-AUTH-11 ← FR-AUTH-11	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-PROFILE	MOB-API-008	MOB-FR-AUTH-11-AC-01, MOB-FR-AUTH-11-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart
MOB-FR-AUTH-12 ← FR-AUTH-12	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-LOGIN	MOB-API-008	MOB-FR-AUTH-12-AC-01, MOB-FR-AUTH-12-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-AUTH-13 ← FR-AUTH-13	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-LOGIN, MOB-SCR-REGISTER, MOB-SCR-FORGOT, MOB-SCR-RESET	MOB-API-006, MOB-API-011, MOB-API-013	MOB-FR-AUTH-13-AC-01, MOB-FR-AUTH-13-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart
MOB-FR-AUTH-14 ← FR-AUTH-14	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-VERIFY, MOB-SCR-FORGOT, MOB-SCR-RESET	MOB-API-006, MOB-API-010, MOB-API-012	MOB-FR-AUTH-14-AC-01, MOB-FR-AUTH-14-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart
MOB-FR-AUTH-16 ← FR-AUTH-16	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-PROFILE	MOB-API-009	MOB-FR-AUTH-16-AC-01, MOB-FR-AUTH-16-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-AUTH-17 ← FR-AUTH-17	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-PROFILE	MOB-API-004	MOB-FR-AUTH-17-AC-01, MOB-FR-AUTH-17-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart
MOB-FR-MOV-02 ← FR-MOV-02	MOB-BR-26	UC-MOB-02	MOB-SCR-MOVIE	MOB-API-024	MOB-FR-MOV-02-AC-01, MOB-FR-MOV-02-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/MovieReviewTests.cs; Cinema_Mobile/test/age_rating_test.dart; Cinema_Mobile/test/coming_soon_test.dart
MOB-FR-MOV-03 ← FR-MOV-03	MOB-BR-26	UC-MOB-02	MOB-SCR-HOME	MOB-API-025	MOB-FR-MOV-03-AC-01, MOB-FR-MOV-03-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/MovieReviewTests.cs; Cinema_Mobile/test/age_rating_test.dart; Cinema_Mobile/test/coming_soon_test.dart
MOB-FR-MOV-04 ← FR-MOV-04	MOB-BR-26	UC-MOB-02	MOB-SCR-HOME	MOB-API-025, MOB-API-027	MOB-FR-MOV-04-AC-01, MOB-FR-MOV-04-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/MovieReviewTests.cs; Cinema_Mobile/test/age_rating_test.dart; Cinema_Mobile/test/coming_soon_test.dart

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-MOV-08 ← FR-MOV-08	MOB-BR-26	UC-MOB-02	MOB-SCR-MOVIE	MOB-API-024	MOB-FR-MOV-08-AC-01, MOB-FR-MOV-08-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/MovieReviewTests.cs; Cinema_Mobile/test/age_rating_test.dart; Cinema_Mobile/test/coming_soon_test.dart
MOB-FR-MOV-09 ← FR-MOV-09	MOB-BR-26	UC-MOB-02	MOB-SCR-MOVIE, MOB-SCR-SUMMARY, MOB-SCR-CHECKIN	MOB-API-024	MOB-FR-MOV-09-AC-01, MOB-FR-MOV-09-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/MovieReviewTests.cs; Cinema_Mobile/test/age_rating_test.dart; Cinema_Mobile/test/coming_soon_test.dart
MOB-FR-MOV-10 ← FR-MOV-10	MOB-BR-26	UC-MOB-02	MOB-SCR-HOME	MOB-API-025, MOB-API-026	MOB-FR-MOV-10-AC-01, MOB-FR-MOV-10-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/MovieReviewTests.cs; Cinema_Mobile/test/age_rating_test.dart; Cinema_Mobile/test/coming_soon_test.dart
MOB-FR-MOV-11 ← FR-MOV-11	MOB-BR-26	UC-MOB-02	MOB-SCR-MOVIE	MOB-API-024	MOB-FR-MOV-11-AC-01, MOB-FR-MOV-11-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/MovieReviewTests.cs; Cinema_Mobile/test/age_rating_test.dart; Cinema_Mobile/test/coming_soon_test.dart

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-MOV-12 ← FR-MOV-12	MOB-BR-26	UC-MOB-02	MOB-SCR-MOVIE, MOB-SCR-TICKET	MOB-API-039, MOB-API-040, MOB-API-041, MOB-API-042	MOB-FR-MOV-12-AC-01, MOB-FR-MOV-12-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/MovieReviewTests.cs; Cinema_Mobile/test/age_rating_test.dart; Cinema_Mobile/test/coming_soon_test.dart
MOB-FR-SEAT-03 ← FR-SEAT-03	MOB-BR-04, MOB-BR-07, MOB-BR-15, MOB-BR-16, MOB-BR-26	UC-MOB-03	MOB-SCR-SEATS	MOB-API-001	MOB-FR-SEAT-03-AC-01, MOB-FR-SEAT-03-AC-02; Cinema_BE/MovieService/MovieService.Tests/Integration/SeatTests.cs; Cinema_Mobile/test/booking_flow_test.dart; Cinema_Mobile/test/seat_map_test.dart
MOB-FR-SHOW-03 ← FR-SHOW-03	MOB-BR-05, MOB-BR-06, MOB-BR-15, MOB-BR-19, MOB-BR-30	UC-MOB-04	MOB-SCR-MOVIE, MOB-SCR-SEATS	MOB-API-002	MOB-FR-SHOW-03-AC-01, MOB-FR-SHOW-03-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_BE/MovieService/MovieService.Tests/Integration/ShowtimeFormatTests.cs; Cinema_BE/MovieService/MovieService.Tests/Integration/ShowtimeTests.cs

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-SHOW-06 ← FR-SHOW-06	MOB-BR-05, MOB-BR-06, MOB-BR-15, MOB-BR-19, MOB-BR-30	UC-MOB-04	MOB-SCR-MOVIE	MOB-API-003	MOB-FR-SHOW-06-AC-01, MOB-FR-SHOW-06-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_BE/MovieService/MovieService.Tests/Integration/ShowtimeFormatTests.cs; Cinema_BE/MovieService/MovieService.Tests/Integration/ShowtimeTests.cs
MOB-FR-SHOW-07 ← FR-SHOW-07	MOB-BR-05, MOB-BR-06, MOB-BR-15, MOB-BR-19, MOB-BR-30	UC-MOB-04	MOB-SCR-MOVIE	MOB-API-003	MOB-FR-SHOW-07-AC-01, MOB-FR-SHOW-07-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_BE/MovieService/MovieService.Tests/Integration/ShowtimeFormatTests.cs; Cinema_BE/MovieService/MovieService.Tests/Integration/ShowtimeTests.cs
MOB-FR-SHOW-08 ← FR-SHOW-08	MOB-BR-05, MOB-BR-06, MOB-BR-15, MOB-BR-19, MOB-BR-30	UC-MOB-04	MOB-SCR-MOVIE	MOB-API-003	MOB-FR-SHOW-08-AC-01, MOB-FR-SHOW-08-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_BE/MovieService/MovieService.Tests/Integration/ShowtimeFormatTests.cs; Cinema_BE/MovieService/MovieService.Tests/Integration/ShowtimeTests.cs

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-SHOW-13 ← FR-SHOW-13	MOB-BR-05, MOB-BR-06, MOB-BR-15, MOB-BR-19, MOB-BR-30	UC-MOB-04	MOB-SCR-MOVIE, MOB-SCR-SEATS	MOB-API-002	MOB-FR-SHOW-13-AC-01, MOB-FR-SHOW-13-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_BE/MovieService/MovieService.Tests/Integration/ShowtimeFormatTests.cs; Cinema_BE/MovieService/MovieService.Tests/Integration/ShowtimeTests.cs
MOB-FR-BOOK-01 ← FR-BOOK-01	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-SUMMARY	MOB-API-021	MOB-FR-BOOK-01-AC-01, MOB-FR-BOOK-01-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs
MOB-FR-BOOK-02 ← FR-BOOK-02	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-RESULT, MOB-SCR-TICKET	MOB-API-017	MOB-FR-BOOK-02-AC-01, MOB-FR-BOOK-02-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-BOOK-04 ← FR-BOOK-04	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-TICKETS	MOB-API-020	MOB-FR-BOOK-04-AC-01, MOB-FR-BOOK-04-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs
MOB-FR-BOOK-05 ← FR-BOOK-05	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-PAYMENT, MOB-SCR-RESULT, MOB-SCR-TICKET	MOB-API-034, MOB-API-036	MOB-FR-BOOK-05-AC-01, MOB-FR-BOOK-05-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs
MOB-FR-BOOK-06 ← FR-BOOK-06	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-TICKET	MOB-API-015	MOB-FR-BOOK-06-AC-01, MOB-FR-BOOK-06-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-BOOK-07 ← FR-BOOK-07	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-SEATS	MOB-API-019	MOB-FR-BOOK-07-AC-01, MOB-FR-BOOK-07-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs
MOB-FR-BOOK-08 ← FR-BOOK-08	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-SEATS	MOB-API-021	MOB-FR-BOOK-08-AC-01, MOB-FR-BOOK-08-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs
MOB-FR-BOOK-09 ← FR-BOOK-09	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-SEATS, MOB-SCR-SUMMARY	MOB-API-021	MOB-FR-BOOK-09-AC-01, MOB-FR-BOOK-09-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-BOOK-10 ← FR-BOOK-10	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-PAYMENT	MOB-API-017	MOB-FR-BOOK-10-AC-01, MOB-FR-BOOK-10-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs
MOB-FR-BOOK-11 ← FR-BOOK-11	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-SEATS	MOB-API-001, MOB-API-019	MOB-FR-BOOK-11-AC-01, MOB-FR-BOOK-11-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs
MOB-FR-BOOK-13 ← FR-BOOK-13	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-SUMMARY, MOB-SCR-TICKET	MOB-API-021, MOB-API-022	MOB-FR-BOOK-13-AC-01, MOB-FR-BOOK-13-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-BOOK-15 ← FR-BOOK-15	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-SUMMARY, MOB-SCR-TICKET	MOB-API-021, MOB-API-038	MOB-FR-BOOK-15-AC-01, MOB-FR-BOOK-15-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs
MOB-FR-BOOK-17 ← FR-BOOK-17	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-SUMMARY, MOB-SCR-TICKET, MOB-SCR-PROFILE	MOB-API-021, MOB-API-023	MOB-FR-BOOK-17-AC-01, MOB-FR-BOOK-17-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs
MOB-FR-BOOK-19 ← FR-BOOK-19	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05	MOB-SCR-TICKET	MOB-API-002	MOB-FR-BOOK-19-AC-01, MOB-FR-BOOK-19-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-PAY-01 ← FR-PAY-01	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20	UC-MOB-06	MOB-SCR-PAYMENT	MOB-API-035	MOB-FR-PAY-01-AC-01, MOB-FR-PAY-01-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PaymentSummaryTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PayOsOrderCodeTests.cs
MOB-FR-PAY-02 ← FR-PAY-02	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20	UC-MOB-06	MOB-SCR-PAYMENT, MOB-SCR-RESULT	MOB-API-034	MOB-FR-PAY-02-AC-01, MOB-FR-PAY-02-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PaymentSummaryTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PayOsOrderCodeTests.cs

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-PAY-03 ← FR-PAY-03	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20	UC-MOB-06	MOB-SCR-TICKET, MOB-SCR-PAYHISTORY	MOB-API-037	MOB-FR-PAY-03-AC-01, MOB-FR-PAY-03-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PaymentSummaryTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PayOsOrderCodeTests.cs
MOB-FR-PAY-04 ← FR-PAY-04	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20	UC-MOB-06	MOB-SCR-PAYHISTORY	MOB-API-037	MOB-FR-PAY-04-AC-01, MOB-FR-PAY-04-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PaymentSummaryTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PayOsOrderCodeTests.cs

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-PAY-06 ← FR-PAY-06	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20	UC-MOB-06	MOB-SCR-TICKET	MOB-API-015, MOB-API-033	MOB-FR-PAY-06-AC-01, MOB-FR-PAY-06-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PaymentSummaryTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PayOsOrderCodeTests.cs
MOB-FR-PAY-07 ← FR-PAY-07	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20	UC-MOB-06		MOB-API-036	MOB-FR-PAY-07-AC-01, MOB-FR-PAY-07-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PaymentSummaryTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PayOsOrderCodeTests.cs

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-PAY-08 ← FR-PAY-08	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20	UC-MOB-06	MOB-SCR-RESULT	MOB-API-034, MOB-API-036	MOB-FR-PAY-08-AC-01, MOB-FR-PAY-08-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PaymentSummaryTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PayOsOrderCodeTests.cs
MOB-FR-PAY-11 ← FR-PAY-11	MOB-BR-09, MOB-BR-10, MOB-BR-11, MOB-BR-12, MOB-BR-13, MOB-BR-14, MOB-BR-20	UC-MOB-06	MOB-SCR-PAYMENT	MOB-API-017, MOB-API-034	MOB-FR-PAY-11-AC-01, MOB-FR-PAY-11-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PaymentSummaryTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/PayOsOrderCodeTests.cs
MOB-FR-REF-01 ← FR-REF-01	MOB-BR-17, MOB-BR-18, MOB-BR-19, MOB-BR-20	UC-MOB-07	MOB-SCR-TICKET	MOB-API-015, MOB-API-036	MOB-FR-REF-01-AC-01, MOB-FR-REF-01-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/RefundTests.cs; Cinema_Mobile/test/booking_details_ui_test.dart; Cinema_Mobile/test/refund_details_ui_test.dart

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-REF-05 ← FR-REF-05	MOB-BR-17, MOB-BR-18, MOB-BR-19, MOB-BR-20	UC-MOB-07	MOB-SCR-TICKET	MOB-API-015	MOB-FR-REF-05-AC-01, MOB-FR-REF-05-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/RefundTests.cs; Cinema_Mobile/test/booking_details_ui_test.dart; Cinema_Mobile/test/refund_details_ui_test.dart
MOB-FR-REF-06 ← FR-REF-06	MOB-BR-17, MOB-BR-18, MOB-BR-19, MOB-BR-20	UC-MOB-07	MOB-SCR-TICKET	MOB-API-033	MOB-FR-REF-06-AC-01, MOB-FR-REF-06-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/RefundTests.cs; Cinema_Mobile/test/booking_details_ui_test.dart; Cinema_Mobile/test/refund_details_ui_test.dart
MOB-FR-REF-07 ← FR-REF-07	MOB-BR-17, MOB-BR-18, MOB-BR-19, MOB-BR-20	UC-MOB-07		MOB-API-015, MOB-API-033	MOB-FR-REF-07-AC-01, MOB-FR-REF-07-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/RefundTests.cs; Cinema_Mobile/test/booking_details_ui_test.dart; Cinema_Mobile/test/refund_details_ui_test.dart

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-CHK-01 ← FR-CHK-01	MOB-BR-21	UC-MOB-08	MOB-SCR-TICKET	MOB-API-017	MOB-FR-CHK-01-AC-01, MOB-FR-CHK-01-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/CheckInTests.cs; Cinema_Mobile/test/check_in_test.dart
MOB-FR-CHK-02 ← FR-CHK-02	MOB-BR-21	UC-MOB-08	MOB-SCR-CHECKIN	MOB-API-018	MOB-FR-CHK-02-AC-01, MOB-FR-CHK-02-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/CheckInTests.cs; Cinema_Mobile/test/check_in_test.dart
MOB-FR-CHK-03 ← FR-CHK-03	MOB-BR-21	UC-MOB-08	MOB-SCR-CHECKIN	MOB-API-016	MOB-FR-CHK-03-AC-01, MOB-FR-CHK-03-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/CheckInTests.cs; Cinema_Mobile/test/check_in_test.dart
MOB-FR-CHK-04 ← FR-CHK-04	MOB-BR-21	UC-MOB-08	MOB-SCR-MGT-LANDING, MOB-SCR-CHECKIN	MOB-API-016, MOB-API-018	MOB-FR-CHK-04-AC-01, MOB-FR-CHK-04-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/CheckInTests.cs; Cinema_Mobile/test/check_in_test.dart

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-CHK-05 ← FR-CHK-05	MOB-BR-21	UC-MOB-08	MOB-SCR-TICKET, MOB-SCR-CHECKIN	MOB-API-017, MOB-API-018	MOB-FR-CHK-05-AC-01, MOB-FR-CHK-05-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/CheckInTests.cs; Cinema_Mobile/test/check_in_test.dart
MOB-FR-NOTI-01 ← FR-NOTI-01	MOB-BR-11, MOB-BR-13	UC-MOB-09	MOB-SCR-NOTIFICATIONS	MOB-API-032	MOB-FR-NOTI-01-AC-01, MOB-FR-NOTI-01-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_Mobile/test/notification_read_test.dart
MOB-FR-NOTI-04 ← FR-NOTI-04	MOB-BR-11, MOB-BR-13	UC-MOB-09	MOB-SCR-NOTIFICATION	MOB-API-029	MOB-FR-NOTI-04-AC-01, MOB-FR-NOTI-04-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_Mobile/test/notification_read_test.dart
MOB-FR-NOTI-06 ← FR-NOTI-06	MOB-BR-11, MOB-BR-13	UC-MOB-09		MOB-API-032	MOB-FR-NOTI-06-AC-01, MOB-FR-NOTI-06-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_Mobile/test/notification_read_test.dart

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-NOTI-07 ← FR-NOTI-07	MOB-BR-11, MOB-BR-13	UC-MOB-09		MOB-API-005, MOB-API-011, MOB-API-014	MOB-FR-NOTI-07-AC-01, MOB-FR-NOTI-07-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_Mobile/test/notification_read_test.dart
MOB-FR-NOTI-08 ← FR-NOTI-08	MOB-BR-11, MOB-BR-13	UC-MOB-09		MOB-API-032	MOB-FR-NOTI-08-AC-01, MOB-FR-NOTI-08-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_Mobile/test/notification_read_test.dart
MOB-FR-NOTI-09 ← FR-NOTI-09	MOB-BR-11, MOB-BR-13	UC-MOB-09	MOB-SCR-HOME, MOB-SCR-NOTIFICATIONS, MOB-SCR-NOTIFICATION	MOB-API-028, MOB-API-030, MOB-API-031	MOB-FR-NOTI-09-AC-01, MOB-FR-NOTI-09-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/ShowtimeReminderTests.cs; Cinema_Mobile/test/notification_read_test.dart
MOB-FR-NAV-01 ← Code-derived	MOB-BR-01, MOB-BR-02, MOB-BR-03, MOB-BR-24, MOB-BR-25	UC-MOB-01	MOB-SCR-LOGIN	—	MOB-FR-NAV-01-AC-01, MOB-FR-NAV-01-AC-02; Cinema_BE/AuthService/AuthService.Tests/Integration/AccountValidationTests.cs; Cinema_BE/AuthService/AuthService.Tests/Integration/AuthFlowTests.cs; Cinema_Mobile/test/account_lock_flow_test.dart

Continued on next page

Table 116 — continued

Requirement / source	Business rules	Use case	Screens	API	AC / test evidence
MOB-FR-STATE-01 ← Code-derived	MOB-BR-07, MOB-BR-08, MOB-BR-14, MOB-BR-15, MOB-BR-16, MOB-BR-17, MOB-BR-27, MOB-BR-28, MOB-BR-29	UC-MOB-05		—	MOB-FR-STATE-01-AC-01, MOB-FR-STATE-01-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/BookingFlowTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/ConcessionTests.cs; Cinema_BE/BookingService/BookingService.Tests/Integration/LoyaltyTests.cs
MOB-FR-MEDIA-01 ← Code-derived	MOB-BR-26	UC-MOB-02	MOB-SCR-HOME	MOB-API-024	MOB-FR-MEDIA-01-AC-01, MOB-FR-MEDIA-01-AC-02; Cinema_BE/BookingService/BookingService.Tests/Integration/MovieReviewTests.cs; Cinema_Mobile/test/age_rating_test.dart; Cinema_Mobile/test/coming_soon_test.dart

23. Constraints

Table 117: 23. Constraints

ID	Constraint / open item
MOB-CON-01	The .NET implementation is an API backend. A separate MVC/Razor/Blazor course UI is [TO BE CONFIRMED].
MOB-CON-02	DB submodule absent/uninitialized; exact SQL DDL/seed and live migration state [TO BE CONFIRMED]. Local EF/migrations are the available evidence.
MOB-CON-03	Live merchant keys, public webhook, SMTP and deployment URLs are environment dependencies. No financial transaction or email was initiated for documentation.
MOB-CON-04	Physical OS/device acceptance, signing/store release, SLA, backup retention and load targets [TO BE CONFIRMED]. Platform scaffolding is not a compatibility certificate.
MOB-CON-05	Android emulator gateway defaults to http://10.0.2.2:5000; other local targets http://localhost:5000. Phones need reachable API_BASE_URL; ADMIN_URL/CUSTOMER_URL/PAYMENT_RETURN_URL depend on deployment.

24. Out of Scope

Exclude phone `CRUD` for movies, rooms, seat layouts, showtimes, accounts, snack menu, promotions or members; counter/cash sales, receipt print, operational/revenue dashboards and refund approve/reject/transfer recording. Those Dart screens are web-only despite sharing the repository.

No implemented `FCM` push or automatic bank-refund integration is claimed. Payroll, inventory and `AI` recommendations are not added. Existing concessions, promotions/voucher codes, membership/loyalty and reviews remain in scope where actually exposed by code.

25. Future Enhancements

Table 118: 25. Future Enhancements

ID	Future work, not a current FR	Reason
MOB-FUT-01	Load more owned-ticket pages	Only first 50 currently returned to UI
MOB-FUT-02	FCM push if approved	Current notifications are email/API history
MOB-FUT-03	Automated refund if a suitable provider is chosen	Current flow records an external manual transfer

26. Conclusion

This specification allocates one system to the course/client boundary, preserves language parity and makes `API`-only functions, shared dependencies and unverified gaps explicit. It supports test-case design and explanation of the actual implementation.

27. Implemented vs Requirement

Status vocabulary: Implemented; Partially Implemented; UI Only; Mock Data; Backend Pending; Not Implemented; To Be Confirmed. Only applicable statuses are used. Production dependencies use AppDependencies.remote; test fakes are not production mock data.

Table 119: 27. Implemented vs Requirement

Feature	SRS Requirement	Code Status	API Status	Notes
Register	MOB-FR-AUTH-01	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Sign in	MOB-FR-AUTH-02	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
JWT claims	MOB-FR-AUTH-03	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Validate JWT	MOB-FR-AUTH-04	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Authorize roles	MOB-FR-AUTH-05	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Unauthenticated and forbidden responses	MOB-FR-AUTH-06	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Verify email OTP	MOB-FR-AUTH-07	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Repeat unverified registration	MOB-FR-AUTH-08	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Recover password	MOB-FR-AUTH-09	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Refresh and sign out	MOB-FR-AUTH-10	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Current account	MOB-FR-AUTH-11	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Lock and role-change enforcement	MOB-FR-AUTH-12	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Account field validation	MOB-FR-AUTH-13	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Authentication rate limits	MOB-FR-AUTH-14	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Edit profile	MOB-FR-AUTH-16	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.

Continued on next page

Table 119 — continued

Feature	SRS Requirement	Code Status	API Status	Notes
Change current password	MOB-FR-AUTH-17	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Movie details	MOB-FR-MOV-02	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Title search	MOB-FR-MOV-03	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Active catalogue	MOB-FR-MOV-04	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Open trailer	MOB-FR-MOV-08	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Age rating	MOB-FR-MOV-09	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Now showing and coming soon	MOB-FR-MOV-10	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Director and cast	MOB-FR-MOV-11	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Movie reviews	MOB-FR-MOV-12	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Read room map	MOB-FR-SEAT-03	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Showtime and prices	MOB-FR-SHOW-03	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Open future showtimes	MOB-FR-SHOW-06	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Showtimes by movie	MOB-FR-SHOW-07	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Bookable movie showtimes	MOB-FR-SHOW-08	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Screening format	MOB-FR-SHOW-13	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Hold booking	MOB-FR-BOOK-01	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Booking details	MOB-FR-BOOK-02	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.

Continued on next page

Table 119 — continued

Feature	SRS Requirement	Code Status	API Status	Notes
Own ticket history	MOB-FR-BOOK-04	Partially Implemented	Controller exists; runtime unverified	Only the first 50 tickets are loaded; older tickets are missing from the UI.
Confirm paid booking	MOB-FR-BOOK-05	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Cancel booking	MOB-FR-BOOK-06	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Occupied seat IDs	MOB-FR-BOOK-07	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Concurrent seat conflict	MOB-FR-BOOK-08	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Sweetbox pair	MOB-FR-BOOK-09	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Expire unpaid hold	MOB-FR-BOOK-10	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Refresh customer map	MOB-FR-BOOK-11	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Concessions with tickets	MOB-FR-BOOK-13	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Promotion and voucher	MOB-FR-BOOK-15	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Membership points	MOB-FR-BOOK-17	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Share and calendar	MOB-FR-BOOK-19	Partially Implemented	Controller exists; runtime unverified	Calendar uses actual show end or a 150-minute fallback; sharing falls back to clipboard.
Create/reuse PayOS link	MOB-FR-PAY-01	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Verify PayOS	MOB-FR-PAY-02	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Payment details	MOB-FR-PAY-03	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Own payment history	MOB-FR-PAY-04	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Refund access	MOB-FR-PAY-06	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.

Continued on next page

Table 119 — continued

Feature	SRS Requirement	Code Status	API Status	Notes
Signed webhook	MOB-FR-PAY-07	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Late payment	MOB-FR-PAY-08	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Automatic payment polling	MOB-FR-PAY-11	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Create refund request	MOB-FR-REF-01	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Reopen rejected refund	MOB-FR-REF-05	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Own refund status	MOB-FR-REF-06	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Unique refund per payment	MOB-FR-REF-07	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Ticket QR	MOB-FR-CHK-01	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Lookup ticket code	MOB-FR-CHK-02	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Check-in validation	MOB-FR-CHK-03	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Camera/manual scanner	MOB-FR-CHK-04	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Display check-in time	MOB-FR-CHK-05	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Notification history	MOB-FR-NOTI-01	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Notification details	MOB-FR-NOTI-04	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Event emails	MOB-FR-NOTI-06	Partially Implemented	Controller exists; runtime unverified	Event deduplication exists; SMTP/DB crash window prevents absolute exactly-once delivery.
OTP and welcome email	MOB-FR-NOTI-07	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Showtime reminder	MOB-FR-NOTI-08	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.

Continued on next page

Table 119 — continued

Feature	SRS Requirement	Code Status	API Status	Notes
Read/unread notifications	MOB-FR-NOTI-09	Implemented	Controller exists; runtime unverified	Static implementation evidence; runtime acceptance not executed.
Route guards and recovery	MOB-FR-NAV-01	Implemented	Client/platform; no dedicated API	Static implementation evidence; runtime acceptance not executed.
Loading, error and empty states	MOB-FR-STATE-01	Partially Implemented	Client/platform; no dedicated API	State widgets exist; exhaustive device/error-path verification was not run.
Platform-specific movie media	MOB-FR-MEDIA-01	Partially Implemented	Controller exists; runtime unverified	Native trailer opens externally; web preview has an empty native stub.

Table 120: 27. Implemented vs Requirement

Feature	SRS Requirement	Code Status	API Status	Notes
.NET-rendered UI	MOB-CON-01	Not Implemented	Existing ASP.NET API	Flutter Web UI only; course necessity [TO BE CONFIRMED]
FCM push	MOB-FUT-02	Not Implemented	—	Future work, not accepted current FR
DB submodule SQL verification	MOB-CON-02	To Be Confirmed	Local EF mappings exist	Submodule uninitialized; no live DB check

28. Evidence and source reconciliation

Table 121: 28. Evidence and source reconciliation

Evidence	Use / reconciliation
docs/SRS_EN.md; docs/SRS_VI.md	125 legacy FR rows per language, scoped with suffix IDs; NAV/STATE/MEDIA refine code behaviour
Cinema_BE/Cinema_Microservice_SRS_Complete.md	Historical six-service intent; current three services take precedence
README.md; Cinema_Mobile/README.md; DEMO.md	Actual entry points/setup, no separate .NET UI
Cinema_Mobile/lib/app/routes/app_router.dart; Cinema_Mobile/lib/admin/admin_router.dart	Channel routes/role guards; native management only check-in
Cinema_Mobile/lib/app/app_dependencies.dart; Cinema_Mobile/lib/core/network/api_client.dart	Remote dependencies, 15-second timeout, refresh and public-session probe
Cinema_BE/ApiGateway/ApiGateway/appsettings.json	Actual route policies/rates/health; old prose route count is stale
.github/workflows/ci.yml; .github/workflows/flutter.yml	Declared CI; Flutter pin 3.47.4 supersedes older environment prose; old test counts are unverified current totals
Cinema_Mobile/lib/core/widgets/trailer_preview_stub.dart; Cinema_Mobile/lib/core/widgets/trailer_player_native.dart	Native preview stub is empty; full trailer opens externally
Cinema_Mobile/lib/features/booking/widgets/ticket_share_actions.dart	Calendar end fallback 150 minutes, not unconditional exact show end
Cinema_BE/Booking Service/Booking Service/Services/Implementations/NotificationService.cs	Event deduplication and non-atomic SMTP/DB delivery; no absolute exactly-once claim

Review inventory: 763 files enumerated; 594 relevant text/code/configuration files indexed, including approximately 64,397 C#/Dart lines. Generated build/cache/vendor folders, binary media, encrypted secret payloads and missing DB-submodule contents are excluded from functional evidence. No secret values are reproduced.